

MATERIAL DOCENTE EN ABIERTO

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel · Natalia Madrueño Sierro · Rubén Rodríguez Fernández



Universidad Rey Juan Carlos · BURJC Digital

Información de publicación

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández. Algunos derechos reservados. El contenido documental original se distribuye bajo CC BY-SA 4.0; el código se distribuye bajo MIT.

Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa. Las imágenes sin licencia de reutilización verificada se han retirado y sustituido por esquemas originales.

Declaración sobre herramientas de inteligencia artificial generativa

En la guía y la documentación editorial se utilizaron herramientas generativas de texto como apoyo parcial para organizar información aportada por el equipo docente, proponer redacciones iniciales de textos de enlace, normalizar el repositorio y comprobar coherencia. No sustituyeron el diseño docente, la selección de contenidos técnicos ni la autoría de presentaciones y ejercicios. No se conservan imágenes generadas por IA en esta edición.

Todo el contenido fue contrastado con la guía docente y los materiales fuente, revisado, corregido, validado y aprobado por el equipo autor, que asume plena responsabilidad sobre la exactitud, originalidad, integridad académica, licencias y versión final.

Índice general

Sección	Contenido	Página
Categoría 0	Guía de estudio	4
Categoría 1	Apuntes de la asignatura	24
Categoría 2	Presentaciones de los temas 1 a 6	47
Categoría 3	Prácticas, ejercicios y proyecto final	477
Categoría 6	Descripción de programas de ordenador	511

Los editables y el código se facilitan como ficheros separados del mismo depósito. La descripción de la categoría 6 debe completarse con el SWHID verificado antes del envío.

MATERIAL DOCENTE EN ABIERTO

Big Data Processing II

Guía de estudio

Máster Universitario en Análisis de Datos Deportivos

Curso 2026–2027 · 3 ECTS · segundo semestre

Equipo docente

Alberto Fernández Isabel · profesor responsable

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados.

Licencia: [Creative Commons Atribución–CompartirIgual 4.0 Internacional \(CC BY-SA 4.0\)](https://creativecommons.org/licenses/by-sa/4.0/)

Depósito institucional: [BURJC Digital](https://burjc-digital.com/)



CC BY-SA 4.0
Atribución-CompartirIgual

Información de publicación

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución–CompartirIgual 4.0 Internacional, disponible en <https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Declaración sobre herramientas de inteligencia artificial generativa

En la preparación de esta guía se emplearon herramientas generativas de texto como apoyo para la organización editorial, la redacción inicial de determinados textos de enlace y la comprobación de coherencia. No se utilizaron para sustituir el diseño docente, los contenidos técnicos de las presentaciones ni la autoría de los ejercicios. Todo el contenido ha sido revisado, editado, validado y aprobado por el equipo autor, que asume plena responsabilidad sobre la versión final.

1. Finalidad y alcance

Esta guía organiza el trabajo recomendado para seguir Big Data Processing II mediante los materiales docentes del repositorio. Traduce el temario oficial en una secuencia de estudio que integra teoría, ejercicios, programas y trabajo personal, sin sustituir la guía docente ni las indicaciones del equipo docente.

La asignatura profundiza en el procesamiento distribuido y en tiempo real de datos deportivos, la optimización de pipelines y la integración responsable de inteligencia artificial generativa. Su enfoque es aplicado: cada bloque culmina en decisiones de diseño, implementación, evaluación o comunicación técnica.

Edición y estado de los materiales

Esta edición se basa en la guía docente oficial publicada el 7 de julio de 2026 y en los materiales disponibles en el repositorio a 30 de septiembre de 2026. Las presentaciones de los seis temas están incorporadas. Las fuentes LaTeX de los temas 5 y 6 son reconstrucciones documentadas a partir de sus PDF, no los originales de autor.

2. Mapa general de materiales

Tema	Bloque	Teoría	Práctica / aplicación
1	Procesamiento distribuido	Presentación · 66 diapositivas	Notebook de streaming y solución
2	Streaming en tiempo real	Kafka · 44; Spark · 60 diapositivas	2 enunciados; Kafka, Docker, PySpark y datos
3	Optimización y escalabilidad	Presentación · 36 diapositivas	Análisis y diseño de pipelines
4	IA generativa	3 presentaciones · 120 diapositivas	3 hojas de ejercicios; RAG y agentes
5	Proyectos integrados	Presentación · 46 diapositivas	Ejercicio Text-to-SQL con Spark y LangGraph
6	Tendencias y futuro	Presentación · 40 diapositivas	Análisis crítico, ético y legal

3. Resultados de aprendizaje y método de uso

La guía docente vincula la asignatura con resultados de aprendizaje relacionados con el trabajo interdisciplinar, el autoaprendizaje, la comunicación técnica, la implantación y optimización de tecnologías Big Data, la automatización y la adaptación de tendencias como el análisis en tiempo real y la inteligencia artificial.

Resultados que vertebran el itinerario

Resultados	Eje	Evidencia esperada
COM1 · COM4	Colaborar y comunicar	Trabajo en equipo, memoria técnica y exposición oral.
COM3	Aprender de forma autónoma	Contrastar documentación y fuentes científicas actuales.
COM6 · COM7	Construir soluciones Big Data	Ingesta, procesamiento, almacenamiento y optimización.
COM11 · COM12	Automatizar y adaptar tendencias	Streaming, IA generativa, RAG y agentes.
CON4 · CON8	Seleccionar tecnologías	Comparar plataformas, arquitecturas y metodologías a escala.
HAB1 · HAB4	Resolver problemas deportivos	Diseñar soluciones aplicadas, innovadoras y justificadas.

Secuencia recomendada para cada bloque

1. Leer los objetivos del tema y activar los conocimientos previos necesarios.
2. Estudiar la presentación o material teórico, anotando conceptos, decisiones y dudas.
3. Ejecutar o inspeccionar los ejemplos de código antes de modificar sus parámetros o entradas.
4. Resolver la actividad práctica y justificar las decisiones adoptadas con evidencias reproducibles.
5. Sintetizar lo aprendido en un esquema, informe breve o explicación técnica dirigida a un stakeholder.
6. Comprobar la lista de autoevaluación y registrar los aspectos que requieren tutoría o repaso.

Integridad académica y uso de IA

En actividades evaluables, las herramientas basadas en inteligencia artificial solo pueden utilizarse cuando el enunciado o el profesorado lo autoricen expresamente y dentro de los usos indicados. En el estudio autónomo deben emplearse de forma ética, legal, honesta y transparente.

Tema 1. Procesamiento de Big Data en entornos distribuidos

Bloque introductorio que conecta el análisis deportivo en vivo con la evolución desde el procesamiento batch hacia arquitecturas distribuidas y orientadas a eventos.

Objetivos de trabajo

- Comprender la transición de batch a streaming en analítica deportiva.
- Distinguir mensajería distribuida, motores de procesamiento y almacenamiento.
- Relacionar arquitecturas Lambda y Kappa con requisitos de latencia y consistencia.
- Justificar decisiones de diseño para casos de uso deportivos en vivo.

Materiales

- BDP11_Tema_01_Procesamiento_Big_Data_Distribuido.pdf.
- streaming.ipynb y streaming_soluciones.ipynb.
- Bibliografía básica de la guía docente: Gama (2010) y Marz & Warren (2015).

Plan de trabajo recomendado

1. Revisar la evolución del análisis deportivo y distinguir dato histórico, microbatch y streaming.
2. Construir un mapa de fuentes, broker, procesamiento y destinos para un caso deportivo.
3. Comparar arquitecturas Lambda y Kappa atendiendo a complejidad, latencia y reprocesamiento.
4. Ejecutar o inspeccionar el notebook y contrastar el procesamiento directo con el incremental.
5. Redactar una decisión de arquitectura indicando requisitos, supuestos y riesgos.

Trabajo personal recomendado

Proponer una arquitectura de alto nivel para monitorizar carga o fatiga durante una competición y explicar por qué la información pierde valor si llega tarde.

Tema 2. Procesamiento de flujos de datos en tiempo real

Bloque aplicado a la ingestión y al procesamiento continuo con Apache Kafka y Spark Structured Streaming, utilizando telemetría deportiva simulada.

Objetivos de trabajo

- Explicar productores, consumidores, topics, particiones, offsets y grupos de consumo en Kafka.
- Diseñar un flujo tolerante a fallos y escalable para eventos deportivos.
- Aplicar Structured Streaming con DataFrames, ventanas y tiempo de evento.
- Razonar sobre latencia, estado, watermarks y modos de salida.

Materiales

- BDPPII_Tema_02_Apache_Kafka.pdf.
- BDPPII_Tema_02_Spark_Structured_Streaming.pdf.
- Enunciados de Apache Kafka y Spark Structured Streaming.
- producer.py, sensores_deportivos_fatiga.csv, docker-compose.yml y spark_streaming.ipynb.

Plan de trabajo recomendado

1. Estudiar Kafka y representar el recorrido de un evento desde el productor hasta sus consumidores.
2. Resolver el enunciado de Kafka utilizando la descripción del conjunto de datos de sensores.
3. Estudiar Structured Streaming y distinguir tiempo de proceso de tiempo de evento.
4. Levantar o inspeccionar el entorno Docker y completar el notebook de streaming.
5. Probar ventanas, agregaciones y tolerancia a datos tardíos; documentar las salidas observadas.
6. Comparar los resultados con los requisitos del caso y proponer ajustes de particionado y recursos.

Trabajo personal recomendado

Entregar una traza reproducible de un flujo Kafka–Spark y explicar cómo se preservan orden, escalabilidad y utilidad temporal de la información.

Tema 3. Optimización y escalabilidad de pipelines deportivos

Bloque centrado en diagnosticar cuellos de botella y convertir un prototipo funcional en un pipeline eficiente, observable y desplegable.

Objetivos de trabajo

- Identificar costes de particionado, shuffle, serialización, estado y almacenamiento.
- Aplicar paralelización, caching y elección de particiones con criterio.
- Utilizar métricas y herramientas de monitorización para formular hipótesis.
- Comparar alternativas de despliegue on-premises y cloud.

Materiales

- BDPII_Tema_03_Optimizacion_Escalabilidad.pdf.
- Pipelines y ejercicios contruidos en los temas 1 y 2.
- Bibliografía básica: Karau, Polak & Warren (2026), High Performance Spark.

Plan de trabajo recomendado

1. Representar el pipeline de ingestión, procesamiento, almacenamiento y consumo.
2. Localizar operaciones costosas y proponer métricas antes de modificar la configuración.
3. Evaluar particionado, skew, shuffles, caching y paralelismo con ejemplos concretos.
4. Definir un experimento controlado con línea base, cambio aislado y criterio de éxito.
5. Documentar el compromiso entre coste, latencia, rendimiento y mantenibilidad.

Trabajo personal recomendado

Preparar una ficha de diagnóstico con síntoma, evidencia, hipótesis, intervención y métrica de verificación para un pipeline deportivo en streaming.

Tema 4. Inteligencia Artificial Generativa aplicada a Big Data Deportivo

Bloque en tres sesiones: control del comportamiento mediante prompting y post-training; control de la información mediante RAG; y control del flujo y la acción mediante agentes, tools y arquitecturas cognitivas.

Objetivos de trabajo

- Explicar las capacidades y límites de un LLM en un sistema de datos deportivo.
- Diseñar prompts evaluables y reconocer cuándo el prompting no resuelve la falta de información.
- Construir conceptualmente un flujo RAG y evaluar recuperación y respuesta.
- Distinguir workflow, agente, skill y tool; controlar coste, seguridad y trazabilidad.

Materiales

- Sesión 1: Prompt Engineering y Post-training; ejercicios asociados.
- Sesión 2: Retrieval-Augmented Generation (RAG); ejercicios y RAG_1.py / RAG_1_comentado.py.
- Sesión 3: Sistemas basados en agentes; ejercicios y tres programas de workflows/agentes.
- requirements.txt y bibliografía básica: Hu et al. (2025), Hands-on LLM-based Agents.

Plan de trabajo recomendado

1. Separar generación lingüística, acceso a información, cálculo y ejecución de acciones.
2. Resolver los ejercicios de la sesión 1 y definir criterios de calidad para las respuestas.
3. Estudiar la arquitectura RAG; ejecutar o inspeccionar los ejemplos y analizar el efecto del chunking y la recuperación.
4. Estudiar agentes, estados, tools y patrones de orquestación; comparar flujo lineal y decisión autónoma.
5. Diseñar un caso deportivo indicando datos, controles humanos, registros, costes y riesgos.
6. Revisar privacidad, propiedad intelectual, sesgos y necesidad de validación experta.

Trabajo personal recomendado

Diseñar una solución que genere un informe deportivo trazable a partir de datos autorizados, separando claramente hechos recuperados, cálculos, inferencias y texto generado.

Tema 5. Proyectos integrados: del dato en tiempo real al análisis automatizado

Bloque de integración tecnológica sobre sistemas que combinan ingestión continua, cómputo en streaming, recuperación de información, agentes e interfaces conversacionales. El ejercicio Text-to-SQL aplica estos contenidos; la práctica final es una actividad evaluable transversal sobre el temario completo.

Objetivos de trabajo

- Comprender la transición desde el Business Intelligence retrospectivo hacia la analítica prescriptiva en tiempo real.
- Relacionar Kafka y Spark Structured Streaming dentro de una arquitectura completa de ingestión y procesamiento.
- Explicar cómo los flujos agénticos, las tools y el patrón ReAct conectan lenguaje natural con datos deportivos.
- Analizar el papel de RAG multimodal, la generación de narrativas y las interfaces conversacionales.

Materiales

- BDPII_Tema_05_Proyecto_Integrado.pdf · 46 diapositivas.
- Ejercicio Text-to-SQL Agent con LangGraph y Apache Spark: README, código inicial, generador de datos y dependencias.
- Materiales de los temas 1–4 como base conceptual y técnica.

Plan de trabajo recomendado

1. Estudiar el recorrido completo desde la telemetría hasta la decisión y representar los componentes del sistema.
2. Revisar particiones, semántica exactly-once, estado y watermarks en la integración Kafka-Spark.
3. Comparar workflows deterministas y agentes que consultan datos mediante tools.
4. Analizar cómo RAG multimodal y la generación de narrativas añaden contexto sin sustituir los cálculos deterministas.
5. Completar el ejercicio Text-to-SQL: generar los datos, registrar las vistas de Spark y construir el agente ReAct.
6. Validar el agente con consultas deportivas y comprobar que las respuestas se apoyan en los datos ejecutados.

Trabajo personal recomendado

Elaborar un diagrama razonado del sistema y una memoria breve del ejercicio Text-to-SQL que documente arquitectura, consultas probadas, resultados y límites del agente.

Tema 6. Tendencias y futuro del procesamiento masivo de datos en el deporte

Bloque final de análisis crítico sobre tecnologías emergentes, analítica predictiva y prescriptiva, y los retos éticos y legales del uso masivo de datos e inteligencia artificial.

Objetivos de trabajo

- Evaluar tendencias sin confundir novedad con adecuación al problema.
- Distinguir analítica descriptiva, predictiva y prescriptiva.
- Identificar impactos sobre privacidad, equidad, seguridad y responsabilidad.
- Formular recomendaciones técnicas justificadas con fuentes fiables y actuales.

Materiales

- BDPII_Tema_06_Tendencias_Futuro.pdf · 40 diapositivas.
- Guía docente oficial y bibliografía básica y complementaria.
- Documentación técnica oficial y literatura científica seleccionada por el equipo docente.

Plan de trabajo recomendado

1. Seleccionar una tendencia relevante para el análisis deportivo y delimitar el problema que pretende resolver.
2. Contrastar al menos una fuente técnica primaria y una referencia científica.
3. Analizar madurez, datos requeridos, coste, integración, explicabilidad y riesgos.
4. Distinguir predicción de recomendación y explicitar los supuestos de cualquier intervención prescriptiva.
5. Preparar una recomendación de adopción, piloto limitado o rechazo razonado.

Trabajo personal recomendado

Elaborar una nota de decisión breve para un responsable deportivo que incluya oportunidad, evidencia, riesgos ético-legales, salvaguardas y próximos pasos.

4. Planificación orientativa por unidades de trabajo

La guía docente sitúa las clases, los ejercicios, los trabajos colectivos y las tutorías entre las semanas 1 y 15. La siguiente secuencia distribuye el aprendizaje en quince unidades; cada unidad puede ocupar una o más sesiones según el calendario oficial del grupo.

Unidad	Foco	Actividad y evidencia
1	T1 · Del batch al streaming	Valor temporal del dato, arquitectura distribuida y caso deportivo.
2	T1 · Mensajería y arquitecturas	Broker, motores y comparación Lambda/Kappa; notebook de streaming.
3	T2 · Fundamentos de Kafka	Topics, particiones, productores, consumidores y contratos de eventos.
4	T2 · Kafka aplicado	Enunciado, productor y telemetría deportiva; escalabilidad y tolerancia a fallos.
5	T2 · Structured Streaming	DataFrames, ventanas, estado, watermarks y notebook PySpark.
6	T3 · Diagnóstico	Pipeline completo, métricas, particionado, shuffles y skew.
7	T3 · Producción	Caching, recursos, observabilidad y opciones on-premises/cloud.
8	T4 · Prompting	Capacidades y límites del LLM; especificación y evaluación de prompts.

Carga de trabajo

La guía docente establece 75 horas totales: 30 horas presenciales/relacionadas y 45 horas de preparación autónoma. Esta planificación debe ajustarse a las fechas y actividades concretas comunicadas por el profesorado.

4. Planificación orientativa por unidades de trabajo (continuación)

Unidad	Foco	Actividad y evidencia
9	T4 · RAG	Recuperación, chunking, vectores, trazabilidad y evaluación de respuestas.
10	T4 · Agentes	Estados, tools, workflows, costes, seguridad y supervisión humana.
11	T5 · Arquitectura integrada	Kafka, Spark, estado, RAG y agentes en un flujo de extremo a extremo.
12	T5 · Text-to-SQL	Datos sintéticos, vistas Spark SQL, tools y construcción del agente ReAct.
13	T5 · Narrativas e interfaces	RAG multimodal, validación de respuestas y visualización conversacional.
14	T6 · Tendencias	Fuentes fiables, madurez, integración, predicción y prescripción.
15	T6 · Ética, legalidad y síntesis	Privacidad, equidad, responsabilidad, límites y decisión final.

4.1 Calendario detallado por sesión

La secuencia siguiente concreta, para cada sesión orientativa, los contenidos, los materiales, los objetivos, la actividad prevista y el trabajo personal. Las fechas exactas se ajustarán al calendario oficial y a las indicaciones publicadas en Aula Virtual.

Sesión 1 Tema 1

Contenidos: Del procesamiento por lotes al streaming; valor temporal del dato.

Materiales: Presentación del tema 1, diapositivas 1–18.

Objetivos de aprendizaje: Distinguir batch, microbatch y streaming y relacionarlos con decisiones deportivas.

Actividad planificada: Mapa conceptual y discusión de un caso de monitorización en vivo.

Trabajo personal recomendado: Resumir requisitos de latencia, consistencia y disponibilidad del caso elegido.

Sesión 2 Tema 1

Contenidos: Mensajería, motores distribuidos y arquitecturas Lambda y Kappa.

Materiales: Presentación del tema 1, diapositivas 19–66; notebook streaming.ipynb.

Objetivos de aprendizaje: Comparar alternativas de arquitectura y justificar sus compromisos.

Actividad planificada: Inspección del notebook y diseño de una arquitectura de alto nivel.

Trabajo personal recomendado: Completar el notebook y redactar una decisión arquitectónica razonada.

Sesión 3 Tema 2

Contenidos: Fundamentos de Kafka: broker, topics, particiones y offsets.

Materiales: Presentación Apache Kafka; documentación oficial de Apache Kafka.

Objetivos de aprendizaje: Explicar el recorrido de un evento y la función de cada componente.

Actividad planificada: Representación del flujo productor-broker-consumidor.

Trabajo personal recomendado: Preparar un esquema de topics, claves y particiones para telemetría deportiva.

4.1 Calendario detallado por sesión

Sesión 4 Tema 2

Contenidos: Productores, consumidores, grupos, replicación y tolerancia a fallos.

Materiales: Presentación Apache Kafka; enunciado Kafka; producer.py y datos de sensores.

Objetivos de aprendizaje: Construir una ingesta reproducible y analizar escalabilidad y fallos.

Actividad planificada: Ejecución guiada del productor y resolución del enunciado.

Trabajo personal recomendado: Documentar configuración, pruebas, resultados y tratamiento de errores.

Sesión 5 Tema 2

Contenidos: Spark Structured Streaming: DataFrames, ventanas, estado y watermarks.

Materiales: Presentación Spark Structured Streaming; enunciado; docker-compose.yml y notebook PySpark.

Objetivos de aprendizaje: Procesar eventos con semántica temporal y manejar datos tardíos.

Actividad planificada: Construcción de consultas, ventanas y agregaciones sobre Kafka.

Trabajo personal recomendado: Completar el notebook y comparar modos de salida y políticas de watermark.

Sesión 6 Tema 3

Contenidos: Diagnóstico de rendimiento: particionado, shuffle, skew y serialización.

Materiales: Presentación del tema 3; pipelines desarrollados en los temas 1 y 2.

Objetivos de aprendizaje: Identificar cuellos de botella a partir de evidencias y métricas.

Actividad planificada: Lectura de planes de ejecución y formulación de hipótesis.

Trabajo personal recomendado: Elaborar una ficha de diagnóstico con línea base y criterio de éxito.

4.1 Calendario detallado por sesión

Sesión 7 Tema 3

Contenidos: Optimización, recursos, observabilidad y despliegue.

Materiales: Presentación del tema 3; interfaces de monitorización de Spark y Kafka.

Objetivos de aprendizaje: Proponer mejoras medibles y distinguir opciones locales y cloud.

Actividad planificada: Experimento controlado con un cambio aislado.

Trabajo personal recomendado: Comparar resultados y justificar coste, latencia y mantenibilidad.

Sesión 8 Tema 4

Contenidos: Prompt engineering y límites de los modelos de lenguaje.

Materiales: Presentación sesión 1; hoja de ejercicios de prompting.

Objetivos de aprendizaje: Diseñar prompts evaluables y reconocer límites y riesgos.

Actividad planificada: Revisión de prompts y definición de criterios de calidad.

Trabajo personal recomendado: Resolver los ejercicios y registrar iteraciones, fallos y mejoras.

Sesión 9 Tema 4

Contenidos: RAG: ingesta, fragmentación, embeddings, recuperación y respuesta.

Materiales: Presentación sesión 2; ejercicios; RAG_1.py y RAG_1_comentado.py.

Objetivos de aprendizaje: Construir un flujo RAG trazable y evaluar recuperación y respuesta.

Actividad planificada: Ejecución y análisis del ejemplo RAG con corpus deportivo.

Trabajo personal recomendado: Comparar configuraciones de recuperación y documentar evidencias y límites.

4.1 Calendario detallado por sesión

Sesión 10 Tema 4

Contenidos: Agentes, tools, skills, estado y supervisión humana.

Materiales: Presentación sesión 3; ejercicios; programas de workflows y agentes.

Objetivos de aprendizaje: Distinguir workflow y agente y controlar coste, seguridad y bucles.

Actividad planificada: Comparación de flujo lineal y flujo agéntico.

Trabajo personal recomendado: Implementar una variante controlada y justificar guardas y criterios de parada.

Sesión 11 Tema 5

Contenidos: Arquitectura integrada Kafka-Spark-RAG-agentes.

Materiales: Presentación del tema 5; materiales de los temas 1 a 4.

Objetivos de aprendizaje: Integrar ingestión, cómputo, recuperación y decisión en un sistema coherente.

Actividad planificada: Diseño de un diagrama de extremo a extremo.

Trabajo personal recomendado: Documentar contratos, estado, observabilidad y puntos de validación humana.

Sesión 12 Tema 5

Contenidos: Agente Text-to-SQL con Spark y LangGraph.

Materiales: README, agent.py, main.py y generate_data.py del ejercicio Text-to-SQL.

Objetivos de aprendizaje: Conectar lenguaje natural con consultas SQL ejecutadas sobre datos deportivos.

Actividad planificada: Generación de datos, registro de vistas y construcción de tools.

Trabajo personal recomendado: Completar los TODO, probar consultas y conservar evidencias de ejecución.

4.1 Calendario detallado por sesión

Sesión 13 Tema 5

Contenidos: RAG multimodal, narrativas e interfaces conversacionales.

Materiales: Presentación del tema 5; resultado del ejercicio Text-to-SQL.

Objetivos de aprendizaje: Separar cálculo determinista, recuperación, inferencia y generación narrativa.

Actividad planificada: Diseño de una respuesta trazable para distintos perfiles de usuario.

Trabajo personal recomendado: Preparar una memoria breve con resultados, limitaciones y riesgos del sistema.

Sesión 14 Tema 6

Contenidos: Tecnologías emergentes, predicción y prescripción.

Materiales: Presentación del tema 6; documentación técnica y literatura seleccionada.

Objetivos de aprendizaje: Evaluar madurez, ajuste al problema e impacto de una tendencia tecnológica.

Actividad planificada: Análisis comparativo de una tendencia aplicada al deporte.

Trabajo personal recomendado: Contrastar una fuente técnica primaria y una referencia científica.

Sesión 15 Tema 6

Contenidos: Ética, privacidad, legalidad, gobernanza y síntesis.

Materiales: Presentación del tema 6; guía docente; normativa y documentación institucional.

Objetivos de aprendizaje: Formular recomendaciones responsables y justificar salvaguardas.

Actividad planificada: Debate y elaboración de una nota de decisión final.

Trabajo personal recomendado: Entregar una síntesis con oportunidad, evidencia, riesgos y próximos pasos.

Uso de tutorías

- Llevar una pregunta concreta, el contexto mínimo y la evidencia del intento realizado.
- Diferenciar dudas conceptuales, errores de entorno, problemas de datos y decisiones arquitectónicas.
- Registrar la conclusión y convertirla en una prueba, una mejora documental o una decisión de diseño.

Trabajo en equipo

En las prácticas y en el proyecto final se recomienda distribuir responsabilidades sin fragmentar el conocimiento: cada integrante debe poder explicar el problema, la arquitectura, los datos, el código, las pruebas, los resultados y las limitaciones.

5. Relación con la evaluación

La evaluación oficial es continua. La guía docente establece tres componentes revaluables y exige una nota mínima de 5 en cada actividad. El proyecto final evalúa el temario completo y no constituye el contenido del tema 5. Las fechas concretas y cualquier ajuste deberán confirmarse siempre en Aula Virtual y con el equipo docente.

Actividad	Peso	Contenido	Evidencia	Referencia temporal
Prácticas 1 y 2	40 %	P1: temas 1–3 · P2: temas 4–6	Memoria y código	Semanas 5 y 10
Exposición oral	20 %	Temario completo	Presentación y preguntas	Fecha oficial
Proyecto final	40 %	Temario completo	Memoria y código	Semanas 13–15

Lista de comprobación de una entrega técnica

- ☐ El problema, los usuarios y los criterios de éxito están definidos.
- ☐ La procedencia, el significado y las limitaciones de los datos están documentados.
- ☐ La arquitectura y las decisiones tecnológicas se justifican frente a alternativas.
- ☐ El entorno y los pasos de ejecución son reproducibles.
- ☐ Las pruebas cubren comportamiento, rendimiento y fallos relevantes.
- ☐ Los resultados se apoyan en evidencias y separan hechos, inferencias y limitaciones.
- ☐ La memoria y la presentación citan fuentes y atribuyen correctamente contenidos de terceros.
- ☐ El equipo puede defender oralmente el trabajo y responder sobre código, datos y decisiones.

6. Localización y estado de los materiales

La carpeta material_original conserva los documentos de trabajo y sus fuentes. La carpeta material_abierto contiene exclusivamente versiones preparadas para publicación o documentación de su estado de revisión.

Material	Ubicación	Destino	Estado
Guía docente	material_original/guias	Referencia institucional	Disponible
Guía de estudio	material_abierto/00_guias	Categoría 0	Disponible
Presentaciones	material_original/teoria	Candidatas a categoría 2	Temas 1–6; auditoría pendiente
Fuentes de teoría	material_original/fuentes/teoria	Editables	Temas 1–6; las de T5–T6 son reconstrucciones desde PDF
Prácticas	material_original/practica	Candidatas a categoría 3	Temas 2, 4 y 5; proyecto final separado
Fuentes de práctica	material_original/fuentes/practica	Editables	Temas 2 y 4; proyecto final
Código y notebooks	material_original/programas	Candidatos a categoría 6	Temas 1, 2, 4 y 5; revisión pendiente

Qué significa «pendiente de publicación»

- El contenido puede utilizarse en la docencia, pero aún debe cerrarse su versión canónica.
- Deben verificarse autoría, licencia, atribuciones, fuentes de imágenes y materiales de terceros.
- Los editables y el software requieren documentación suficiente para reutilización autónoma.
- Solo las copias validadas pasarán al paquete definitivo de BURJC Digital.

Los PDF de los seis temas y el enunciado de la práctica final están incorporados. Las fuentes de los temas 5 y 6 son reconstrucciones fieles desde PDF, están identificadas como tales y han sido validadas mediante compilación y comparación textual con los documentos de trabajo.

7. Referencias y licencia

Documento de referencia

Universidad Rey Juan Carlos (2026). Guía docente de Big Data Processing II, Máster Universitario en Análisis de Datos Deportivos, curso 2026–2027. Fecha de publicación indicada en el documento: 7 de julio de 2026.

Bibliografía recogida en la guía docente

- Gama, J. (2010). Knowledge Discovery from Data Streams.
- Marz, N., & Warren, J. (2015). Big Data: Principles and Best Practices of Scalable Real-Time Data Systems.
- Hu, S., Ren, S., Chen, Y., Mu, C., Liu, J., Cui, Z., et al. (2025). Hands-on LLM-based Agents: A Tutorial for General Audiences.
- Karau, H., Polak, A., & Warren, R. (2026). High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark. O'Reilly Media.
- Recursos complementarios: documentación de Apache Kafka y Apache Flink; bases de datos NoSQL; herramientas de procesamiento de texto.

Licencia de esta guía

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández. Algunos derechos reservados.

Esta guía se distribuye bajo la licencia [Creative Commons Atribución–CompartirIgual 4.0 Internacional \(CC BY-SA 4.0\)](#).

La licencia se aplica exclusivamente al contenido original de esta guía. No se extiende a materiales de terceros citados, enlazados o reproducidos, que conservan su licencia o régimen jurídico propio.

Atribución recomendada

Forma breve

Fernández Isabel, A.; Madrueño Sierro, N.; y Rodríguez Fernández, R. (2026). Big Data Processing II: Guía de estudio. Universidad Rey Juan Carlos. CC BY-SA 4.0.

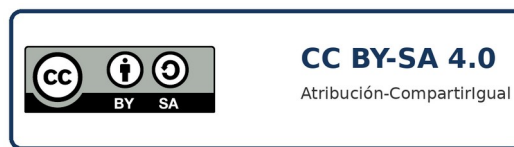
**MATERIAL DOCENTE EN ABIERTO
DE LA UNIVERSIDAD REY JUAN CARLOS**

Apuntes de Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel · Natalia Madrueño Sierro · Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de publicación

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados.

Este documento se distribuye bajo la licencia [Creative Commons Atribución-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/)

La licencia se aplica al texto, los ejemplos, las tablas y los esquemas originales. Los nombres, logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Esta edición no incorpora fotografías ni ilustraciones de terceros. Los fragmentos de código se presentan con finalidad docente y forman parte del contenido original de los apuntes.

Asignatura: Big Data Processing II. Titulación: Máster Universitario en Análisis de Datos Deportivos. Lugar previsto de depósito: BURJC Digital. Fecha de esta edición: 2026.

Índice

Nivel	Contenido
Tema	1. Procesamiento distribuido de Big Data
Tema	2. Kafka y Spark Structured Streaming
Tema	3. Optimización y observabilidad
Tema	4. IA generativa, RAG y agentes
Tema	5. Proyecto integrado
Tema	6. Tendencias y gobernanza
Tema	7. Guía de prácticas
Tema	8. Autoevaluación
Tema	9. Glosario
Tema	10. Bibliografía y documentación

Presentación

Big Data Processing II estudia cómo diseñar sistemas que reciben, procesan, almacenan y explotan datos a gran velocidad. La asignatura combina procesamiento distribuido, Apache Kafka, Spark Structured Streaming, optimización, inteligencia artificial generativa, recuperación aumentada y sistemas basados en agentes.

El contexto conductor es la analítica deportiva. Un partido o entrenamiento produce eventos procedentes de marcadores, tracking, dispositivos vestibles, sensores, aplicaciones y documentos. El reto no es solo acumularlos: hay que preservar su significado temporal, controlar duplicados y retrasos, calcular métricas reproducibles y transformar los resultados en información útil.

Los apuntes siguen seis temas:

1. procesamiento distribuido y arquitecturas Big Data;
2. procesamiento en tiempo real con Kafka y Spark;
3. optimización, escalabilidad y observabilidad;
4. modelos generativos, RAG y agentes;
5. integración de un sistema completo;
6. tendencias, gobernanza y uso responsable.

Cada tema conecta conceptos, decisiones de diseño, ejemplos deportivos y prácticas. El objetivo no es memorizar productos, sino justificar una arquitectura a partir de requisitos verificables.

1. Procesamiento distribuido de Big Data

1.1. Motivación

Un sistema centralizado ejecuta todo en una máquina. Es sencillo, pero queda limitado por sus recursos y constituye un punto único de fallo. Un sistema distribuido reparte datos y cómputo entre varios nodos.

La distribución responde a cuatro necesidades:

- **volumen:** los datos no caben o no se procesan a tiempo en un nodo;
- **velocidad:** el paralelismo reduce el tiempo de respuesta;
- **disponibilidad:** el servicio continúa si falla una máquina;
- **elasticidad:** la capacidad se adapta a la carga.

Distribuir no elimina complejidad: aparecen redes variables, relojes no perfectamente sincronizados, fallos parciales, particiones descompensadas y operaciones repetidas.

1.2. Escalado vertical y horizontal

El escalado vertical aumenta CPU, memoria o almacenamiento de una máquina. Es sencillo, pero tiene límites físicos y económicos. El horizontal añade nodos y reparte la carga. Aporta elasticidad y tolerancia a fallos, pero exige particionado, coordinación y observabilidad.

Una prueba deportiva puede funcionar localmente. Una competición con encuentros simultáneos, miles de deportistas y sensores múltiples requiere distribuir ingesta y procesamiento.

1.3. Batch, microbatch y streaming

En **batch**, los datos se acumulan y procesan como conjunto finito. Es adecuado para estadísticas históricas, entrenamiento de modelos o informes diarios.

En **streaming**, los eventos se procesan continuamente. Es útil cuando el valor disminuye con el tiempo: alertas, incidencias o analítica en directo.

El **microbatch** agrupa eventos durante intervalos breves. Spark Structured Streaming utiliza normalmente este modelo. “Tiempo real” no significa siempre milisegundos: una retransmisión puede aceptar segundos; una alerta de seguridad quizá necesite cientos de milisegundos.

1.4. Semántica temporal

Un evento puede tener:

- **event time**: cuándo ocurrió;
- **ingestion time**: cuándo entró;
- **processing time**: cuándo se procesó.

Una carrera registrada a las 18:02 puede llegar a las 18:05. Agruparla por processing time la atribuiría al intervalo equivocado.

Ventanas habituales:

- *tumbling*: intervalos consecutivos sin solapamiento;
- *sliding/hopping*: ventanas solapadas que avanzan con un salto;
- *session*: agrupan actividad separada por inactividad.

El **watermark** expresa cuánto retraso se acepta. Permite liberar estado, pero obliga a decidir qué hacer con eventos posteriores: descartarlos, almacenarlos aparte o corregir resultados publicados.

1.5. Particionado

El particionado asigna cada evento a una unidad de almacenamiento o procesamiento. Una clave adecuada mantiene juntos datos que deben ordenarse o agregarse.

Ejemplos:

- `match_id` mantiene eventos de un partido;
- `player_id` facilita métricas por deportista;
- una clave compuesta puede equilibrar partidos muy activos.

Una mala clave produce **skew**: una partición recibe la mayor parte del tráfico. La clave se elige por semántica, distribución y patrón de consulta.

1.6. Replicación y CAP

Replicar mantiene copias en varios nodos. Mejora disponibilidad y durabilidad, pero exige coordinación.

CAP indica que, durante una partición de red, hay que priorizar consistencia o disponibilidad. No obliga a elegir permanentemente una propiedad: describe el compromiso cuando falla la comunicación.

El marcador oficial puede necesitar más consistencia que un panel exploratorio.

1.7. Lambda y Kappa

Lambda combina una capa batch, que recalcula resultados completos, y una capa de velocidad, que produce resultados recientes. Es robusta, pero duplica lógica.

Kappa trata todo como flujo reproducible. Los cambios se aplican reprocesando el registro de eventos. Simplifica el modelo, pero requiere retención y procesadores capaces de reconstruir estado.

1.8. Semánticas de entrega e idempotencia

- **At most once:** puede perderse un mensaje, pero no repetirse.
- **At least once:** no debería perderse, pero puede duplicarse.
- **Exactly once:** el efecto observable se produce una vez bajo condiciones concretas.

Exactly once debe analizarse extremo a extremo. Una API externa no idempotente puede duplicar efectos aunque el motor gestione bien sus checkpoints.

Una operación idempotente produce el mismo resultado al repetirse. Identificadores de evento, claves deterministas y operaciones *upsert* ayudan a controlar duplicados.

1.9. Decisiones mínimas

Antes de implementar deben documentarse:

7. fuentes y tasa esperada;
8. latencia objetivo;
9. clave de particionado;
10. política de duplicados y retrasos;
11. estado mantenido;
12. recuperación;
13. destinos;
14. métricas de éxito.

2. Kafka y Spark Structured Streaming

2.1. Kafka como registro distribuido

Kafka organiza eventos en **topics**, divididos en **particiones**. Dentro de cada partición se conserva orden. Cada mensaje recibe un **offset**.

Componentes:

- productor;
- broker;
- consumidor;
- consumer group;
- controlador;
- réplicas.

Kafka no es solo una cola: conserva eventos, permite releerlos y admite consumidores independientes.

2.2. Diseño de topics

Una estructura posible:

- sports.raw.events;
- sports.validated.events;
- sports.metrics;
- sports.alerts;
- sports.dead-letter.

Para cada topic deben definirse esquema, clave, particiones, replicación, retención, compactación y responsables.

Crear un topic por deportista o partido suele ser inmanejable. Es preferible agrupar y usar claves.

2.3. Productores

El productor serializa, selecciona partición y espera confirmación. Deben configurarse:

- acknowledgements;
- reintentos;
- batching;
- compresión;
- idempotencia;
- timeout;
- validación de esquema.

Evento conceptual:

```
{
  "event_id": "evt-1042",
  "timestamp": "2026-10-01T18:24:12Z",
  "match_id": "match-27",
  "team": "home",
  "player": "p-8",
  "event_type": "shot",
  "zone": "box",
  "minute": 63,
  "value": 0.18
}
```

Usar `match_id` como clave conserva el orden relativo del encuentro dentro de una partición.

2.4. Consumidores

Cada partición se asigna como máximo a un consumidor del mismo grupo. Más consumidores que particiones no aumentan paralelismo.

Debe decidirse:

- cuándo confirmar offsets;
- qué ocurre si falla el trabajo;
- cómo responder a rebalances;
- qué hacer con mensajes inválidos;
- cuánto tarda un lote.

Confirmar antes del procesamiento puede producir pérdida lógica. Confirmar después exige tolerar repeticiones.

2.5. Replicación y KRaft

Cada partición tiene líder y réplicas. Si falla el líder, una réplica sincronizada puede asumirlo.

Kafka moderno utiliza KRaft y un quorum Raft para metadatos, sustituyendo la dependencia histórica de ZooKeeper. La disponibilidad sigue dependiendo de réplicas actualizadas y configuración coherente.

2.6. Esquemas

JSON es cómodo, pero producción necesita contratos. Avro, Protobuf o JSON Schema permiten validar y evolucionar.

Añadir un campo opcional suele ser compatible; eliminar o cambiar tipos puede romper consumidores. El esquema debe tratarse como API: versionado, compatibilidad, validación, documentación y pruebas.

2.7. Structured Streaming

Structured Streaming representa un flujo como una tabla que crece. El flujo habitual es:

15. leer Kafka;
16. convertir clave y valor;
17. aplicar esquema;
18. validar;

19. transformar;
20. agregar;
21. escribir.

Ejemplo abreviado:

```
raw = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", brokers)
        .option("subscribe", "sports.raw.events")
        .load()
)

events = (
    raw.selectExpr("CAST(value AS STRING) AS json")
        .select(from_json(col("json"), schema).alias("e"))
        .select("e.*")
)
```

El esquema explícito evita inferencias inconsistentes.

2.8. Ventanas y watermarks

```
metrics = (
    events
        .withWatermark("timestamp", "10 minutes")
        .groupBy(
            window(col("timestamp"), "5 minutes", "1 minute"),
            col("match_id"),
            col("team")
        )
        .agg(count("*").alias("events"))
)
```

La ventana dura cinco minutos y avanza cada minuto. El watermark permite cerrar estado según la tolerancia definida.

2.9. Modos de salida

- **append:** solo filas finales;
- **update:** filas modificadas;
- **complete:** toda la tabla agregada.

El modo debe ser compatible con consulta y destino. Complete puede resultar costoso con estados grandes.

2.10. Checkpoints

Los checkpoints conservan offsets y estado para reanudar. Deben almacenarse en un sistema duradero y cada consulta debe usar una ubicación propia.

Cambiar radicalmente una consulta reutilizando su checkpoint puede ser incompatible. Consulta, salida y checkpoint deben versionarse.

2.11. Datos tardíos e inválidos

Un pipeline debe separar eventos válidos, tardíos, inválidos y fallos transitorios.

Una dead-letter queue conserva mensajes no procesables con motivo, momento y versión del esquema. Necesita métricas y procedimiento de reprocesado.

2.12. Ejercicio integrado

Construya un productor y una consulta Spark que:

- valide esquema;
- calcule tiros y goles;
- agrupe por ventanas;
- use watermark;
- persista resultados;
- documente duplicados, eventos tardíos y recuperación.

3. Optimización y observabilidad

3.1. Medir primero

Defina:

- tasa de entrada y procesada;
- latencia y percentiles p90/p95/p99;
- CPU y memoria;
- tamaño del estado;
- lag;
- duración de microbatches;
- errores y reintentos.

El promedio oculta episodios críticos. Un p95 de ocho segundos significa que el 5 % supera ese valor.

3.2. Paralelismo

Está limitado por particiones, ejecutores y distribución de claves. Pocas particiones limitan; demasiadas incrementan metadatos, ficheros pequeños y coordinación. Debe decidirse mediante pruebas.

3.3. Shuffle

Un shuffle redistribuye datos para agrupar, unir u ordenar. Consume red, serialización, disco y memoria.

Para reducirlo:

- filtrar pronto;
- seleccionar columnas necesarias;
- agregar localmente;
- evitar reparticionados;
- equilibrar claves;
- usar broadcast join solo con tablas realmente pequeñas.

3.4. Skew

Síntomas:

- pocas tareas mucho más lentas;
- memoria desigual;
- colas crecientes con recursos libres.

Soluciones:

- salting;
- agregación en dos fases;
- separar claves calientes;
- aumentar particiones;

- rediseñar clave.

3.5. Estado

Ventanas y deduplicación mantienen estado. Si no se acota, crece indefinidamente. Se controla con watermarks, expiración, ventanas limitadas, cardinalidad y monitorización.

3.6. Formatos y compresión

Kafka usa compresión para reducir red y almacenamiento. Parquet facilita lectura columnar y filtrado. CSV es interoperable, pero pierde tipos y ocupa más.

La elección depende de lectura, tamaño, tipos, evolución y coste.

3.7. Caché

Persistir ayuda si un DataFrame se reutiliza y recomputarlo es caro. Cachear todo consume memoria y puede empeorar el sistema. Debe medirse y liberarse.

3.8. Backpressure

Si la entrada supera la capacidad, crecen lag y latencia. Opciones:

- escalar;
- aumentar particiones;
- reducir coste;
- limitar lectura;
- degradar funciones no esenciales;
- priorizar.

3.9. Observabilidad

Tres pilares:

- métricas;
- logs;
- trazas.

Conviene propagar event_id, match_id, versión de esquema y versión de procesamiento.

Kafka: producción, consumo, lag, réplicas, errores.

Spark: duración de lote, filas, memoria, spill, estado y tareas fallidas.

3.10. Experimentos reproducibles

Toda optimización debe registrar hipótesis, línea base, carga, cambio aislado, métricas, efectos secundarios y conclusión. Mayor velocidad con pérdida de corrección no es mejora global.

4. IA generativa, RAG y agentes

4.1. Papel del modelo

Un modelo puede resumir métricas o adaptar explicaciones. No debe sustituir cálculos deterministas.

Deben separarse:

- hechos calculados;
- contexto recuperado;
- instrucciones;
- salida generada;
- evidencias;
- validaciones.

4.2. Prompt engineering

Un prompt robusto incluye contexto, tarea, datos, restricciones, formato y conducta ante información insuficiente.

Ejemplo:

```
Genera un resumen para el cuerpo técnico.
Usa únicamente métricas y evidencias proporcionadas.
Distingue hechos de contexto documental.
No inventes lesiones, causas ni intenciones.
Devuelve resumen, tres hallazgos, limitaciones y evidencias.
```

Few-shot añade ejemplos. Deben ser representativos y coherentes.

4.3. Salidas estructuradas

```
{
  "match_id": "match-27",
  "summary": "...",
  "findings": [
    {"claim": "...", "evidence": ["metric:shots_home"]}
  ],
  "limitations": ["..."]
}
```

Debe validarse sintaxis, campos, tipos, límites y consistencia.

4.4. Post-training

Incluye ajuste supervisado, ajuste eficiente, preferencias, alineamiento y destilación.

No siempre es necesario. Si falta conocimiento actualizado, RAG suele ser mejor. Si se necesita comportamiento repetido o formato estable, el ajuste puede aportar valor.

4.5. RAG

Etapas:

22. ingesta;
23. limpieza;

24. fragmentación;
25. embeddings;
26. indexación;
27. consulta;
28. recuperación;
29. prompt;
30. respuesta;
31. evaluación.

RAG no garantiza veracidad. Puede recuperar contenido irrelevante o ser ignorado.

4.6. Fragmentación y metadatos

Fragmentos pequeños pierden contexto; grandes diluyen señales. Deben conservar fuente, fecha, entidad, tipo, permisos, versión y sección.

4.7. Recuperación

Puede ser densa, léxica o híbrida. MMR favorece diversidad; los filtros restringen por metadatos; un reranker reordena candidatos.

En deporte, conviene filtrar equipo, competición y temporada.

4.8. Evaluación

Recuperación:

- Recall@k;
- Precision@k;
- MRR;
- nDCG.

Respuesta:

- fidelidad;
- cobertura;
- corrección;
- utilidad;
- citas;
- afirmaciones no respaldadas.

4.9. Workflows y agentes

Un workflow ejecuta pasos prefijados. Un agente elige acciones según estado y objetivo.

Componentes:

- estado;
- modelo;
- tools;
- transiciones;
- memoria;

- criterios de parada;
- validación;
- supervisión.

Si la secuencia es conocida, un workflow es más controlable.

4.10. Tools

Una tool necesita nombre, descripción, esquema, salida estructurada, errores, permisos mínimos y límites. Separar lectura y escritura reduce riesgos. Acciones externas deben requerir aprobación.

4.11. LangGraph

Un grafo puede coordinar métricas, recuperación, validación, generación, verificación y revisión humana.

El estado debe ser pequeño, tipado y auditable. Cada nodo registra entradas, salidas y transición.

4.12. Límites

Defina máximo de iteraciones, presupuesto, timeout, detección de repetición, condición de éxito y salida segura.

4.13. Trazabilidad

Toda afirmación debe vincularse con una métrica, consulta, documento o decisión humana. La trazabilidad convierte el informe en producto revisable.

5. Proyecto integrado

5.1. Arquitectura

```

Fuentes -> Kafka -> validación -> Spark -> métricas
                                     |
Documentos -> índice RAG -> recuperación
                                     |
Métricas + contexto -> workflow/agente -> informe -> revisión
  
```

Cada flecha representa un contrato.

5.2. Capas

- **Ingesta:** recibe, identifica y conserva el original.
- **Procesamiento:** valida, deduplica, gestiona tiempo y agrega.
- **Persistencia:** separa crudo, depurado, métricas y productos.
- **Recuperación:** indexa documentos autorizados.
- **Generación:** crea un borrador estructurado.
- **Gobernanza:** registra versiones, costes, errores y permisos.

5.3. Contrato de eventos

```

{
  "event_id": "evt-1042",
  "schema_version": "1.0",
  "event_time": "2026-10-01T18:24:12Z",
  "match_id": "match-27",
  "team_id": "team-a",
  "player_id": "p-8",
  "event_type": "shot",
  "zone": "box",
  "value": 0.18
}
  
```

Reglas: identificador único, UTC, catálogos versionados, obligatorios y evolución compatible.

5.4. Métricas

Cada métrica debe indicar nombre, fórmula, unidad, ventana, población, datos faltantes, responsable y versión. Un índice sin fórmula reproducible no es auditable.

5.5. Datos heterogéneos

Se combinan eventos, alineaciones, perfiles, históricos, documentos y catálogos. Deben usarse identificadores estables, no nombres ambiguos.

5.6. Text-to-SQL

Flujo seguro:

32. interpretar;
33. consultar esquema permitido;
34. generar SQL;
35. validar;
36. bloquear escritura;

37. estimar coste;
38. ejecutar con límites;
39. devolver resultado y consulta;
40. explicar.

El modelo no accede a credenciales ni ejecuta sin restricciones.

5.7. Informe

Debe distinguir:

- hechos calculados;
- contexto recuperado;
- interpretación;
- limitaciones.

No debe atribuir causalidad cuando solo hay correlación.

5.8. Pruebas

- unitarias;
- integración;
- reproducción;
- fallos de broker;
- mensajes corruptos;
- duplicados;
- retrasos;
- documentos no autorizados;
- timeout;
- respuesta inválida.

5.9. Operación

Documento variables, dependencias, datos de ejemplo, arranque, health checks, logs, secretos, recuperación y parada.

5.10. Entregables

Código, arquitectura, contratos, pruebas, métricas, resultados, informe, evaluación, limitaciones y decisiones justificadas.

6. Tendencias y gobernanza

6.1. Lakehouse

Combina flexibilidad de data lake con garantías de warehouse. Formatos de tabla abiertos permiten transacciones, evolución de esquema, versiones y batch/streaming común. Evalúe interoperabilidad, catálogo, seguridad y bloqueo de proveedor.

6.2. Streaming avanzado

Tendencias: estado, unificación batch-streaming, consultas incrementales, CDC, event sourcing, grafos y edge. Edge reduce latencia y tráfico, pero complica operación.

6.3. Multimodalidad

Texto, vídeo, audio, tracking y sensores pueden combinarse. Exige sincronización, derechos y evaluación. No debe asumirse comprensión correcta de táctica, biomecánica o medicina.

6.4. Agentes especializados

La coordinación de agentes aumenta modularidad y superficie de fallo. Requiere protocolos, permisos, observabilidad, memoria controlada y resolución de conflictos.

6.5. Privacidad

Los datos deportivos pueden ser personales, biométricos o de salud. Principios: finalidad, minimización, base jurídica, transparencia, acceso, conservación, seguridad y derechos.

Eliminar nombres no garantiza anonimato: trayectorias y atributos pueden reidentificar.

6.6. Sesgos

El rendimiento puede variar por deporte, categoría o perfil. Deben documentarse composición, cobertura, subgrupos, métricas desagregadas y limitaciones.

6.7. Explicabilidad

La explicación se adapta:

- ingeniería: consulta, versión y logs;
- analista: métricas y evidencia;
- entrenador: hallazgos y límites;
- deportista: información comprensible;
- jurídico: datos, finalidad y automatización.

6.8. Seguridad

Amenazas: manipulación, acceso indebido, secretos, prompt injection, tool abuse, dependencias comprometidas, denegación y exfiltración.

Medidas: autenticación, cifrado, segmentación, validación, allowlists, escaneo, registros, límites y revisión humana.

6.9. Gobernanza de modelos

Registre modelo, versión, parámetros, prompt, tools, documentos, fecha, evaluación, aprobaciones, costes e incidentes.

6.10. Coste y sostenibilidad

Más nodos o modelos mayores no implican mejor solución. Evalúe utilización, coste por evento y posibilidad de usar reglas o modelos pequeños.

6.11. Lista de control

41. ¿Finalidad definida?
42. ¿Datos necesarios?
43. ¿Métricas reproducibles?
44. ¿Fuentes autorizadas?
45. ¿Afirmaciones con evidencia?
46. ¿Límites y parada?
47. ¿Salida corregible?
48. ¿Supervisión humana?
49. ¿Evaluación por subgrupos?
50. ¿Trazabilidad?

7. Guía de prácticas

Kafka

Diseñar esquema y clave; generar eventos; configurar productor; comprobar particiones; consumir; simular duplicados; medir lag; documentar recuperación.

Spark

Conectar Kafka; definir esquema; validar; añadir event time; watermark; ventanas; modo de salida; persistencia; checkpoint y reinicio.

Prompting

Definir tarea medible; prompt base; identificar fallos; añadir restricciones; comparar zero-shot/few-shot; validar formato; decidir entre RAG y ajuste.

RAG

Preparar corpus autorizado; metadatos; fragmentar; indexar; preguntas; medir recuperación; generar con evidencias; registrar fallos.

Agentes

Definir estado; tools pequeñas; workflow determinista; añadir decisión agéntica solo donde aporte valor; limitar iteraciones; probar errores; registrar trayectoria.

Proyecto final

Contrato, productor, Spark, métricas, persistencia, corpus, recuperación, orquestación, informe, pruebas y memoria.

8. Autoevaluación

51. ¿Qué diferencia existe entre event time y processing time?
52. ¿Por qué una clave puede crear skew?
53. ¿Cuándo usar batch?
54. ¿Qué representa un offset?
55. ¿Por qué más consumidores que particiones no ayudan?
56. ¿Qué función cumple un watermark?
57. ¿Qué diferencia append, update y complete?
58. ¿Por qué exactly once debe analizarse extremo a extremo?
59. ¿Qué métricas detectan backpressure?
60. ¿Cuándo es contraproducente cachear?
61. ¿Qué diferencia RAG y fine-tuning?
62. ¿Cómo separar evaluación de recuperación y generación?
63. ¿Cuándo usar workflow?
64. ¿Cómo limitar Text-to-SQL?
65. ¿Qué hace trazable una afirmación?
66. ¿Qué riesgos introduce multimodalidad?
67. ¿Por qué eliminar nombres no anonimiza?
68. ¿Qué contiene una optimización reproducible?
69. ¿Cómo separar hecho, contexto e interpretación?
70. ¿Qué evidencia exigir antes de publicar?

9. Glosario

- **Backpressure:** entrada superior a capacidad.
- **Broker:** servidor Kafka.
- **Checkpoint:** estado persistido.
- **Consumer group:** consumidores que reparten particiones.
- **Embedding:** representación vectorial.
- **Event time:** momento real.
- **Idempotencia:** repetir no cambia el efecto.
- **Lag:** distancia entre offset disponible y procesado.
- **Microbatch:** pequeños lotes periódicos.
- **MMR:** equilibrio entre relevancia y diversidad.
- **Offset:** posición en partición.
- **Partición:** unidad ordenada y paralelizable.
- **RAG:** generación aumentada con recuperación.
- **Shuffle:** redistribución entre nodos.
- **Skew:** reparto descompensado.
- **Tool:** función externa de un agente.
- **Watermark:** umbral de datos tardíos.
- **Workflow:** secuencia explícita.

10. Bibliografía y documentación

- Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- Apache Spark Structured Streaming Guide. <https://spark.apache.org/docs/latest/streaming/index.html>
- Apache Spark SQL Performance Tuning. <https://spark.apache.org/docs/latest/sql-performance-tuning.html>
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Kreps, J. (2014). *I Heart Logs*. O'Reilly.
- Burns, B. (2018). *Designing Distributed Systems*. O'Reilly.
- Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- LangGraph Documentation. <https://docs.langchain.com/oss/python/langgraph/>
- NIST AI Risk Management Framework. <https://www.nist.gov/itl/ai-risk-management-framework>
- Reglamento (UE) 2016/679, Reglamento General de Protección de Datos.

Atribución recomendada

Fernández Isabel, A.; Madrueño Sierro, N.; Rodríguez Fernández, R. (2026). *Apuntes de Big Data Processing II*. Universidad Rey Juan Carlos. CC BY-SA 4.0.

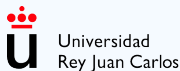
Big Data Processing II

Tema 1: Procesamiento de Big Data en Entornos Distribuidos

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición del procesamiento **batch** a **streaming** en analítica deportiva
- Identificar las características de los sistemas de **mensajería distribuida**
- Conocer los fundamentos de los motores de **procesamiento en tiempo real**
- Familiarizarse con los **principios de diseño** de arquitecturas distribuidas
- Relacionar estos conceptos para casos de uso en el **análisis deportivo en vivo**



- 1 **Introducción**
- 2 **Mensajería distribuida**
- 3 **Procesamiento en streaming**
- 4 **Diseño a nivel de sistema**



- 1 **Introducción**
- 2 Mensajería distribuida
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema



- **El análisis de datos puede apoyar la toma de decisiones durante el evento**
 - Antes $\Rightarrow$ análisis después del entrenamiento o partido
 - Ahora $\Rightarrow$ decisiones en vivo sobre táctica, rotaciones y gestión de la carga
- El foco se desplaza de **datos estáticos** a **eventos continuos** procesados en tiempo real



- **El valor del dato deportivo cae con el tiempo**

- Información procesada tarde es una oportunidad perdida
- Lo correcto a destiempo puede no ser de utilidad en la práctica

- **Ejemplos de analítica deportiva en vivo**

- Fatiga y lesión, con alertas durante el esfuerzo, no después
- Tracking de jugadores, detectando patrones de presión, desmarques y desajustes en vivo



- **Procesamiento de datos en batch**

- Se recopila un conjunto de datos y se analiza después
- Se asume que el dataset está completo antes de procesar

- **Casos de uso en el ámbito deportivo**

- Reporting / scouting, histórico, modelos offline y análisis exploratorio
- Presenta una alta latencia, el resultado llega cuando la acción ya terminó



- **Se usa un conjunto de datos cerrado y finito**
 - Hay un final conocido, se puede procesar todo
 - *input cerrado* → procesamiento → *output final*
- **Optimiza la capacidad global del sistema**
 - Throughput → cuántos datos se procesan por unidad de tiempo
- **No prioriza el tiempo de respuesta individual**
 - Latencia → cuánto tarda un evento en producir un resultado



Procesamiento por lotes

Esquema conceptual original de la edición abierta

Fuente: <https://www.decube.io/post/streaming-vs-batch-data>



- **Tecnologías habituales**

- Hadoop y MapReduce para grandes lotes distribuidos
- Spark batch como API de alto nivel e iteración más rápida

- **Almacenamiento**

- Data Lakes y Data Ware Houses para almacenar históricos

Ecosistema Hadoop

Esquema conceptual original de la edición abierta

Procesamiento distribuido con Spark

Esquema conceptual original de la edición abierta



- **Big Data se centra en volumen**

- Almacena y procesa grandes volúmenes de datos
- Procesamiento en batch analiza datos finitos
- Ejemplos incluyen datos históricos deportivos

- **Fast Data se centra en velocidad**

- Procesa y toma decisiones sobre los datos mientras se generan
- Procesamiento en streaming analiza flujos continuos
- Ejemplos incluyen datos de tracking en vivo



- **Datos finitos (bounded)**

- Datos cerrados con fin definido
- Se da un único resultado al final
- Caso particular de datos continuos

- **Datos continuos (unbounded)**

- Flujo de datos potencialmente infinito
- Hay que producir resultados en continuo



- **La temperatura del dato se refiere a su frescura y utilidad**
- **Datos fríos (cold)**
 - Histórico, auditoría y entrenamiento de modelos
 - Procesamiento en batch opera sobre frío
- **Datos calientes (hot)**
 - Máximo valor ahora, requieren decisión inmediata
 - Procesamiento en streaming opera sobre caliente



- **Concepto de panta rei o “todo fluye”**
 - La realidad está en cambio continuo
 - Es una secuencia de eventos con contexto
 - No se piensa en una tabla final, sino en una historia que se actualiza
- **Cambio de mentalidad hacia flujos dinámicos de eventos**



- **Computación incremental sobre eventos**
 - Se prioriza la velocidad en la toma de decisiones
 - Procesa eventos conforme llegan, sin esperar a un dataset completo
 - Genera resultados incrementales que se refinan con cada nuevo evento
- **Requiere un trade-off entre latencia, coste y exactitud**



Procesamiento continuo de eventos

Esquema conceptual original de la edición abierta

<https://www.decube.io/post/streaming-vs-batch-data>



- **Desacopla quién produce de quién consume**
 - Fuente → distribuidor → procesador → destino
- **Convierte eventos en señales útiles**
 - Métricas, alertas y dashboards
 - Cálculo en vivo de velocidad, fatiga y patrones tácticos



- **Se requiere un buffer intermedio que gestione el flujo continuo**
 - Amortigua picos de eventos que no llegan a ritmo constante
 - Desacopla productores y consumidores para que evolucionen independientemente
 - Retiene temporalmente eventos si el procesador se satura
 - Facilita reintentos ante fallos y, a veces, replay
- **Requiere una infraestructura de mensajería distribuida**



- **Se requiere procesar el flujo continuo de datos**
 - Transformar eventos continuos en señales útiles
 - Se puede hacer tratando cada evento de forma independiente
 - Se puede recordar eventos pasados para computar vistas acumuladas
- **Requiere un motor de procesamiento en streaming**



- **Piezas típicas de un sistema de streaming**

- Fuentes → sensores, wearables, logs
- Mensajería → Kafka, Pulsar, RabbitMQ
- Procesador → Flink, Spark Streaming, Storm, Beam
- Destinos → bases de datos, dashboards, data lake, alertas

- **Streaming no es una herramienta, es un sistema de piezas coordinadas**



Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

<https://www.scaler.com/topics/spark-streaming-with-kafka/>



- 1 Introducción
- 2 Mensajería distribuida**
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema



- **Los eventos no se producen a un ritmo constante**
 - Sin una infraestructura intermedia el sistema colapsa
 - Existen picos de carga y consumidores lentos
 - Las fuentes de datos emiten a ritmo irregular
 - Los consumidores no pueden ir sincronizados
- **Middleware Orientado a Mensajes (MOM)**
 - Infraestructura intermedia que absorbe picos y reparta trabajo



- **Integración punto a punto (P2P)**

- Muchas conexiones directas suponen un acoplamiento fuerte
- Cambios en contratos y formato implican integraciones frágiles
- Cada nuevo consumidor depende excesivamente de los productores

- **Arquitectura dirigida por eventos (EDA)**

- Productor → Distribuidor → Consumidor
- Productores y consumidores se encuentran desacoplados
- Añadir consumidores no implica cambiar productores

Arquitectura orientada a eventos y agentes

Esquema conceptual original de la edición abierta

<https://solace.com/blog/agentic-ai-the-new-microservices/>



- **Infraestructura de eventos para datos en movimiento**
 - Ingiere, retiene y distribuye eventos de forma desacoplada
 - Absorbe picos mediante buffering y controles de carga
 - Fuentes → Analítica en vivo → Histórico
- **Sistemas de mensajería distribuida para eventos**



- **Un mensaje es el “sobre” que transporta un evento**
 - Unidad de comunicación → payload + headers
 - Foco en entrega → ack, routing, reintentos y colas
- **Un evento tiene un significado en el dominio**
 - Algo ocurrió (quién, qué, cuándo) + contexto
 - Foco en significado → métricas, alertas y reconstrucción de historia
- **Se transportan eventos dentro del payload de un mensaje**



- **Cola de mensajes (Queue)**

- Un evento lo procesa un único consumidor
- Útil para tareas y reparto de carga

- **Publicación/Suscripción (Pub/Sub)**

- Un evento lo reciben muchos consumidores (fan-out)
- Útil para métricas + alertas + persistencia en paralelo

Colas y publicación-suscripción

Esquema conceptual original de la edición abierta

<https://docs.cloud.google.com/solutions/event-driven-architecture-pubsub>



- **Los mensajes no siempre llegan en orden ni en tiempo**
 - Latencia en red, buffering y dispositivos que se desconectan y vuelven
 - Algunas métricas son provisionales y se corrigen si llega un evento tarde
- **El sistema de mensajería debe tolerar desorden y retardos**
- **Se debe decidir entre perder eventos o tolerar duplicados**



- **At-most-once**

- Más rápido $\Rightarrow$ mínima latencia y overhead
- Menos fiable $\Rightarrow$ puede perder mensajes

- **At-least-once**

- Más fiable $\Rightarrow$ entrega una o más veces
- Más lento $\Rightarrow$ más overhead/latencia

- **Exactly-once**

- Mismo resultado final (efecto único)
- No significa “cero duplicados en la red”
- Suele ser más lento y depende de la plataforma



- **Todo sistema de mensajería gestiona cuatro aspectos clave**
 - Canales para organizar mensajes en colas, topics o streams
 - Escalado y paralelismo mediante particiones o consumidores
 - Fiabilidad mediante acks, reintentos y persistencia
 - Consumo destructivo tras ack o replay histórico
- **Cada sistema los prioriza e implementa de forma distinta**



- **Brokers clásicos**

- Optimiza la entrega por mensaje
- *Smart broker* → más lógica y decisiones en el broker

- **Events logs**

- Optimiza retención, offsets y replay del flujo de eventos
- *Dumb broker, smart consumer* → más lógica y decisiones en consumidores

- **Tareas y routing vs histórico, fan-out y reproceso**



- **Optimiza entrega y routing por mensaje**
 - Colas y pub/sub con routing flexible (topics/keys/exchanges)
 - Patrones de integración habituales (filtering, routing, request/reply)
- **Poseen una alta fiabilidad operativa**
 - Acks y reintentos, control de redelivery
 - Dead-letter queues y políticas de error
- **Ideal para mensajes/tareas asíncronas e integración empresarial**
 - Ejemplos incluyen RabbitMQ, ActiveMQ, Amazon SQS, ...

Broker RabbitMQ

Esquema conceptual original de la edición abierta

Servicio de colas gestionado

Esquema conceptual original de la edición abierta



- **Optimiza el flujo de eventos como un log distribuido**
 - Append-only (commit log) con retención (por tiempo/tamaño)
 - Lectura por offsets → cada consumidor controla su progreso
- **Habilita fan-out, retención y replay**
 - Muchos consumidores independientes sobre el mismo flujo
 - Retención para recomputar vistas, auditoría y nuevos consumidores
- **Ideal para analítica en vivo y pipelines de datos**
 - Ejemplos incluyen Kafka, Pulsar, AWS Kinesis, ...

Plataforma de eventos Kafka

Esquema conceptual
original de la
edición abierta



- 1 Introducción
- 2 Mensajería distribuida
- 3 Procesamiento en streaming**
- 4 Diseño a nivel de sistema



- **Streaming procesa flujos continuos de eventos**
 - No se trata solo de hacerlo rápido
 - Se computa sobre un flujo continuo que no termina
- **Batch es un caso particular de un stream finito**



- **Dimensiones clave en el procesamiento en streaming**
 - Tiempo → ¿Cuándo ocurrió el evento vs cuándo lo proceso?
 - Estado → ¿Necesito recordar el pasado para procesar el presente?
 - Correctitud → ¿Qué garantías doy ante fallos, retrasos y duplicados?
- **Cada sistema en streaming prioriza unas dimensiones sobre otras**
- **Existen diferentes enfoques de procesamiento según estas dimensiones**



- **Simple Event Processing (SEP)**

- Filtra, enruta y transforma evento a evento
- Ejemplos incluyen filtrar posiciones imposibles

- **Event Stream Processing (ESP)**

- Consultas continuas, como ventanas, agregaciones y métricas online
- Ejemplos incluyen sprints por jugador en 5 min

- **Complex Event Processing (CEP)**

- Patrones complejos y temporales, como secuencias, correlación y anomalías
- Ejemplos incluyen análisis de presión sostenida



- **Procesamiento sin estado (stateless)**

- Cada evento se procesa de forma independiente
- Operaciones típicas incluyen map, filter y enrich

- **Procesamiento con estado (stateful)**

- El procesamiento depende de eventos previos
- Operaciones típicas incluyen agregaciones, joins, sesiones y ventanas

- **El estado introduce complejidad en el sistema**



- **Tiempo de proceso**

- Cuándo el sistema procesa el evento
- Es el reloj del sistema
- Monitoriza latencia y rendimiento

- **Tiempo de evento**

- Cuándo ocurrió en el mundo real
- Es el reloj del evento
- Necesario para métricas correctas

- **La verdad del juego es el tiempo de evento**

Tiempo de procesamiento y tiempo de evento

Esquema conceptual original de la edición abierta

Hesse, Guenter. (2022). A benchmark for enterprise stream processing architectures. 10.25932/publishup-56600.



- **Alinear el tiempo entre fuentes supone un reto**
 - Múltiples fuentes con relojes distintos
 - Desfases de reloj y necesidad de normalizar timestamps
 - Retrasos por red, buffering o dispositivos offline
 - Eventos llegan desordenados o en bloque tras reconexión
- **Ventanas en tiempo de evento con tolerancia a tardíos**
 - Métricas se calculan en tiempo de evento y no de proceso
 - El sistema acepta tardíos y aplica correcciones con triggers y watermarks



- **En flujos continuos no hay un fin definido**
 - Para agregar lo infinito se trocea en fragmentos finitos
 - Se agrupan eventos en ventanas en tiempo de evento
 - Cada ventana produce un resultado actualizable
- **El tipo de ventana depende de la pregunta**
 - Intervalos fijos, solapados o por inactividad (sesiones)
 - Es necesario definir tamaño, salto y tolerancia a tardíos
- **Mayor correctitud implica mayor complejidad**



- **Tumbling**

- Ventanas con segmentos contiguos de tamaño fijo y sin solape
- Cada evento cae en una única ventana

- **Hopping/Sliding**

- Ventanas fijas con un salto definido con posible solape
- Un evento puede contribuir a varias ventanas

- **Session (por inactividad)**

- Ventanas de tamaño variable delimitadas por un gap sin eventos
- Con tardíos, pueden requerir merge de sesiones

Ventanas de intervalo fijo

Esquema conceptual original de la edición abierta

Ventanas deslizantes

Esquema conceptual original de la edición abierta

Ventanas de sesión

Esquema conceptual original de la edición abierta



- **Triggers para emitir resultados por ventana**
 - Early → rápido y aproximado
 - On-time → cuando la ventana se considera completa
 - Late → actualizaciones por eventos tardíos
- **Streaming involucra resultados revisables**



- **En tiempo de evento los datos pueden llegar tarde y desordenados**
- **Un watermark indica el progreso del tiempo de evento**
 - El sistema necesita una noción de progreso para no esperar indefinidamente
 - Los eventos con $\text{timestamp} < \text{watermark}$ se consideran tardíos (late data)
 - A partir de ese instante asume que eventos más antiguos serán excepcionales
- **Decide cuándo cerrar ventanas y emitir resultados**
 - Agresivo $\rightarrow$ menor latencia, más tardíos y posibles correcciones
 - Conservador $\rightarrow$ más completitud/exactitud, mayor latencia y más estado



- **Se debe definir hasta cuándo se aceptan tardíos y correcciones**
 - Para eventos que llegan con $\text{timestamp} < \text{watermark}$
 - Se define un trade-off entre latencia, coste y exactitud
- **Políticas de late data más habituales**
 - Ignorar/descartar $\rightarrow$ menor coste pero pérdida de exactitud
 - Emitir correcciones $\rightarrow$ resultados actualizables pero mayor cómputo
 - Retener estado más tiempo $\rightarrow$ acepta tardíos pero requiere más estado



- **Micro-batches**

- Acumula eventos durante un intervalo y los procesa como bloque
- Simplifica fallos y permite obtener un alto throughput

- **Streaming nativo**

- Procesa de forma continua evento a evento
- Menor latencia y carga más uniforme sin esperas por lotes

- **Throughput y simplicidad vs latencia mínima**

Microprocesamiento por lotes

Esquema conceptual original de la edición abierta



- **Optimiza throughput y simplicidad operativa**
 - Trata el stream como secuencia de mini-datasets finitos
 - Simplifica recuperación ante fallos y checkpointing
- **A menudo integra batch y streaming**
 - API similar para batch y streaming
 - Aprovecha optimizaciones del motor batch
- **Ideal para analítica con latencias bajas**
 - Ejemplos incluyen Spark Structured Streaming

Motor Apache Spark

Esquema conceptual original de la edición abierta



- **Optimiza baja latencia evento a evento**
 - Procesa eventos de forma continua al llegar
 - Carga más uniforme sin esperas por lotes
- **Estado de primera clase gestionado por el motor**
 - Diseñado para event time, watermarks y estado distribuido
- **Ideal para latencias extremadamente bajas**
 - Ejemplos incluyen Apache Flink, Storm

Motor Apache Flink

Esquema conceptual original de la edición abierta

Motor Apache Storm

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Mensajería distribuida
- 3 Procesamiento en streaming
- 4 Diseño a nivel de sistema**



- **Pasar de batch y monolitos a tiempo real distribuido supone un reto**
 - Nuevas piezas → mensajería distribuida y motor streaming
 - Nuevas propiedades → latencia, estado, tiempo de evento
 - Nuevas complejidades → desorden, tardíos, fallos parciales
- **El objetivo es equilibrar latencia, throughput y exactitud**



- **No es posible garantizar simultáneamente**

- **Consistency** → todos ven lo mismo a la vez
- **Availability** → el sistema siempre responde
- **Partition** → sobrevives a particiones de red

- **En sistemas distribuidos siempre hay particiones**

- Se debe elegir entre consistency (CP) o availability (AP)

Compromisos del teorema CAP

Esquema conceptual original de la edición abierta

<https://www.geeksforgeeks.org/system-design/>

[cap-theorem-in-system-design/](#)



- **AP prioriza disponibilidad y baja latencia**
 - Responder rápido aunque sea con una vista desactualizada
 - La consistencia eventual puede no estar al milisegundo
- **CP prioriza consistencia y exactitud**
 - Responder coherente aunque sea más lento
 - Puede bloquear o dar error antes que dar un dato inconsistente
- **Para analítica en vivo se suele priorizar AP**



- **Idealmente se quiere velocidad y exactitud a la vez**
 - Streaming da rapidez pero puede ser aproximado
 - Batch da precisión pero llega tarde
- **Lambda propone dos caminos en paralelo**
 - Capa rápida para resultados inmediatos
 - Capa batch para corregir el histórico
- **El usuario ve resultados rápidos que se refinan con el tiempo**



- **Speed layer para procesamiento en tiempo real**
 - Eventos recientes con baja latencia
 - Puede ser aproximado o tener gaps
- **Batch layer para reproceso del histórico completo**
 - Recalcula todo desde el principio periódicamente
 - Resultados exactos y completos pero que tardan más
- **Serving layer para combinar ambas vistas**
 - Speed para lo reciente + batch para el histórico corregido



Arquitectura Lambda

Esquema conceptual original de la edición abierta

<https://www.chaosgenius.io/blog/lambda-architecture/>



- **Proporciona velocidad y exactitud**
- **Supone una alta complejidad operativa**
 - Dos sistemas separados para batch y streaming
 - Misma lógica implementada dos veces
 - Difícil depurar cuando no coinciden los resultados
- **Lambda no vence el teorema CAP**



- **En Kappa todo es streaming**
 - Un solo motor para procesar eventos
 - Batch es simplemente un stream acotado con inicio y fin
- **Simplifica con un solo camino y una sola lógica**



- **Log persistente como fuente de verdad**
 - El sistema de mensajería conserva eventos
 - El motor de streaming lee desde el log
- **Procesamiento único con replay cuando se necesita**
 - Normalmente procesa eventos en tiempo real
 - Si hay que recalcular lee el log desde el principio (replay)
- **El motor de streaming procesa en vivo y reprocesa con replay**



Arquitectura Kappa

Esquema conceptual original de la edición abierta

<https://www.chaosgenius.io/blog/kappa-architecture/>



- **Si se descubre un error en el procesamiento**
 - Se corrige la lógica y se lanza un nuevo proceso
 - El proceso lee el log completo desde el principio
 - Procesa todo a máxima velocidad hasta alcanzar el presente
 - Se sustituye el proceso antiguo por el nuevo
- **Se requiere un log persistente y un motor streaming robusto**



- **Elegir entre Lambda o Kappa no es suficiente**
- **Existen múltiples aspectos clave a considerar**
 - ¿Cómo servir los datos según cada necesidad? (serving)
 - ¿Cómo operar el sistema en producción? (fiabilidad)
 - ¿Cómo garantizar calidad de datos en tiempo real?
 - ¿Cómo proteger privacidad y definir accesos?



- **Un mismo evento tiene diferentes vistas según el uso**
- **Dashboard en vivo (hot)**
 - KPIs por ventana (1–3 min) + alertas inmediatas
- **Histórico para ML y análisis (cold)**
 - Dataset curado para entrenar y validar modelos



- **El sistema debe manejar picos y fallos**
 - Durante un partido ocurren muchos eventos simultáneos
 - Si algo falla el sistema debe recuperar sin perder datos
- **Existen diferentes métricas clave a vigilar**
 - ¿Existen retrasados? (lag)
 - ¿Los datos llegan a tiempo? (latencia)
 - ¿Se están perdiendo eventos? (errores)
- **Si el sistema se satura se debe escalar o priorizar eventos**



- **En streaming no se puede parar para limpiar**
 - Se necesitan reglas claras de calidad
 - El sistema debe seguir funcionando con datos imperfectos
- **Se requiere validación y limpieza en tiempo real**
 - Eventos malformados van a cuarentena (dead-letter queue)
 - Deduplicar eventos repetidos por ID
 - Validar esquemas y compatibilidad
- **El reto no es solo ser rápido sino también coherente**



- **Es necesario definir quién accede a qué datos**
 - Diferentes roles con distintos permisos
 - Datos agregados vs datos individuales
- **Existen diferentes preguntas clave**
 - ¿Quién puede ver qué? (staff, analistas, médicos)
 - ¿Para qué se usa? (rendimiento, prevención, investigación)
 - ¿Cuánto tiempo se guarda? (retención y borrado)
- **Se deben definir responsabilidades y límites**



- **Elección de arquitectura según necesidades**
 - Batch si latencia no importa (reportes diarios)
 - Streaming si necesitas reaccionar rápido (alertas en vivo)
 - Lambda (robusta, dos caminos) vs Kappa (simple, un camino con replay)
- **Streaming consta de mensajería + motor + serving**
 - Mensajería que desacopla fuentes y consumidores
 - Motor streaming procesa con posible estado y ventanas de tiempo
 - Serving genera vistas hot (alertas en vivo) y cold (ML e histórico)
- **Trade-offs inevitables son velocidad vs exactitud vs complejidad**



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

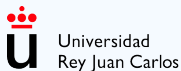
Big Data Processing II

Tema 2: Apache Kafka

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Entender qué problema resuelve en el **análisis de datos deportivos**
- Conocer la arquitectura y los **conceptos clave** de Kafka
- Identificar **casos de uso** en datos deportivos en tiempo real
- Aprender **buenas prácticas** de diseño y operación de Kafka



- 1 **Introducción**
- 2 **Componentes**
- 3 **Infraestructura**
- 4 **Ejemplos**



- 1 **Introducción**
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos



- **Con pocas integraciones todo es simple**
 - Antes $\Rightarrow$ una aplicación alimenta directamente a otra
 - Ahora $\Rightarrow$ múltiples fuentes y múltiples destinos
- **La integración punto a punto introduce deuda técnica**
 - Cada fuente habla con cada destino
 - Formatos y contratos inconsistentes
 - Difícil rastrear el flujo de información end-to-end
 - Un cambio en un productor rompe consumidores en cascada
- **La complejidad crece rápidamente con cada nueva integración**

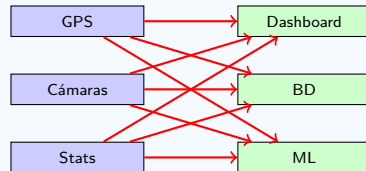


- **Sistema de datos en un estadio**

- Sensores GPS de jugadores
- Cámaras de tracking
- Sistema de estadísticas en vivo

- **El resultado es caos y complejidad**

- Acoplamiento fuerte entre sistemas
- n fuentes $\times$ m destinos = $n \times m$ integraciones



9 conexiones para 6 sistemas



- **Introducir un intermediario central**

- Productor → Distribuidor → Consumidor

- **Hub central de datos deportivos**

- Productores → clasifican y publican eventos
- Consumidores → se suscriben a lo que necesitan

- **Menos acoplamiento y mayor flexibilidad**

- Añadir consumidores no implica cambiar productores y viceversa
- Permite múltiples consumidores con diferentes necesidades

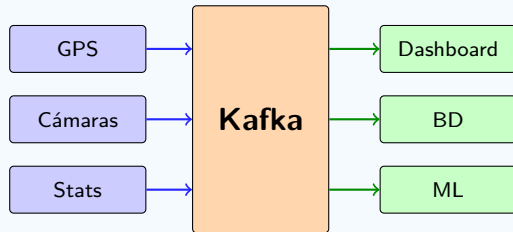


- **Kafka como hub central de mensajería**

- Todas las fuentes envían a Kafka
- Todas las herramientas leen de Kafka
- Los distribuye a múltiples destinos

- **Desacoplamiento entre sistemas**

- $n + m$ integraciones vs $n \times m$
- Sistemas independientes
- Formato estandarizado
- Escalabilidad horizontal



6 conexiones para 6 sistemas



- **Colas tradicionales**

- Diseñadas para distribución de tareas
- Mensajes eliminados tras consumo
- Un consumidor por mensaje

- **Logs persistentes**

- Diseñadas para análisis de datos y eventos
- Mensajes persistentes con retención configurable
- Múltiples consumidores independientes
- Replay de eventos históricos

- **Kafka es un commit log distribuido**



- **Desacopla fuentes (productores) de herramientas (consumidores)**
- **Múltiples consumidores para el mismo flujo de datos**
 - Dashboard en vivo para entrenadores
 - Análisis táctico con machine learning
 - Almacenamiento histórico en base de datos
- **Facilita análisis de datos deportivos en vivo**
 - Posiciones GPS de jugadores cada segundo
 - Eventos del juego, como goles, pases, tiros, tarjetas
 - Métricas fisiológicas, como ritmo cardíaco, velocidad



Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

<https://www.geeksforgeeks.org/apache-kafka/kafka-architecture/>



- 1 Introducción
- 2 Componentes**
- 3 Infraestructura
- 4 Ejemplos



- **Estructura append-only**

- Escrituras secuenciales al final
- Inmutabilidad de eventos pasados
- Alta eficiencia de escritura y lectura
- Durabilidad de eventos configurable

- **Offsets como punteros de lectura**

- Identificador único por mensaje en partición
- Orden secuencial de eventos por partición
- Permite lectura determinista por partición
- Checkpointing del progreso del consumidor

Consumidor de eventos

Esquema conceptual original de la edición abierta



- **Estructura de un mensaje**

- **Key** (opcional) → ayuda a decidir partición y mantener orden por entidad
- **Value** → payload del mensaje en bytes
- **Timestamp** → marca temporal (por defecto del productor)

- **Kafka no impone un formato específico**

- JSON (recomendado)
- Apache Avro para consistencia



- Ejemplo de evento para gol en un partido

```
// Kafka Key liga_2025_j27_001, Timestamp 2025-03-15T16:42:18Z
{
  "schema_version": "1.0",
  "event_id": "3f7a2b9c-5d8e-4c1f-a6b3-9e4d2c1b5a7f",
  "event_time": "2025-03-15T16:42:18Z",
  "match_id": "liga_2025_j27_001",
  "event_type": "goal",
  "team_id": "real_madrid",
  "opponent_id": "barcelona",
  "scorer_id": "vinicius_jr",
  "assister_id": "modric",
  "minute": 67
}
```



- **Contrato que define la estructura de los datos**
 - Plantilla que define qué campos tiene cada evento
 - Facilita entendimiento entre productores y consumidores
 - Evita errores → si falta un campo se detecta inmediatamente
- **Ejemplo de esquema para evento de gol**
 - `event_id: string`
 - `event_time: timestamp`
 - `match_id: string`
 - `event_type: string`
 - `scorer_id: string`
 - `assister_id: string`
 - `minute: integer`



- **Lotes de mensajes**

- Colección de mensajes del mismo Topic y partición
- Reduce overhead de red y permite compresión
- En lugar de enviar 1 evento a la vez envía varios juntos
- Trade-off → lotes grandes pueden incrementar la latencia

- **Compresión de datos**

- Ahorra espacio y velocidad de red
- Kafka soporta compresión configurable
- Existen varios algoritmos de compresión



- **Canales lógicos donde se organizan los mensajes**
 - Categoría de mensajes relacionados
 - Organiza eventos por tipo o dominio
 - Patrón común de nombrado →
`dominio.subdominio.tipo.version`
- **Configuraciones clave**
 - Número particiones para paralelismo
 - Factor de replicación para durabilidad
 - Tiempo de retención de los mensajes
- **Ejemplo** `olympics.swimming.events.v1`

Particionado de un topic

Esquema conceptual original de la edición abierta



• **Procesamiento en paralelo**

- Un Topic se divide en múltiples particiones
- Unidad de paralelismo y escalado horizontal
- Cada partición es un log ordenado independiente
- Es *append-only* → se escribe al final

• **Orden garantizado dentro de una partición**

- No garantiza orden en todo el Topic
- Key → misma partición
- Sin key → round-robin o sticky

Particionado de un topic

Esquema conceptual original de la edición abierta



- **Productores**

- Escriben mensajes en Topics
- Balancean carga entre particiones
- Con *key* garantizan orden por entidad en una partición

- **Consumidores**

- Leen mensajes manteniendo orden dentro de cada partición
- Mantienen offsets con la posición del último mensaje leído
- Para recuperación tras fallo reinician desde el último offset guardado



- **Marcadores de lectura en partición**

- Permite recordar qué eventos se han procesado
- Como un marcador en un libro

- **Ejemplo práctico de uso de offsets**

- Se disponen 1000 eventos GPS
- La lectura se cae en el evento 750
- Al reiniciar se continúa desde el 751

- **Commit automático vs manual**

- En *Auto* → Guarda el offset automáticamente
- En *Manual* → Tú decides cuándo guardarlo

Gestión de offsets

Esquema conceptual original de la edición abierta



- **Indica qué tan atrasado está el consumidor**

- Diferencia entre ultimo offset disponible y offset del consumidor
- End Offset=1000, Current Offset=750 → Lag=250 mensajes

- **Importancia de monitorear el lag**

- Lag alto → consumidor lento o problemas de procesamiento
- Lag creciente → consumidor no puede mantener el ritmo
- Alertar cuando supera umbrales críticos



- **Un Topic puede ser leído por múltiples consumidores**
- **Mismo grupo de consumidores**
 - Múltiples consumidores procesando en paralelo un Topic
 - Si un consumidor procesa rápido, reduce el lag del grupo
 - Si un consumidor falla, otros continúan
 - Ejemplo → 3 consumidores procesando eventos GPS
- **Único consumidor (distinto grupo)**
 - Aplicaciones independientes
 - Ejemplo → Dashboard + Análisis ML + DB histórica



- **Permite paralelizar la lectura de un Topic**
 - Escalabilidad horizontal de consumidores
 - Consumidores agrupados por `group_id`
 - Paraleliza lectura dentro de un Topic
- **Los mensajes se distribuyen en particiones**
 - Dentro del grupo cada partición la lee un solo consumidor a la vez
 - Para rendimiento máximo de lectura $\rightarrow$ particiones $\approx$ consumidores (en el grupo)
- **Si un consumidor falla los restantes se reparten sus particiones**

Grupo de consumidores

Esquema conceptual original de la edición abierta



- **Reasignación automática de particiones**
 - Se activa cuando un consumidor se une, sale o falla
 - Las particiones se redistribuyen entre consumidores activos
- **Impacto durante el rebalance**
 - El grupo deja de consumir temporalmente
 - Puede causar duplicados si no se commitean offsets antes
 - Tiempo de pausa depende del tamaño del grupo
- **Varias estrategias de asignación**
 - Range, Round-robin, Sticky



- 1 Introducción
- 2 Componentes
- 3 Infraestructura**
- 4 Ejemplos



- **Broker**

- Almacena particiones asignadas
- Es un servidor individual (nodo) del clúster
- Tienen un identificador único

- **Clúster**

- Conjunto de brokers coordinados
- Un broker actúa como controlador
- Distribución de particiones entre brokers

- **Permite escalabilidad y alta disponibilidad**



- **Modelo líder-seguidor**

- Líder maneja lecturas y escrituras
- Seguidores copian log del líder

- **In-Sync Replicas (ISR)**

- Réplicas sincronizadas con el líder
- Pueden ser elegidas como líder

- **Si el líder falla se elige otro**

- Nuevo líder es un seguidor en ISR
- El sistema sigue funcionando sin pérdida

Replicación y tolerancia a fallos

Esquema conceptual original de la edición abierta



- **Acknowledgment (ack)**

- Recibido por el productor
- Brokers confirman mensajes recibidos

- **Tipos de acknowledgment**

- `acks=0` → sin confirmación
- `acks=1` → líder confirma
- `acks=all` → todas las ISR confirman



- **At-most-once**

- El dato puede perderse pero no se duplica
- Cuando velocidad es más importante que completitud
- Ejemplo → Sensor GPS falla, pierdes una posición

- **At-least-once** - Opción más común

- El dato llega pero puede duplicarse
- Garantiza que no se pierden eventos
- Ejemplo → El mismo gol aparece dos veces en tu sistema

- **Exactly-once**

- El dato se procesa una vez
- Kafka lo soporta pero requiere configuración avanzada



- **Durabilidad es no perder datos**
- **Tres pilares para garantizar durabilidad**
 - Producers esperan confirmación del broker
 - Múltiples copias en ISR
 - Datos escritos a disco
- **Mayor durabilidad implica mayor latencia**



- **Zookeeper (legacy)**

- Almacena metadatos del clúster y coordina elección de líderes
- Ensemble con número impar de nodos (3, 5, 7)
- Requiere gestión adicional de infraestructura

- **KRaft (Kafka 3.0+)**

- Gestiona metadatos y líderes sin Zookeeper
- Protocolo Raft para consenso y elección de líderes
- Menos complejidad y arranque más rápido

- **Se recomienda KRaft para nuevos clústeres**

Coordinación mediante KRaft

Esquema conceptual original de la edición abierta



- **Regla práctica**

- Particiones $\approx$ **throughput deseado** / **velocidad del consumidor**
- Ejemplo $\rightarrow 1 \text{ GB/s} / 50 \text{ MB/s} \approx 20$ particiones

- **Consideraciones**

- Se pueden aumentar, pero no disminuir
- Más particiones $\rightarrow$ más concurrencia
- Demasiadas $\rightarrow$ mayor uso de memoria y coste de coordinación



- **Retención por tiempo**

- Se borran segmentos cuando el último mensaje supera la edad límite
- Ejemplo → conservar datos de un partido durante 7 días

- **Retención por tamaño**

- Útil para evitar llenar el disco con flujos impredecibles
- Ejemplo → máximo 1GB por partición

- **Log compaction**

- Mantiene el último valor por clave
- Elimina versiones antiguas de la misma clave
- Ejemplo → Mantener última posición GPS de cada jugador



- 1 Introducción
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos**



- **Kafka 4.1+ incluye KRaft (sin Zookeeper)**
 - Arranque rápido con un solo contenedor
 - Ideal para desarrollo y pruebas locales
- **Comando para lanzar Kafka localmente**
 - Puerto 9092 expuesto para productores y consumidores
 - Broker disponible en localhost:9092

Terminal Shell Terminal

```
docker run -p 9092:9092 apache/kafka:4.1.1
```



Terminal Shell Terminal

```
docker run -p 9092:9092 \  
  -e KAFKA_HEAP_OPTS="-Xmx1G -Xms1G" \  
  -e KAFKA_LOG_RETENTION_HOURS=168 \  
  -e KAFKA_NUM_PARTITIONS=3 \  
  -e KAFKA_DEFAULT_REPLICATION_FACTOR=1 \  
  -e KAFKA_COMPRESSION_TYPE=gzip \  
  apache/kafka:4.1.1
```

Memoria JVM
Retencion 7 dias
Particiones defecto
Factor replicacion
Compresion



```
services:
  kafka:
    image: apache/kafka:4.1.1
    ports:
      - "9092:9092"
    volumes:
      - kafka-data:/var/lib/kafka/data      # Persistencia
    environment:
      KAFKA_HEAP_OPTS: "-Xmx1G -Xms1G"    # Memoria JVM
      KAFKA_LOG_RETENTION_HOURS: 168       # Retencion 7 dias
      KAFKA_NUM_PARTITIONS: 3              # Particiones defecto
      KAFKA_DEFAULT_REPLICATION_FACTOR: 1 # Factor replicacion
      KAFKA_COMPRESSION_TYPE: gzip         # Compresion
```



- **Librería kafka-python para casos de uso sencillos**
 - Cliente básico de Kafka para Python
 - Soporte completo para Producer y Consumer
- **Componentes principales**
 - KafkaProducer: Envía mensajes a topics
 - KafkaConsumer: Lee mensajes de topics
 - KafkaAdminClient: Gestión de topics y configuración
- **Librería confluent-kafka para producción y alto rendimiento**



Python Python Code

```
from kafka import KafkaProducer
import json
import random
import time

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

for i in range(100):
    message = {'id': i, 'value': random.randint(1, 100)}
    producer.send('random.numbers.v1', value=message)
    time.sleep(1)

producer.close()
```



Python Python Code

```
producer = KafkaProducer(  
    bootstrap_servers=['localhost:9092'],  
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),  
  
    # Garantías de entrega  
    acks='all',                # 0, 1, o 'all'  
    retries=3,                 # reintentos  
    enable_idempotence=True,   # exactly-once  
  
    # Rendimiento  
    compression_type='gzip',  # gzip, snappy, lz4, zstd  
    batch_size=16384,          # tamaño lote (bytes)  
    linger_ms=10               # espera antes de enviar  
)
```



Python Python Code

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'random.numbers.v1',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='latest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

print("Listening for messages...")

for message in consumer:
    data = message.value
    print(f"Received: ID={data['id']}, Value={data['value']}")
```



Python Python Code

```
consumer = KafkaConsumer(  
    'random.numbers.v1',  
    bootstrap_servers=['localhost:9092'],  
  
    # Deserialización de mensajes  
    key_deserializer=lambda k: k.decode('utf-8') if k else None,  
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),  
  
    # Grupo de consumidores  
    group_id='my-consumer-group',  
  
    # Gestión de offsets  
    auto_offset_reset='latest',      # earliest, latest, none  
)
```



- **Kafka como hub central de datos deportivos**
 - Reduce complejidad y fragilidad frente a integraciones P2P
 - Hub central para datos deportivos en tiempo real
- **Kafka es un commit log distribuido**
 - Persistencia y retención → replay y múltiples consumidores
 - Topics/particiones → orden por partición y paralelismo
 - Offsets → control del progreso y recuperación ante fallos
- **Operación y escalado requieren diseño**
 - Replicación líder/seguidores, dimensionamiento de particiones
 - Políticas de retención según tipo de dato deportivo
 - Configuración apropiada para garantías de entrega



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

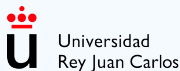
Big Data Processing II

Tema 2: Spark en Streaming

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Entender qué problema resuelve en el **análisis de datos deportivos**
- Conocer los conceptos clave de **Spark Structured Streaming**
- Aplicar **casos de uso** en datos deportivos en tiempo real
- Saber **optimizar consultas** y rendimiento en streaming



- 1 **Introducción**
- 2 **Componentes**
- 3 **Infraestructura**
- 4 **Ejemplos**



- 1 **Introducción**
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos



- **Procesamiento de datos en tiempo real**
 - Datos que llegan continuamente (logs, sensores, eventos)
 - Procesamiento incremental a medida que llegan
 - Latencia baja (segundos) vs batch (horas/días)
- **Structured Streaming proporciona una API unificada**
 - Misma API que en batch (DataFrames/SQL)
 - Spark SQL se encarga de la gestión del streaming
 - Garantías de consistencia y tolerancia a fallos
- **Para monitorización en tiempo real, alertas, ETL continuo**



- **Spark DStream (legacy)**
 - Basado en RDDs y micro-batching rígido
 - Opera a nivel de tiempo de proceso
 - API de más bajo nivel
- **Structured Streaming (recomendado)**
 - Basado en DataFrames/Datasets
 - Soporte nativo en tiempo de evento
 - Optimizado por Catalyst Optimizer
- **Se trabajará con Spark Structured Streaming**



- **Structured Streaming tiene dos modos**
- **Micro-batch (por defecto)**
 - Ejecuta el stream como una secuencia de micro-batches
 - Compatibilidad con la mayoría de operadores y destinos
 - Latencia de ~ 100 ms (mínimo) a segundos
- **Continuous processing (experimental)**
 - Pensado para latencias muy bajas
 - Menos operadores soportados y más restricciones
 - Latencia de ~ 1 ms



- **Un stream es una tabla que crece continuamente**
 - Cada evento nuevo es una fila que se añade
 - Una consulta genera resultados de forma incremental
- **Un flujo continuo de eventos es una tabla infinita**

Modelo tabular de streaming

Esquema conceptual original de la edición abierta



- **Se programa como en procesamiento en batch**
 - API SQL/DataFrame con transformaciones encadenadas
 - Misma lógica para datos finitos y para flujos continuos
- **Modelo mental: *tabla* → *query* → *salida***
 - El *stream* se interpreta como una tabla que crece
 - La query produce resultados de forma incremental
- **Enfoque declarativo**
 - Defines *qué* calcular, no los detalles de ejecución
 - Spark planifica y optimiza el *cómo*



- **Misma API que en batch**

- Operaciones habituales (`select`, `join`, `count`, ...)
- Permite reutilizar lógica batch en pipelines de streaming

- **Particularidades en streaming**

- Resultados incrementales → la salida se actualiza con nuevos datos
- Gestión de estado → agregaciones/ventanas
- Semánticas de salida → *append*, *update*, *complete*



- **Consulta declarativa sobre un stream de eventos**
 - Se expresa como una query de tipo batch (DataFrame/SQL)
 - Se evalúa incrementalmente en cada trigger con los datos nuevos
 - Spark SQL la planifica y optimiza (Catalyst + Tungsten)
- **Los resultados se escriben en un destino de salida**
- **Ejemplos son resultados que se actualizan en cada trigger**



Ejemplo de conteo continuo

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Componentes**
- 3 Infraestructura
- 4 Ejemplos



Arquitectura de Structured Streaming

Esquema conceptual original de la edición abierta

Entrada → Transformaciones (sin/con estado) → Salida



Flujo de Structured Streaming

Esquema conceptual original de la edición abierta



- **SparkSession es el punto de entrada**
 - Las entradas (sources) se leen con `spark.readStream`
 - Las salidas (sinks) se escriben con `df.writeStream`
- **Del mismo stream se pueden ejecutar múltiples queries**
 - Query 1 → resultados por ventana (como posesión) para un dashboard
 - Query 2 → detecta eventos críticos (posible lesión) y los publica en Kafka



- **Lectura de streams con `spark.readStream`**
- **Soporta múltiples fuentes habituales**
 - Kafka como sistema de mensajería
 - File para nuevos ficheros en un directorio
 - Socket para pruebas rápidas (Netcat)
 - Rate para testing y benchmarking
- **El paralelismo suele venir de la fuente**
 - Como particiones de tópicos de Kafka
 - Más particiones $\Rightarrow$ más tareas en paralelo

Plataforma de eventos Kafka

Esquema conceptual original de la edición abierta



- **Control de ejecución de una query**

- `query = df.writeStream... .start()` inicia una query
- `query.awaitTermination()` bloquea hasta que termine
- `query.stop()` detiene la query (graceful stop)

- **Operación de queries en producción**

- Poner un nombre descriptivo a la query
- Un mismo job (programa) puede ejecutar múltiples queries en paralelo



- **En streaming el esquema es esencial**
 - En un stream infinito no se recomienda inferir
 - Es importante definir el esquema (JSON)
 - Esto incluye tipar los campos de eventos
- **Decisiones de parsing de eventos**
 - Registros corruptos → enviar a cola aparte o fallar
 - Evolución de esquema → versionado/validación



- **Operaciones sin recordar el pasado**

- Suelen escalar mejor → no requieren estado compartido entre micro-batches
- Algunas son `select`, `filter`, `withColumn`, `explode`, `map`, ...
- Filtrar `event_type == "SHOT"` o extraer minutos de un timestamp

- **Si no hay agrupación/ventanas/join temporal/dedup normalmente es stateless**



- **Matienen el estado entre micro-batches**
 - Recuerdan eventos anteriores
- **Tipos de operaciones con estado**
 - Agregaciones → `groupBy`, `count/sum/avg`
 - Deduplicación → `dropDuplicates`
 - Joins → `stream-static` y `stream-stream`
 - Estado custom → `mapGroupsWithState`



- **Propiedades a alto nivel**

- Estado actualizado incrementalmente entre micro-batches
- Recuperación y tolerancia a fallos mediante checkpointing
- Limpieza y estado acotado con watermarks o timeouts

- **Ejemplo → resultados por ventanas de últimos 5 minutos**



- **Stream-to-Static**

- Enriquecer un stream con datos (relativamente) estáticos
- Ejemplo → eventos del partido en streaming + datos de la plantilla

- **Stream-to-Stream**

- Correlacionar eventos entre dos streams
- Requiere tiempo de evento + watermarks + rango temporal para acotar estado
- Ejemplo → eventos del partido + biometría en streaming

- **Joins stream-stream sin cota temporal crecen el estado indefinidamente**



- **Eventos tardíos / fuera de orden**
 - Cuando ocurrió $\neq$ Cuando llegó
 - Ventanas/agregaciones usan tiempo de evento
 - El trigger marca cada cuánto se procesa
- **Watermark define la tardanza aceptada**
 - Cierra ventanas y acota estado
 - Define hasta cuándo se procesan eventos tardíos



- **Agrupaciones en intervalos de tiempo**
- **Ventanas fijas (tumbling)**
 - Bloques fijos y no superpuestos
- **Ventanas deslizantes (sliding)**
 - Bloques que se superponen
- **Ventanas de sesión (session)**
 - Separados por gap de inactividad
- **Es una columna más de agrupación**

Ventanas temporales

Esquema conceptual original de la edición abierta



Agregación por ventana

Esquema conceptual original de la edición abierta



- **Define cada cuánto se ejecuta la query para procesar nuevos datos**
 - Default → tan rápido como sea posible (micro-batch)
 - ProcessingTime → intervalo fijo (por ejemplo cada 10s)
 - AvailableNow → procesa lo disponible y se detiene
 - Continuous → sólo en modo continuous (con restricciones)



- **Define cuánto esperar un evento tardío**
 - Los eventos no siempre llegan en orden ni a tiempo
 - El GPS de un jugador puede llegar 2 minutos tarde al servidor
 - El watermark define cuánto retraso toleramos antes de cerrar una ventana
- **Su uso es fundamental para controlar el estado**
 - Un watermark de 2 minutos (tiempo de evento) acepta tardíos hasta 2 min
 - Más allá los tardíos no actualizan ventanas/estado
- **Supone un trade-off**
 - Watermark amplio → más precisión pero más memoria usada
 - Watermark corto → resultados más rápidos pero posibles eventos perdidos



- **Es habitual que lleguen eventos tardíos**

- Dentro del watermark → se incluye en el cálculo y puede corregir un resultado emitido
- Fuera del watermark → se descarta para mantener los resultados estables

- **Ejemplo → tiros a puerta en los últimos 5 minutos**

- Un sensor registra un tiro a las 18:32 pero el dato llega a las 18:34
- Con watermark de 3 minutos → se acepta y corrige del intervalo 18:30–18:35
- Con watermark de 1 minuto → llega demasiado tarde y se descarta



Gestión de datos tardíos

Esquema conceptual original de la edición abierta



- **Append escribe solo filas nuevas y finales**
 - No reescribe filas
 - En ventanas emite cuando la ventana se “cierra”
 - Útil para eventos crudos o resultados finales
- **Update escribe filas que cambian**
 - Reescribe solo las filas actualizadas desde el último trigger
 - Útil para resultados en vivo (contadores en evolución)
- **Complete reescribe la tabla completa en cada trigger**
 - Solo viable si la tabla de resultados es pequeña



Watermark en modo append

Esquema conceptual original de la edición abierta



Watermark en modo update

Esquema conceptual original de la edición abierta



- **Escritura de resultados con `writeStream`**
- **Soporta múltiples destinos habituales**
 - Sistemas de mensajería (Kafka)
 - Bases de datos SQL y NoSQL (InfluxDB)
 - Archivos como HDFS/S3 (Parquet/JSON)
 - Consola/memoria para depuración y desarrollo
 - `foreach/foreachBatch` para lógica personalizada

Visualización operativa

Esquema conceptual original de la edición abierta



- 1 Introducción
- 2 Componentes
- 3 Infraestructura**
- 4 Ejemplos



- **Eliminar duplicados en streaming**

- La fuente puede reemitir eventos (retries, reconexiones, *at-least-once*)
- Se deduplica por una clave estable (como `event_id`)
- Se usa watermark → limita cuánto tiempo recordamos



- **Las operaciones con estado consumen recursos**
 - Spark necesita recordar eventos pasados para calcular agregaciones y ventanas
 - Más claves de agrupación o ventanas más amplias $\Rightarrow$ más memoria necesaria
- **El watermark limita cuánto estado se acumula**
 - Permite a Spark olvidar ventanas ya cerradas
 - Sin watermark el estado crece indefinidamente
- **Regla práctica: agrupa por pocas claves y usa ventanas razonables**



- **El trigger controla cada cuánto se actualizan los resultados**
 - Trigger corto → datos más frescos pero más carga en el sistema
 - Trigger largo → datos menos frescos pero menos carga
- **Regla práctica: elige el trigger según la necesidad del análisis**
 - Dashboard en tiempo real → trigger de segundos
 - Reporte por minuto → trigger de 60s es suficiente



- **Spark recuerda hasta dónde llegó**
 - Guarda un punto de control tras cada micro-batch
 - Si el proceso cae, retoma desde ahí sin reprocesar datos ya calculados
 - Evita duplicados y pérdida de resultados intermedios
- **Checkpointing para recuperación tras fallos**
 - Persiste progreso y estado
 - Spark persiste logs de progreso/commits asociados a cada trigger
 - Tras un reinicio reanuda desde el último progreso consistente



- **Spark garantiza que cada evento se procesa exactamente una vez**
 - Los resultados no se duplican ni se pierden aunque el sistema falle y se reinicie
- **Para que funcione se necesitan tres condiciones**
 - La fuente permite releer datos (como Kafka)
 - Spark guarda el progreso (checkpointing)
 - El destino tolera escrituras duplicadas sin problemas



- 1 Introducción
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos**



- Se recomienda despliegue local / Docker para pruebas
- **Requisitos de instalación en despliegue local de Spark**
 - Java instalado (`java --version`) y configurar `JAVA_HOME`
 - Instalar PySpark y crear una sesión local con todos los núcleos `[*]`

Terminal Shell Terminal

```
java --version           # verificar Java
apt install -y openjdk-21-jdk  # instalar Java (Colab / Linux)
pip install pyspark       # instalar PySpark
```



- Si `JAVA_HOME` no está definida hay que indicarla manualmente

Python Python Code

```
import os
import pyspark

# En Debian/Ubuntu con version 21 de Java:
# os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-21-openjdk-amd64"
# En Windows con version 21 de Java:
# os.environ["JAVA_HOME"] = "C:\\Program Files\\Java\\jdk-21"

print(os.environ["JAVA_HOME"]) # comprobar si esta definida
print(pyspark.__version__)
```



- Jupyter + PySpark ya instalados y configurados

Terminal Shell Terminal

```
docker run \  
  --network=host \  
  -e JUPYTER_TOKEN="MY_SECRET" \  
  -v ${PWD}:/home/jovyan/work \  
  jupyter/pyspark-notebook
```



```
services:
  jupyter:
    image: jupyter/pyspark-notebook
    network_mode: host
    environment:
      - JUPYTER_TOKEN=MY_SECRET
    restart: on-failure
    volumes:
      - ./:/home/jovyan/work
```



❶ Leer líneas Spark desde un socket en el puerto 9999

- Cada línea representa un evento de un partido de fútbol
- Pueden ser goal, shot, foul y pass

❷ Filtrar solo eventos válidos y normalizar cada línea

❸ Agregar el conteo acumulado por tipo de evento

❹ Escribir los resultados en consola



- **Paso 0 - Simular un flujo de tipos de eventos de un partido de fútbol**
 - Socket con Netcat para enviar eventos de prueba
 - En Windows instalar Nmap (<https://nmap.org/download>)

Terminal Shell Terminal

En local

```
nc -lv 9999
```

En docker

```
docker run --rm -it --init -p 9999:9999 madrueno/nc -lv 9999
```




• Paso 1 - Iniciar una sesión de Spark Structured Streaming

Python Python Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, trim

spark = (
    SparkSession.builder                # Construye la sesión de Spark
    .appName("FootballStreamingEventCount") # Nombre descriptivo
    .getOrCreate()                      # Crea o reutiliza la sesión existente
)
```



- Paso 2 - Leer líneas de eventos desde el socket abierto

Python Python Code

```
lines = (  
    spark.readStream                # Lee un stream de una fuente  
    .format("socket")              # Fuente de tipo socket  
    .option("host", "localhost")  # Host del socket  
    .option("port", 9999)         # Puerto del socket  
    .load()                       # Carga como DataFrame  
)
```



• Paso 3 - Normalizar texto y filtrar eventos válidos

- Convertir a minúsculas y eliminar espacios
- Filtrar solo los tipos de eventos de interés

Python Python Code

```
valid = ["goal", "shot", "foul", "pass"]           # Tipos de eventos válidos

events = (
    lines                                           # DataFrame (string por defecto)
    .select(trim(lower(col("value"))).alias("event")) # Normaliza texto
    .where(col("event").isin(valid))               # Filtra solo eventos válidos
)
```



● Paso 4 - Agregar el conteo acumulado por tipo de evento

- Agrupa por tipo y cuenta el número de eventos por tipo
- La query no se ejecuta hasta que se escriba la salida

Python Python Code

```
counts = (  
    events                # DataFrame con eventos  
    .groupBy("event")     # Agrupa por tipo de evento  
    .count()              # Cuenta el número de eventos por tipo  
)
```



- Paso 5 - Escribir resultados continuamente en consola

Python Python Code

```
query = (  
    counts.writeStream          # Define el destino de salida  
        .outputMode("complete") # Muestra la tabla completa de conteos  
        .format("console")      # Escribe en consola  
        .start()                # Inicia la query de streaming  
)  
  
query.awaitTermination()      # Espera hasta que se detenga la query
```



• Entradas de prueba enviadas por netcat

- Eventos con timestamp, equipo, tipo de evento y xG
- Se define un esquema explícito para parsear el JSON
- El campo ts permite aplicar ventanas temporales y watermark

Terminal Shell Terminal

```
{"ts": "2025-06-01 10:00:10", "team": "RMA", "event": "goal", "xg": 0.9}  
{"ts": "2025-06-01 10:00:40", "team": "FCB", "event": "shot", "xg": 0.2}  
{"ts": "2025-06-01 10:02:55", "team": "RMA", "event": "shot", "xg": 0.3}  
{"ts": "2025-06-01 09:45:40", "team": "FCB", "event": "shot", "xg": 0.2}
```



Python Python Code

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, TimestampType
)

spark = (
    SparkSession.builder
    .appName("FootballStreamingGoalEvents")
    .getOrCreate()
)
```



- **Definir el esquema explícito del evento JSON**

- Cada línea del socket es un objeto JSON con 4 campos

Python Python Code

```
schema = StructType([
    StructField("ts",    TimestampType()), # Timestamp del evento
    StructField("team",  StringType()),     # Equipo que realizó la acción
    StructField("event", StringType()),     # Tipo de evento (goal, shot, etc.)
    StructField("xg",    DoubleType()),     # Goles esperados del evento
])
```




- **Leer y parsear eventos JSON desde el socket**
 - Cada línea recibida se parsea con el esquema definido

Python Python Code

```
events = (  
    spark.readStream  
        .format("socket")  
        .option("host", "localhost")  
        .option("port", 9999)  
        .load() # Columna "value" con el JSON crudo  
        .select(F.from_json("value", schema).alias("e")) # Parsea JSON con el schema  
        .select("e.*") # Desempaqueta los campos  
)
```



- **Aplicar watermark y agregar en ventanas**

- Tomamos ventanas de 1 minuto a modo de ejemplo
- El watermark descarta eventos que llegan con más de 2 minutos de retraso
- La ventana fija agrupa en bloques de 1 minuto según el timestamp del evento

Python Python Code

```
counts = (  
    events  
    .filter(F.col("event").isin("goal", "shot")) # Filtra solo acciones de gol  
    .withWatermark("ts", "5 minutes")           # Descarta eventos muy tardíos  
    .groupBy(F.window("ts", "1 minute"), "team") # Ventana de 1 min por equipo  
    .agg(F.count("*").alias("total"))           # Cuenta acciones por ventana  
)
```



- **Escribir resultados en consola con trigger periódico**

- El trigger actualiza los resultados cada 10 segundos

Python Python Code

```
query = (  
    counts.writeStream(  
        .outputMode("update")           # Emite resultados actualizados  
        .format("console")  
        .trigger(processingTime="10 seconds") # Ejecuta un micro-batch cada 10 s  
        .start()  
    )  
  
    query.awaitTermination()
```



- Enviar eventos con timestamps desordenados

Terminal Shell Terminal

Batch 1 de eventos

```
{"ts": "2025-06-01 10:00:10", "team": "RMA", "event": "goal", "xg": 0.9}  
{"ts": "2025-06-01 10:00:40", "team": "FCB", "event": "shot", "xg": 0.2}  
{"ts": "2025-06-01 10:02:55", "team": "RMA", "event": "shot", "xg": 0.3}
```

Terminal Shell Terminal

Evento tardío (llega después pero ocurrió antes)

```
{"ts": "2025-06-01 09:45:40", "team": "FCB", "event": "shot", "xg": 0.2}
```



- **Spark Structured Streaming para análisis en tiempo real**
 - Unifica batch y streaming con un modelo declarativo
 - Actualiza resultados incrementalmente y gestiona estado, tardíos
- **Checklist práctico de desarrollo**
 - Establecer fuente y definir el esquema de eventos
 - Identificar tiempo de evento y configurar watermarks/ventanas
 - Escoger destino y modo de escritura según necesidad
 - Definir trigger para controlar frescura de resultados
 - Ejecutar `start()` y `awaitTermination()`



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

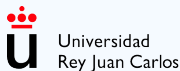
Big Data Processing II

Tema 3: Optimización y Escalabilidad de Pipelines Deportivos

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Conocer **buenas prácticas** de diseño de pipelines de análisis de datos
- Identificar técnicas de **optimización y escalabilidad** en Kafka y Spark
- Reconocer opciones de infraestructura en producción **on-premises y cloud**



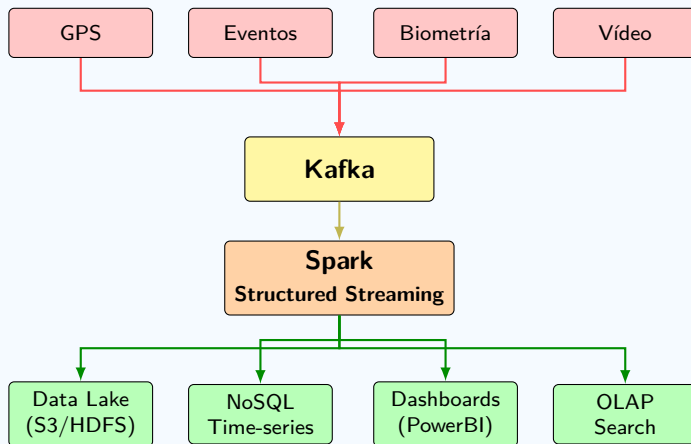
- 1 **Introducción**
- 2 **Buenas prácticas**
- 3 **Puesta en producción**



- 1 **Introducción**
- 2 Buenas prácticas
- 3 Puesta en producción



- **Múltiples fuentes de datos en vivo** → Como tracking, sensores, eventos y vídeo
- **Kafka para mensajería distribuida** → Bus de ingestión y retención de eventos
- **Spark para análisis de datos** → Tanto en batch como en streaming
- **Múltiples salidas operativas** → Como data lakes, BBDD y dashboards





Arquitectura distribuida de referencia

Esquema conceptual original de la edición abierta

Fuente: <https://www.scaler.com/topics/spark-streaming-with-kafka/>



- **Rapidez en el análisis**

- Respuesta en tiempo real durante el partido
- Alertas y apoyo a decisiones en vivo

- **Capacidad y fiabilidad del sistema**

- Procesamiento de grandes volúmenes de datos deportivos
- Integración de dispositivos tracking, eventos y sensores
- Funcionamiento estable en momentos críticos de competición

- **Los objetivos marcan la arquitectura y los recursos**

- SLA/SLO claros para guiar decisiones
- Latencia objetivo y disponibilidad esperada



- **El cuello de botella se desplaza entre componentes**

- Optimizar un componente no garantiza mejorar todo el pipeline
- Al mejorar una parte la limitación suele aparecer en otra
- La optimización debe evaluarse a nivel global

- **La monitorización es imprescindible**

- Comparar métricas antes y después de cada cambio
- Monitorizar retrasos en Kafka y tiempos en Spark
- Vigilar uso de recursos como disco, red y memoria



- **Kafka escala mejor cuando**

- Hay suficientes particiones para paralelizar producción y consumo
- Se añaden brokers para repartir carga y almacenamiento
- La infraestructura ofrece buena red y discos rápidos

- **Spark escala mejor cuando**

- Hay suficiente particionado para el paralelismo de tareas
- Se asignan executors y memoria adecuados
- Se reducen los shuffles innecesarios



- 1 Introducción
- 2 Buenas prácticas**
- 3 Puesta en producción



- **Gobierno y estabilidad del dato**

- Mantener un esquema estable en pipelines críticos
- Definir reglas de evolución para evitar cambios incompatibles

- **Organización del almacenamiento**

- Elegir un particionado coherente con las consultas más frecuentes
- Compactar periódicamente para mejorar el rendimiento
- Un buen diseño mejora el rendimiento de lectura y escritura

- **Diseño según coste, latencia y tipo de procesamiento**

- En streaming se priorizan baja latencia y estabilidad operativa
- En batch se priorizan throughput y coste por volumen procesado
- Separar datos recientes (calientes) de históricos (fríos) ayuda a equilibrar ambos



- **Un buen particionado reparte la carga del pipeline**
 - Nº de particiones en Kafka = paralelismo máximo de consumo
 - En Spark cada partición se procesa como una tarea
 - Si hay pocas particiones el clúster queda infrautilizado
- **Mantiene consistencia en los datos**
 - Agrupa eventos relacionados en la misma partición
 - Conserva el orden dentro de cada partición de Kafka
- **La clave debe tener suficiente granularidad**
 - Claves generales concentran demasiada carga en una partición
 - Clave más granulares mejoran el reparto y el rendimiento
 - Ejemplos son `player_id` o `match_id`



- **Rendimiento es un equilibrio entre throughput, latencia y resiliencia**
 - Depende del tamaño del mensaje, compresión y acks
 - Compresión reduce tráfico de red pero añade coste de CPU
 - acks=all y mayor replicación mejoran durabilidad pero aumentan latencia
- **La infraestructura condiciona el resultado**
 - Kafka es intensivo en entrada/salida → conviene usar discos rápidos
 - La red debe ser estable y con suficiente capacidad para picos de tráfico
 - Es buena práctica separar discos de datos y distribuir réplicas entre racks o zonas



- **Número de particiones**

- Aumentan paralelismo y throughput
- Demasiadas añaden overhead

- **Factores de replicación**

- Aumenta resiliencia
- Aumenta el uso de disco y red

- **Retención y compactación del log**

- Retención de mensajes → definir según su uso
- Compactación del log → mantiene último estado por clave



Replicación en Kafka

Esquema conceptual original de la edición abierta

Fuente: <https://developer.confluent.io/courses/architecture/data-replication/>



- **Batching para agrupar mensajes antes de enviar**
 - Definir tiempo de espera para acumular más mensajes
 - Definir tamaño máximo del batch de mensajes (bytes)
 - Ajustes típicos son `linger.ms` y `batch.size`
 - Más throughput a costa de algo más de latencia de envío
- **Compresión de mensajes para reducir tráfico de red y disco**
 - Distintos algoritmos de compresión con diferentes trade-offs
 - Elegir según el tamaño de los mensajes y la CPU disponible
- **Fiabilidad para no perder eventos críticos**
 - Definir el nivel de confirmación de escritura (`acks`)
 - Activar idempotencia para evitar duplicados por reintentos
 - Más fiabilidad suele implicar más latencia de escritura



- **Consumer groups para escalado horizontal**
 - Añadir consumidores al mismo grupo aumenta el paralelismo
 - El máximo útil es 1 consumidor por partición (extra quedan ociosos)
- **Commits de offset para controlar el progreso**
 - Los commits automáticos simplifican la configuración
 - Los commits manuales dan más control sobre cuándo confirmar
- **Reintentos y DLQs para gestionar errores**
 - Definir una política de reintentos antes de descartar mensajes
 - Enviar a una DLQ los eventos que fallan repetidamente
- **Monitorizar retrasos en consumidores para detectar cuellos de botella**



Coordinación de consumidores

Esquema conceptual original de la edición abierta

Fuente: <https://developer.confluent.io/courses/architecture/consumer-group-protocol/>



- **Particiones, tareas y executors están relacionados**
 - Spark divide los datos en particiones lógicas para procesarlos en paralelo
 - Cada partición suele ejecutarse como una tarea independiente
 - Los executors son los procesos que ejecutan esas tareas en los nodos del clúster
- **El shuffle redistribuye datos entre nodos**
 - Es habitual en operaciones como join, groupBy y agregaciones
 - Suele ser una de las fases más costosas por uso de red, disco y tiempo
- **Las particiones impactan en el rendimiento**
 - Si hay pocas particiones se desaprovecha el clúster
 - Si hay demasiadas aumenta la sobrecarga de gestión
 - Si los executors tienen pocos recursos el procesamiento se ralentiza



- **Las particiones marcan el paralelismo**

- Cada partición suele ejecutarse como una tarea por etapa
- Conviene ajustar su número a la capacidad real del clúster
- El objetivo es equilibrar paralelismo y sobrecarga

- **El desbalanceo de particiones es un enemigo del rendimiento**

- Aparece cuando pocas particiones concentran muchos datos (skew)
- Algunas tareas terminan rápido y otras tardan mucho más
- El rendimiento global queda limitado por la partición más grande
- Primero se corrige el skew y después se ajusta el número de particiones

- **Decidir entre repartir o compactar particiones**

- `repartition` → redistribuye datos con *shuffle*
- `coalesce` → reduce particiones con menos coste (normalmente sin *shuffle*)



- **En skew hay tareas que dominan el tiempo total del job**
 - Pocas tareas lentas dominan el job (stragglers)
 - Uso intensivo de disco mientras otros executors quedan ociosos
- **Es clave comparar la duración de tareas en Spark**
 - Si hay grandes diferencias entre tareas, suele haber desbalanceo
 - Revisar también el tamaño de particiones tras joins y agregaciones
- **Existen diferentes técnicas para mitigar el skew**
 - Repartir mejor la carga (claves más granulares o subclaves)
 - Tratar por separado casos con volumen anómalo
 - Usar AQE para rebalanceo automático



Distribución de datos con skew

Esquema conceptual original de la edición abierta



Distribución equilibrada

Esquema conceptual original de la edición abierta

Fuente: <https://blog.scottlogic.com/2018/03/22/apache-spark-performance.html>



- **El shuffle es la operación más costosa**
 - Implica serialización, red y escritura a disco intermedio
 - Suele dominar el tiempo de ejecución en joins y agregaciones
- **Reducir datos antes del shuffle**
 - Filtrar filas y proyectar solo las columnas necesarias
 - Preagregar cuando sea posible antes de un join
 - Menos datos en movimiento implica menos coste y menor latencia
- **Usar broadcast join con tablas pequeñas**
 - Si una tabla cabe en memoria, se copia a los executors
 - Así se evita redistribuir la tabla grande
 - Genera menos shuffle y joins más rápidos



Operación de shuffle

Esquema conceptual original de la edición abierta

Fuente: <https://blog.scottlogic.com/2018/03/22/apache-spark-performance.html>



- **Guarda datos intermedios para reutilizarlos**
 - Solo compensa cuando esos datos se usan varias veces
 - Si un resultado se usa una sola vez suele empeorar el rendimiento
- **La cache ocupa memoria y compite con la ejecución de tareas**
 - Si no hay memoria suficiente aumenta el uso de disco y baja el rendimiento
 - Conviene priorizar datasets intermedios costosos y estables
- **Buenas prácticas de uso**
 - Cachear después de transformaciones costosas que se reutilizan
 - Liberar el cache cuando deja de ser necesario y validar impacto con métricas



- **Persistir extiende la idea de cache**

- El cache es una forma de persistencia en memoria
- La persistencia permite elegir cómo guardar los datos
- Se usa cuando el mismo DataFrame se reutiliza y el volumen puede ser alto

- **Existen varios niveles de persistencia**

- MEMORY_ONLY → más rápido si los datos caben en memoria
- MEMORY_AND_DISK → más robusto cuando el volumen es grande

- **Checkpointing para análisis en streaming**

- Guarda el estado del pipeline para recuperarse tras fallos
- Es clave en ventanas temporales, deduplicación y joins con estado
- Debe guardarse en almacenamiento fiable (HDFS, S3 o equivalente)



- 1 Introducción
- 2 Buenas prácticas
- 3 Puesta en producción**



- **Elegir el modelo de despliegue**

- Bare metal prioriza rendimiento y control de la infraestructura
- Máquinas virtuales ofrecen más flexibilidad con algo de sobrecarga
- Kubernetes facilita la operación pero exige cuidar el almacenamiento persistente

- **Kafka depende mucho del rendimiento de disco y de red**

- Conviene usar almacenamiento rápido para los logs
- También es importante una red estable entre brokers

- **Es fundamental asegurar la continuidad del servicio**

- Se recomiendan al menos 3 brokers y factor de replicación 3 en tópicos críticos
- Distribuir réplicas en distintos nodos o racks mejora la disponibilidad
- KRaft simplifica el despliegue frente a ZooKeeper

- **Dimensionar almacenamiento y límites de mensajes**



- **Conviene elegir el gestor del clúster según el entorno**
 - YARN encaja bien en ecosistemas Hadoop
 - Kubernetes facilita orquestación, aislamiento y elasticidad
 - Spark standalone suele reservarse para desarrollo y pruebas
- **El almacenamiento condiciona el rendimiento y la operación**
 - HDFS es una opción sólida para cargas batch
 - Checkpointing debe guardarse en almacenamiento fiable
- **Es importante separar cargas y fijar dependencias**
 - Conviene separar recursos de batch y streaming
 - Streaming necesita mayor prioridad para mantener la latencia
 - Docker ayuda a mantener dependencias y versiones estables



- **Conviene elegir entre servicio administrado y control directo**
 - Amazon MSK es Kafka administrado por AWS y reduce trabajo de operación
 - MSK Serverless ajusta recursos automáticamente cuando la carga cambia
 - Kafka en EC2 da más control de configuración pero exige más operación
- **La red y el acceso se controlan dentro de AWS**
- **Seguridad y monitorización deben configurarse desde el inicio**
 - IAM puede gestionar autenticación y autorización de clientes
 - CloudWatch centraliza métricas para monitorizar brokers, tópicos y consumo



- **Conviene elegir la plataforma según el tipo de carga**
 - Amazon EMR encaja bien en batch y en streaming
 - Spark sobre EKS aporta flexibilidad y encaje con contenedores
 - Databricks en AWS simplifica desarrollo y operación
- **En AWS Spark se suele lanzar como jobs**
 - Como EMR Steps, spark-submit en EKS o Jobs en Databricks
- **Es importante diferenciar el uso en streaming y en batch**
 - Un flujo habitual conecta MSK o Kinesis con Spark Structured Streaming
 - En streaming se priorizan latencia y estabilidad del pipeline
 - En batch se priorizan throughput y coste por volumen procesado



- **La monitorización debe centrarse en la salud del pipeline**
 - El retraso del consumidor es la métrica principal para detectar retraso
 - También conviene seguir la tasa de producción y consumo por tópico
 - La latencia de extremo a extremo ayuda a comprobar si se cumplen los objetivos
- **La replicación y el disco son claves para la estabilidad**
 - Problemas en el ISR pueden señalar riesgo de pérdida o menor disponibilidad
 - El uso de disco por broker debe vigilarse de forma continua
- **Las alertas y herramientas deben adaptarse al entorno**
 - Configurar alertas por consumer lag, timeouts y errores de broker
 - En on-premises es habitual usar Prometheus y Grafana
 - En AWS se suele usar CloudWatch y métricas de MSK



Observabilidad de Kafka

Esquema conceptual original de la edición abierta

Fuente: <https://grafana.com/blog/get-comprehensive-monitoring-for-your-apache-kafka-ecosystem-instances-quickly-with-grafana-cloud/>



- **Spark Web UI permite localizar cuellos de botella**
 - En la vista de *stages* se ve qué etapas concentran más tiempo
 - Dentro de cada etapa las *tasks* permiten comparar duraciones y detectar skew
 - Las métricas de *shuffle read/write* muestran cuánto dato se mueve entre nodos
- **En streaming hay que vigilar el ritmo del pipeline**
 - La duración del micro-batch debe ser menor que el intervalo del trigger
 - Comparar datos que entran y datos que se procesan ayuda a ver retrasos
 - Los registros pendientes muestran si se está acumulando trabajo
 - Medir tasa de input y procesamiento, duración del batch y watermark
- **Las alertas deben avisar antes de que el pipeline falle**
 - Alertar si aumenta el uso de disco o el tiempo de limpieza de memoria
 - Alertar de inmediato ante jobs fallidos con contexto del error



Interfaz de monitorización de Spark

Esquema conceptual original de la edición abierta

Fuente: <https://spark.apache.org/docs/latest/web-ui.html>



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Prompt Engineering & Post-training

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos



Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Contexto



- Entender qué es un LLM desde el punto de vista computacional.
- Comprender su papel en la Ciencia de Datos Deportivos.
- Aprender a controlar su comportamiento mediante Prompt Engineering.
- Identificar límites del prompting.



- Modelo probabilístico de generación de texto.
- Aprende patrones lingüísticos.
- Predice la siguiente palabra dada una secuencia.
- Simula coherencia, no conocimiento real.

Arquitectura conceptual de un modelo de lenguaje

Esquema conceptual original de la edición abierta



- **No es una base de datos.**
- **No tiene acceso** a sensores o APIs.
- **No realiza cálculos fiables** por sí mismo.
- **No distingue verdad de plausibilidad.**

Datos deportivos y modelos generativos

Esquema conceptual original de la edición abierta



- Generar narrativas.
- Estandarizar informes.
- Traducir estadísticas a lenguaje humano.
- Interfaz conversacional.

Ciclo de diseño y evaluación

Esquema conceptual original de la edición abierta

Prompt Engineering



Un LLM no se “usa”, se **configura mediante lenguaje**.

- El prompt es el mecanismo con el que definimos:
 - Qué tarea debe realizar.
 - Qué información puede usar.
 - Qué formato debe producir.
- En un sistema de datos, el prompt actúa como:
 - Capa de control sobre el comportamiento del modelo.
 - Sustituto de una interfaz de programación tradicional.
- El modelo no cambia internamente → solo cambia su comportamiento en esa interacción.

Idea clave: Prompt Engineering = programación en lenguaje natural.



Un error común es tratar al LLM como si fuera un buscador o una persona.

- **Pregunta informal:**

“Resume el partido de ayer.”

- Ambigua.
- Sin contexto.
- Sin criterio de calidad.

- **Especificación de tarea:**

*“Genera un resumen técnico de máximo 5 líneas usando SOLO las métricas dadas.
Estructura: contexto, acciones clave, impacto táctico.”*

Diferencia fundamental: Las preguntas producen texto. Las especificaciones producen resultados utilizables en un sistema.



Un prompt profesional no es una frase, es una estructura:

- **Rol (Role)** Define el perfil desde el que debe responder el modelo. Ej.: “Eres analista de rendimiento en un club profesional”.
- **Tarea (Task)** Especifica exactamente qué debe hacer. Ej.: “Genera un informe técnico del jugador X”.
- **Contexto (Context)** Información disponible para realizar la tarea. Ej.: estadísticas, partido, periodo, posición del jugador.
- **Restricciones (Constraints)** Limita el comportamiento del modelo. Ej.: “No inventes datos”, “Usa solo las métricas dadas”.
- **Formato de salida (Output Format)** Cómo debe entregarse la respuesta. Ej.: JSON, tabla, bullets, informe estructurado.

Idea clave: un buen prompt reduce grados de libertad del modelo.



Prompt: *“Analiza el rendimiento del jugador.”*

- **Ambigüedad de objetivo:** ¿rendimiento ofensivo, defensivo, físico, táctico?
- **Sin contexto:** no hay periodo (partido, mes, temporada) ni rival ni rol.
- **Sin datos:** el modelo tenderá a **inventar** métricas o asumirlas.
- **Sin criterio:** no defines qué significa “bueno/malo” (percentiles, umbrales).
- **Sin formato:** imposible integrar la salida en un pipeline (ETL/dashboard).

Diagnóstico Big Data: prompt mal definido = salida no reutilizable + riesgo alto de alucinación.



Prompt (plantilla):

- **Rol:** “Eres analista de rendimiento en un club profesional”.
- **Tarea:** “Genera un informe técnico del jugador X”.
- **Contexto:** “Usa SOLO las métricas proporcionadas (tabla adjunta)”.
- **Restricciones:** “No inventes datos; si falta una métrica, marca null”.
- **Formato:** “Devuelve JSON con: resumen, puntos fuertes, puntos débiles, alertas”.

Efecto: reduces grados de libertad y conviertes la interacción en una “especificación”.



Entrada (ejemplo): métricas por 90 minutos (últimos 5 partidos)

- minutes=432, goals=2, assists=1, xG=1.6, xA=0.9
- shots_p90=2.8, key_passes_p90=1.7, passes_acc=84%
- pressures_p90=14.2, tackles_p90=1.1, interceptions_p90=0.8
- distance_km=10.9, sprints=21, high_intensity_runs=52

Nota: esto es intencionalmente “poco” para forzar que el prompt gestione faltantes sin inventar.



Objetivo: que la salida sea consumible por procesos automáticos (API/ETL/dashboard)

```
{
  "player": "X",
  "period": "last_5_matches",
  "summary": "... (3-5 lineas)",
  "strengths": ["...", "..."],
  "weaknesses": ["...", "..."],
  "alerts": [{"type": "fatigue_risk", "evidence": "..."}],
  "metrics_used": ["minutes", "xG", "xA", "pressures_p90"],
  "missing_metrics": ["duels_won_pct", "progressive_passes_p90"]
}
```

Clave: el modelo no “se luce”; produce un contrato de datos.



- **Trazabilidad:** puedes auditar qué campos rellena y cuáles faltan.
- **Evaluación automática:** validación de esquema (JSON schema), checks.
- **Integración:** dashboard/BI, storage, pipelines batch/stream.
- **Reproducibilidad:** mismo prompt + mismos datos → salida comparable.
- **Mitigación de alucinación:** forzar null/unknown evita inventar.

Regla de oro: si lo vas a usar en un sistema, **no pidas texto libre**.



- Un prompt operativo funciona como un **contrato** entre:
 - **Entrada** (datos disponibles) y **Salida** (estructura/criterios).
 - **Responsabilidades** del modelo (qué puede inferir vs qué no).
- En un proyecto real debes:
 - **Versionar** prompts (v1, v1.1...) igual que código.
 - Registrar: modelo, temperatura, prompt, y cambios de esquema.
 - Mantener tests con ejemplos (golden set) para evitar regresiones.

Puente a técnicas: para consistencia, entra Zero/One/Few-shot.



- Un prompt largo no es “mejor”: puede introducir contradicciones o ruido.
- Buen prompt = **instrucciones cortas + estructura + ejemplos mínimos**.
- Coste típico en sistemas: tokens (entrada + salida) $\Rightarrow$ dinero + latencia.
- Estrategia práctica:
 - Plantillas reutilizables.
 - Contexto **solo** necesario.
 - Ejemplos (few-shot) compactos.
 - Formato fijo para minimizar longitud de salida.

Mensaje para el máster: Prompt engineering también es **optimización de sistema**.

Técnicas de prompting



- El modelo resuelve la tarea solo con instrucciones.
- Se apoya en su conocimiento general aprendido en pretraining.
- Ventajas:
 - Bajo coste en tokens.
 - Rápido de diseñar.
 - Flexible ante tareas variadas.
- Limitaciones:
 - Menor consistencia de formato.
 - Más variabilidad en la respuesta.



Tarea: Clasificar tipo de evento táctico.

Prompt:

“Clasifica el siguiente evento como: transición ofensiva, ataque posicional o contraataque: ‘Recuperación en campo propio, pase largo al espacio, finalización en 8 segundos.’”

Observación: funciona, pero puede variar la explicación y formato.



- Se añade un ejemplo de entrada–salida correcta.
- El modelo imita patrón, estructura y nivel de detalle.
- Útil para:
 - Informes de scouting.
 - Clasificaciones consistentes.
 - Resúmenes con plantilla fija.



Ejemplo dado al modelo:

Entrada: métricas de un jugador.

Salida: informe estructurado con fortalezas, debilidades y alerta física.

Luego se pide lo mismo para otro jugador.

Efecto: se estandariza la estructura sin entrenar el modelo.



- Varios ejemplos refuerzan el patrón deseado.
- Reduce ambigüedad semántica.
- Mejora estabilidad del formato.
- Especialmente útil cuando:
 - hay terminología técnica
 - la tarea es subjetiva (evaluaciones)

Aprendizaje mediante ejemplos en contexto

Esquema conceptual original de la edición abierta



Comparación zero-shot, one-shot y few-shot

Esquema conceptual original de la edición abierta



- Obliga al modelo a generar pasos intermedios.
- Aumenta probabilidad de razonamientos coherentes.
- Reduce errores en tareas lógicas y causales.
- No es razonamiento real, es secuencialización probabilística.

Descomposición explícita de una tarea

Esquema conceptual original de la edición abierta



Pregunta: ¿Por qué bajó el rendimiento ofensivo?

Modelo debe razonar:

- Reducción de minutos.
- Menos acciones en último tercio.
- Aumento de carga física.



CoT es útil cuando la tarea requiere descomposición lógica o causal:

- **Diagnóstico de rendimiento deportivo** Identificar causas de variaciones en métricas (minutos, carga, acciones).
- **Evaluación táctica** Analizar secuencias de juego paso a paso para inferir decisiones tácticas.
- **Explicaciones causales** Relacionar eventos: fatiga → menor intensidad → menos acciones decisivas.
- **Razonamiento multi-variable** Cuando intervienen estadísticas físicas, técnicas y contextuales.

Efecto: fuerza al modelo a estructurar el proceso antes de la conclusión.



Aunque mejora la coherencia, CoT introduce efectos adversos importantes:

- **Aumento del coste computacional** Generar pasos intermedios incrementa tokens de salida → mayor latencia y coste en sistemas en producción.
- **Ilusión de racionalidad** El modelo puede producir razonamientos estructurados pero incorrectos, dando una falsa sensación de validez.
- **Propagación de errores** Un fallo en un paso intermedio contamina el resto del razonamiento.
- **Mayor exposición a alucinaciones** Cuanto más texto genera el modelo, más oportunidades tiene de introducir información no fundamentada.
- **No aporta verificación factual** CoT mejora forma, no veracidad. No sustituye RAG ni validación externa.

Conclusión: CoT mejora estructura del razonamiento, pero no garantiza corrección.

Errores comunes en Prompt Engineering



La ambigüedad introduce variabilidad no controlada en la salida del modelo.

- No se define con precisión la **tarea objetivo**.
- El modelo debe inferir intención → introduce suposiciones.
- Aumenta la **varianza entre ejecuciones**.
- Complica la evaluación automática y la comparación de resultados.

Impacto en sistemas: salida impredecible → difícil integrar en pipelines. **Ejemplo:** “Analiza el partido” → ¿rendimiento físico, táctica, psicología?



El modelo no detecta “lagunas”; las rellena probabilísticamente.

- Si faltan datos, el modelo usa conocimiento general.
- Aumenta riesgo de **alucinaciones plausibles**.
- Se diluye la frontera entre información real e inferida.
- En deporte: puede inventar estadísticas, roles o situaciones.

Regla operativa: lo que no está en el prompt no forma parte de la realidad del sistema.



Sin formato definido, la salida es texto narrativo no estructurado.

- No se puede validar automáticamente.
- Difícil de almacenar y consultar.
- No integrable en pipelines Big Data.
- Alta variabilidad de estructura entre ejecuciones.

Impacto técnico: se rompe la trazabilidad del sistema. **Solución:** definir siempre esquema de salida (JSON, tabla, claves fijas).



Un prompt multiobjetivo diluye el foco del modelo.

- Clasificar + explicar + evaluar en un solo prompt.
- El modelo reparte capacidad entre subtareas.
- Respuestas más superficiales y menos precisas.
- Mayor probabilidad de incoherencias internas.

Principio de diseño: un prompt → una tarea principal.



La coherencia lingüística no implica veracidad.

- Los LLMs son modelos probabilísticos, no sistemas de verificación.
- Pueden producir respuestas seguras pero incorrectas.
- Mayor riesgo cuando la respuesta parece bien argumentada.
- En deporte: decisiones basadas en datos inventados pueden ser críticas.

Requisito de sistema: validación humana o cruzada con fuentes reales.



Un prompt no evaluado es un componente no controlado del sistema.

- Sin métricas no se puede optimizar ni comparar versiones.
- Los cambios en prompts afectan al comportamiento global del sistema.
- La calidad percibida puede no reflejar estabilidad real.

Resultado: deriva del sistema sin que se detecte.



Un prompt debe evaluarse como cualquier otro componente software.

- Consistencia del formato (JSON válido, claves presentes).
- Tasa de campos vacíos o incorrectos.
- Variabilidad entre ejecuciones con la misma entrada.
- Coherencia con datos reales.
- Estabilidad tras cambios de modelo o temperatura.

Objetivo: minimizar varianza y maximizar fiabilidad operativa.

Post-training de LLMs



- Controla formato y tarea.
- Reduce ambigüedad.
- Mejora consistencia.
- Pero no cambia lo que el modelo sabe ni cómo está alineado.

Por eso existe el post-training.



Un modelo tras el pretraining:

- Ha aprendido a predecir texto.
- Conoce patrones lingüísticos.
- Tiene conocimiento general.

Pero:

- No está optimizado para seguir instrucciones.
- No está alineado con expectativas humanas.
- No está diseñado para ser útil como asistente.



- Conjunto de técnicas que se aplican tras el entrenamiento base.
- Objetivo: modificar el **comportamiento** del modelo.
- No enseña nuevo lenguaje, enseña **cómo responder**.

Idea clave: transforma un generador de texto en un asistente usable.



Ciclo de post-entrenamiento

Esquema conceptual original de la edición abierta



- Que el modelo siga instrucciones.
- Mejorar claridad y estructura de respuestas.
- Controlar tono y estilo.
- Reducir respuestas peligrosas o inapropiadas.
- Enseñar cuándo abstenerse de responder.



- Ajuste con ejemplos humanos correctos.
- El modelo imita respuestas bien estructuradas.
- Introduce estilo conversacional.

Limitación: solo aprende por imitación, no por preferencias.



- Variante de SFT centrada en seguir instrucciones.
- Se entrena con muchas tareas distintas.
- Mejora versatilidad del modelo.

Resultado: menos necesidad de prompts complejos.



- Reinforcement Learning from Human Feedback.
- Humanos comparan respuestas del modelo.
- El modelo aprende qué respuestas son preferidas.

Efecto: mayor naturalidad y alineación con expectativas humanas.



- RLAIIF: feedback generado por modelos IA.
- DPO: optimización directa por preferencias.
- Más simples y escalables que RLHF clásico.



- Adaptación a áreas específicas (medicina, derecho, deporte).
- Mejora precisión terminológica.
- Respuestas más especializadas.

Riesgo: pérdida de generalidad y sesgos de dominio.



- El modelo:
 - Sigue mejor instrucciones.
 - Es más conversacional.
 - Es más prudente.
- Pero:
 - Sigue siendo probabilístico.
 - Sigue alucinando.
 - No tiene acceso a datos externos.



- Prompting = control externo.
- Post-training = comportamiento interno.

Metáfora:

- Post-training educa al modelo.
- Prompting le da instrucciones concretas.



- Alineación imperfecta.
- Alucinaciones persistentes.
- Puede parecer seguro sin ser correcto.
- No sustituye acceso a datos reales.



- Modelos demasiado conservadores.
- Respuestas genéricas y seguras.
- Sesgos heredados del feedback humano.
- Puede priorizar parecer correcto sobre ser útil.



- Un LLM no es una fuente de conocimiento.
- El prompt es una herramienta de ingeniería.
- Las técnicas de prompting mejoran control, no verdad.
- El comportamiento base del modelo viene del post-training.
- Para trabajar con datos reales necesitaremos RAG.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Retrieval-Augmented Generation (RAG)

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Introducción



Un LLM modela una distribución condicional:

$$P(y \mid x)$$

donde:

- x = prompt + contexto disponible
- y = secuencia generada

Pero:

- No tiene acceso directo a bases de datos externas.
- No consulta sistemas internos.
- No distingue entre dato real y patrón aprendido.

Conclusión: conocimiento aprendido $\neq$ acceso a información actualizada.



El Prompt Engineering:

- Controla estructura.
- Reduce ambigüedad.
- Define formato.

Pero:

Prompting $\nrightarrow$ Acceso a datos externos

No modifica la base informacional del modelo.

Arquitectura general de recuperación aumentada

Esquema conceptual original de la edición abierta



Un sistema real dispone de:

- Tracking físico por partido.
- Métricas avanzadas (xG, PPDA, presiones).
- Informes médicos.
- Observaciones de scouting.

Pregunta crítica:

¿Ha disminuido la intensidad de presión tras acumular más de 300 minutos en 10 días?

Un LLM aislado no puede responder correctamente.

Definición de RAG



Retrieval-Augmented Generation introduce un mecanismo externo:

$$Output = f(Prompt + Retrieve(Query))$$

- Retrieve(Query) devuelve fragmentos relevantes.
- El LLM genera condicionado por ese contexto.

No cambia parámetros del modelo. Cambia la evidencia disponible en inferencia.



Componente	Rol
Embeddings	Representación semántica
Vector DB	Indexación eficiente
Retriever	Selección de contexto
LLM	Síntesis condicionada

RAG es una arquitectura distribuida, no una simple extensión del modelo.

Flujo de recuperación y generación

Esquema conceptual original de la edición abierta

Diseño técnico de RAG



Dividir documentos implica:

- Elegir **tamaño de fragmento**.
- Definir **solapamiento**.
- Preservar **coherencia semántica**.

Errores comunes:

- Chunk demasiado grande → baja precisión.
- Chunk demasiado pequeño → pérdida de contexto.
- Solapamiento excesivo → redundancia y coste.

Estrategias de fragmentación

Esquema conceptual original de la edición abierta



Un embedding es una función:

$$E : \text{Texto} \rightarrow \mathbb{R}^d$$

Permite medir similitud semántica.

Limitaciones:

- No captura bien relaciones numéricas exactas.
- Puede perder contexto temporal.
- Similaridad $\neq$ relevancia estadística.



Similitud típica:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Pero:

- Similaridad semántica no garantiza pertinencia.
- Sensible a redacción del query.

La calidad del retrieval es crítica.

Representación vectorial

Esquema conceptual original de la edición abierta



Un sistema robusto debe permitir:

- Filtrado por temporada, jugador, competición.
- Indexación por tipo de documento.
- Actualización incremental.

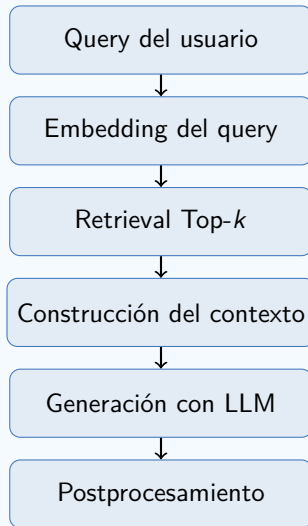
Sin metadatos, el retrieval pierde precisión contextual.

Caso de recuperación aplicado al fútbol

Esquema conceptual original de la edición abierta



Cada etapa introduce posible error.





Veracidad $\uparrow \Rightarrow$ *Coste* $\uparrow$

- Más contexto $\rightarrow$ más tokens.
- Más tokens $\rightarrow$ mayor latencia.
- Mayor latencia $\rightarrow$ menor escalabilidad.

Diseñar **RAG** es **optimizar** este equilibrio.

Compromisos entre recuperación y generación

Esquema conceptual original de la edición abierta

Aplicaciones en deporte



Base interna:

- Métricas por partido.
- Indicadores avanzados.
- Carga física acumulada.

Consulta:

Compara evolución de presión alta con descanso < 72h.

Requiere filtrado + recuperación + síntesis.

Pipeline RAG

Esquema conceptual original de la edición abierta



Documentos:

- Informes técnicos.
- Observaciones cualitativas.

Permite:

- Búsqueda semántica.
- Detección de patrones recurrentes.
- Síntesis multi-documento.

Ejemplo de consulta con contexto deportivo

Esquema conceptual original de la edición abierta



Problema:

Embeddings no gestionan bien números exactos.

Solución:

- **Recuperación textual** (búsqueda semántica en RAG).
- **Cálculo determinista externo** (fórmulas programadas).
- **Inyección de resultados estructurados** (creación de tablas numéricas).

Arquitectura híbrida = mayor precisión.

Limitaciones



RAG introduce dos fuentes de error:

$$Error_{total} = Error_{retrieval} + Error_{generation}$$

Un mal retrieval no se compensa con mejor modelo.

Fuentes de error en RAG

Esquema conceptual original de la edición abierta



Si el documento contiene error:

- **El modelo lo amplifica.**
- **No detecta contradicciones internas.**
- **No valida consistencia factual.**

RAG mejora acceso, no garantiza verdad.

Evaluación



- Precision@k
- Recall@k
- Mean Reciprocal Rank (MRR)
- nDCG

Sin evaluar retrieval, no se puede optimizar el sistema.

Evaluación del flujo RAG

Esquema conceptual original de la edición abierta



Se debe medir:

- **Coherencia final.**
- **Consistencia con fuente.**
- **Estabilidad entre ejecuciones.**
- **Trazabilidad** (citar fragmentos).

RAG sin trazabilidad es **opaco** y sin estabilidad **no es confiable**.

Conclusiones



En datos dinámicos, RAG suele ser preferible.

Fine-tuning	RAG
Cambia parámetros	No cambia parámetros
Integra conocimiento	Consulta conocimiento
Coste alto inicial	Coste por consulta
Riesgo de olvido	Modelo intacto



Es una arquitectura de **acceso a información**, no un mecanismo de aprendizaje.

- RAG no hace al modelo más inteligente.
- El sistema se especializa en un contexto.
- Incrementa el tamaño del prompt.
- Afecta al número de tokens utilizados.

Siguiente paso: arquitecturas agénticas.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

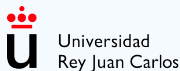
IA Generativa aplicada al deporte

Sistemas basados en agentes

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Introducción



Un LLM tradicional:

$$\text{Output} = f(\text{prompt})$$

Un agente:

$$\text{acción}_{t+1} = f(\text{estado}_t, \text{objetivo}, \text{observación}_t)$$

- No solo responde.
- Decide.
- Actúa.
- Aprende del entorno.

Arquitectura de un agente

Esquema conceptual original de la edición abierta



LLM + RAG permiten:

- Responder preguntas.
- Sintetizar información.

Pero no permiten:

- Ejecutar **cálculos complejos**.
- **Consultar** múltiples sistemas.
- **Tomar decisiones** secuenciales.
- **Adaptarse dinámicamente**.

Integración de RAG y agentes

Esquema conceptual original de la edición abierta

Skills y tools



Un agente se compone de **habilidades (skills)**:

- consultar bases de datos
- ejecutar modelos estadísticos
- calcular métricas avanzadas
- generar visualizaciones
- enviar alertas automáticas

Un skill es una **capacidad reutilizable**.

Relación entre agentes, skills y tools

Esquema conceptual original de la edición abierta



Tool	Skill
Función aislada	Capacidad compuesta
Ejecuta acción	Orquesta acciones
Ej: query DB	Ej: análisis rendimiento



Ejemplos en deporte:

- API de tracking físico.
- Motor de cálculo de xG (goles esperados).
- Base histórica de rendimiento.
- Generador de dashboards.

El valor del agente crece con su ecosistema.

Agente de inteligencia artificial

Esquema conceptual original de la edición abierta

Planificación y razonamiento iterativo



Los agentes complejos no actúan en un solo paso.

Proceso:

- 1 **Analizar objetivo.**
- 2 **Descomponer subtareas.**
- 3 **Ejecutar pasos.**
- 4 **Evaluar resultados.**
- 5 **Continuar o finalizar.**

Flujo de decisión agéntico

Esquema conceptual original de la edición abierta



Arquitectura **ReAct**:

- **Pensamiento** intermedio.
- **Selección** de acción.
- **Ejecución** de tool.
- **Observación** del resultado.
- **Iteración**.

Combina **razonamiento y acción**.

Patrón ReAct

Esquema conceptual original de la edición abierta



Arquitectura **moderna**:

- **Planner**: diseña los pasos
- **Executor**: ejecuta herramientas
- **Evaluator**: valida resultados

Permite tareas complejas multi-etapa.

Orquestación de agentes

Esquema conceptual original de la edición abierta

Arquitecturas dirigidas por estado (control de flujo)



Los flujos lineales (chains) fallan cuando:

- La ruta depende del contenido (distintos casos de uso).
- Hay pasos opcionales (solo si faltan datos, solo si hay anomalía).
- Hay iteración (refinar consulta, reintentar retrieval, validar).
- Se necesitan **invariantes** (no continuar si el JSON no valida).

Necesidad: control explícito del flujo y de las condiciones de parada.



Una forma rigurosa de diseñar agentes es modelarlos como:

$$S \xrightarrow{\text{evento/condición}} S'$$

- **Estados** = etapas del proceso (retrieve, compute, write-report, validate).
- **Transiciones** = reglas (si falta dato → consultar DB; si inconsistente → reintentar).
- **Guardas** = condiciones que evitan errores (no generar si no hay evidencias).

Ventaja: control del sistema > “improvisación del LLM”.

Estado y memoria

Esquema conceptual original de la edición abierta



Patrones típicos en agentes:

- **Router**: elegir herramienta según intención (consulta histórica vs cálculo).
- **Validator**: comprobar esquema/consistencia antes de seguir.
- **Retry with backoff**: reintentos controlados ante fallos de tool/timeout.
- **Loop bounded**: iteración con límite de pasos para evitar loops infinitos.

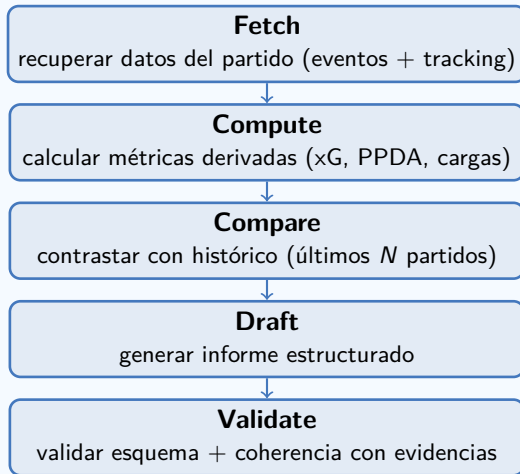
Idea clave: la robustez viene del control de flujo, no del “prompt perfecto”.

Patrones de arquitectura agéntica

Esquema conceptual original de la edición abierta



Estados de alto nivel:



Routing: si faltan métricas → volver a Fetch; si el JSON falla → volver a Draft.



- **Loops infinitos:** imponer límite de iteraciones y condiciones de parada.
- **Tool misuse:** validación de argumentos + esquemas estrictos.
- **Saltos incorrectos:** transiciones solo si se cumplen guardas.
- **Acumulación de latencia:** cortar ramas no necesarias y cachear resultados.

Conclusión: el estado gobierna el flujo; el LLM decide dentro de límites.

Memoria y estado



Estado:

- Representación del **proceso actual** (qué sabemos ahora y qué falta).
- Vive dentro de la ejecución (por ejemplo, un diccionario/JSON de trabajo).

Memoria:

- Persistencia entre ejecuciones: historial, preferencias, perfiles, conocimiento externo.
- Vive fuera del flujo principal (DB, vector store, logs, cache).

Problema habitual: mezclar ambos genera agentes incoherentes y difíciles de depurar.



- **Conversacional (corto plazo):** historial reciente (context window).
- **Episódica:** interacciones previas resumidas (resúmenes por sesión).
- **Semántica:** documentos y conocimiento recuperable (RAG).
- **Estructurada:** perfiles, preferencias, configuraciones (tablas/JSON persistente).

Idea clave: cada tipo tiene coste, latencia y riesgos distintos.

Razonamiento y planificación

Esquema conceptual original de la edición abierta



Preguntas que deben tener respuesta:

- **Qué se guarda** (señal vs ruido).
- **Cuándo se guarda** (al final de tarea, tras validación).
- **Cómo se recupera** (por claves, por embeddings, por filtros).
- **Caducidad** (TTL): lo viejo puede perjudicar.
- **Auditoría**: logs trazables para explicar decisiones.

Objetivo: memoria útil, no acumulación de texto.

Modelo cognitivo de un agente

Esquema conceptual original de la edición abierta



Memoria estructurada: perfil del jugador (posición, rol, minutos, lesiones).

Memoria semántica: informes textuales de scouting y clips anotados.

Memoria episódica: decisiones previas del agente (por qué recomendó A vs B).

Consulta:

“Dame 3 perfiles similares a este extremo, justificando con evidencias de informes.”

Resultado: respuestas consistentes porque el agente recuerda y cita.



- **Contaminación** (guardar alucinaciones)
→ guardar solo tras validación.
- **Obsolescencia** → TTL y versionado (temporada, fecha, fuente).
- **Privacidad/seguridad** → control de acceso + minimización de datos.
- **Sesgo por historiales** → resumir y normalizar, no copiar todo.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Riesgos de memoria persistente

Esquema conceptual original de la edición abierta

Riesgos y fallos reales en agentes



A mayor autonomía del agente:

- mayor número de componentes,
- mayor superficie de fallo,
- mayor dificultad de depuración.

Los fallos rara vez provienen del modelo aislado.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Complejidad de un sistema agéntico

Esquema conceptual original de la edición abierta



Problemas frecuentes:

- iteraciones sin condición de parada,
- reintentos infinitos de retrieval,
- planificación redundante.

Mitigación:

- límites de iteración,
- condiciones de parada explícitas,
- control del estado.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Bucle de percepción y acción

Esquema conceptual original de la edición abierta



El agente puede:

- invocar tools equivocadas,
- usar parámetros inválidos,
- interpretar mal los resultados.

Mitigación:

- esquemas de entrada estrictos,
- validación automática,
- manejo robusto de errores.

Herramientas externas

Esquema conceptual original de la edición abierta



Riesgo crítico:

- el sistema produce resultados plausibles pero incorrectos.

Soluciones:

- Logging detallado,
- Trazabilidad de evidencias,
- Auditoría de decisiones.

Sin trazabilidad, el sistema no es confiable.

Coordinación multiagente

Esquema conceptual original de la edición abierta



Los agentes pueden:

- ejecutar múltiples herramientas,
- generar grandes cantidades de tokens,
- realizar múltiples iteraciones.

Resultado:

- latencias elevadas,
- costes operativos altos.

Optimizar flujo es esencial.

Control de costes

Esquema conceptual original de la edición abierta



Si los datos son incorrectos:

- el agente amplifica errores,
- las conclusiones pueden ser erróneas,
- aumenta el riesgo en decisiones deportivas.

Regla: garbage in → garbage out.

Evaluación de calidad

Esquema conceptual original de la edición abierta



- Validar entradas y salidas.
- Limitar iteraciones y costes.
- Registrar decisiones y evidencias.
- Diseñar flujos controlados por estado.
- Mantener supervisión humana.

Caso deportivo de ejemplo

Esquema conceptual original de la edición abierta

Conclusión: robustez > autonomía.

Conclusiones



1. Un agente no es un LLM con más prompts

- Es un sistema que percibe, decide y actúa.
- El LLM es el motor de decisión, no el sistema completo.

2. El valor real surge de integrar herramientas y datos

- Los agentes delegan cálculos y consultas a tools externas.
- La precisión proviene de combinar IA generativa y procesos deterministas.

3. El control del flujo y del estado determina la robustez

- Arquitecturas dirigidas por estado evitan comportamientos erráticos.
- Sin control del flujo aparecen loops, errores y costes excesivos.

4. La memoria y la trazabilidad permiten sistemas fiables

- Registrar evidencias y decisiones es esencial para confiar en el sistema.

Conclusión: los agentes no añaden “inteligencia mágica”; permiten construir sistemas que toman decisiones útiles en entornos reales.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Tema 5: Proyectos integrados

Del dato en tiempo real al análisis automatizado

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Abril 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición del análisis retrospectivo (BI tradicional) a la toma de decisiones prescriptiva en tiempo real.
- Diseñar infraestructuras integradas combinando ingestión masiva (Kafka) y cómputo continuo (Spark).
- Desplegar sistemas de IA generativa y ecosistemas multiagente sobre flujos de datos deportivos.
- Implementar arquitecturas RAG multimodales para la comprensión espacio-temporal del juego.
- Automatizar la generación de informes y narrativas adaptadas a diferentes perfiles de stakeholders.



- 1 **Introducción: El paradigma del análisis automatizado**
- 2 **Arquitecturas completas de ingestión y procesamiento**
- 3 **Orquestación de IA Generativa y flujos agénticos**
- 4 **RAG Multimodal y Generación de Narrativas**
- 5 **Visualización avanzada e interfaces conversacionales**
- 6 **Conclusiones**

Introducción al paradigma automatizado



- La ciencia de datos deportivos ya no puede depender exclusivamente de los informes post-partido.
- El valor del dato decae exponencialmente desde el momento en que ocurre la acción.
- **Objetivo moderno:** Proveer a los entrenadores y analistas de herramientas que sugieran ajustes tácticos durante el desarrollo de la competición.
- Se exige una transición arquitectónica hacia la analítica predictiva y automatizada.



La infraestructura actual requiere la convergencia de tres pilares:

- **Ingestión continua:** Captura ininterrumpida de biometría, GPS y vídeo.
- **Motor determinista:** Cálculos estadísticos en ventanas temporales.
- **Capa cognitiva (IA):** Modelos de lenguaje que interpretan y traducen matemáticas en táctica.

Esta convergencia permite pasar de paneles de control estáticos a "**sistemas observacionales**" inteligentes.



- Un LLM tradicional es una "foto fija" probabilística basada en su entrenamiento.
- **El problema en el deporte:** Carece de contexto del estado actual del mundo; no sabe quién está fatigado ni cuál es el marcador en el minuto 75.
- **Solución:** Integrar al LLM como un motor de razonamiento (*reasoning engine*) dentro de un pipeline de datos alimentado en vivo.



- La automatización total abarca desde la captura del evento micro-táctico hasta la narrativa macroscópica.
- Requiere superar los límites epistemológicos de los enfoques batch.
- El contexto inmediato permite extraer valor competitivo en la ventana crítica de la toma de decisiones.



- 1 **Fuentes (Productores):** Tracking óptico, sensores de balones, wearables.
- 2 **Middleware de Mensajería:** Enrutamiento masivo y ordenado.
- 3 **Motor de Streaming:** Agregación e identificación de anomalías.
- 4 **Agentes LLM / RAG:** Generación de diagnósticos e informes.
- 5 **Destinos (Consumidores):** Aplicaciones de cuerpo técnico, retransmisiones (broadcasters).

Arquitecturas completas de ingestión



- Entornos de alta presión (estadios repletos) con conectividad variable.
- **Teorema CAP aplicado al deporte:**
 - Consistencia (C), Disponibilidad (A), Tolerancia a Particiones (P).
- En un partido en directo, la latencia es crítica.
- Generalmente, se prioriza la **Disponibilidad** y la **Tolerancia a fallos** de red (AP) sobre la consistencia matemática perfecta que llegaría tarde.



- Apache Kafka actúa como el núcleo de mensajería (Pub/Sub).
- **Desacoplamiento:** Los sensores (visión, GPS) no saben quién consume los datos.
- **Ventaja:** Permite que múltiples sistemas (motores de cálculo de xG, sistemas de IA generativa para alertas tácticas y bases de datos históricas) consuman el mismo evento de forma independiente y concurrente.



- Los proveedores de datos (Opta, StatsBomb) o estándares de la FIFA (EPTS) envían flujos jerárquicos.
- **Granularidad:** Un simple "pase" incluye decenas de metadatos asociados.
- La comprensión de estos esquemas JSON es vital para el diseño de bases de datos vectoriales y algoritmos analíticos.



Campo (Opta/EPTS)	Tipo	Implicación Analítica
event_id	UUID	Deduplicación semántica (exactly-once).
type_id	Entero	Filtrado en capa de ingesta (separar pases de tiros).
start_x, start_y	Flotante	Coordenadas que alimentan modelos espaciales.
timestamp	Time	Reloj del evento, crítico para latencias.
qualifiers	JSON	Contexto profundo (presión, pie débil) para enriquecer prompts de la IA.



- **Paralelismo horizontal:** El número de particiones define el rendimiento máximo.

$$\text{Particiones} = \frac{\text{Throughput total deseado}}{\text{Velocidad máxima del consumidor}}$$

- **Eficiencia:** Afinación de `batch.size` y `linger.ms` combinados con compresión (Snappy/Gzip) para reducir la carga de red.
- **Resiliencia:** Uso de `acks=all` y `enable.idempotence=true` para garantizar que ningún dato crítico (como una alerta de fatiga) se pierda o duplique.



- Se trata al flujo ininterrumpido de eventos como una tabla relacional en crecimiento perpetuo.
- Permite a los científicos de datos abstraerse de la complejidad subyacente.
- Motor de cálculo incremental para preparar métricas (ej. Posesión en el último tercio) antes de enviarlas al modelo de IA generativa.



- En un estadio, los dispositivos se desconectan y reconectan continuamente.
- Spark gestiona la semántica exactly-once usando:
 - **Write-Ahead Logs (WAL)** en sistemas distribuidos persistentes (HDFS/S3).
 - **Checkpointing distribuido:** Guarda el progreso de manera asíncrona.
- Asegura que el LLM no reciba alertas duplicadas sobre el mismo evento de juego.



- **Problema de asincronía:** El retraso entre que el jugador hace el sprint y el paquete de datos llega al servidor en la nube.
- **Watermarking:** Umbral temporal de gracia que el sistema espera antes de cerrar el cálculo de una ventana.

$$W(t) = T_{max}(t) - \Delta$$

- Garantiza que la IA analice ventanas de tiempo completas sin colapsar la memoria del servidor.



- **Desventaja de Lambda:** Mantener código dual (Speed y Batch) duplica el riesgo de error algorítmico.
- **Ventaja de Kappa para la IA:** Kafka persiste todo el historial; el motor de streaming reprocesa todo desde cero si cambia un modelo subyacente.
- Garantiza que el agente cognitivo de IA reciba distribuciones de datos y métricas idénticas, ya sea auditando un partido de 2024 o analizando el directo.

IA Generativa y flujos agénticos



- La interacción ya no es reactiva (humano preguntando a una máquina).
- Se avanza hacia arquitecturas **agénticas y orquestadas orientadas a la acción** (Agentic Workflows).
- Un agente no solo resume texto, sino que ejecuta código, consulta bases de datos vectoriales y planifica respuestas estructuradas.



- LangChain funciona como el marco orquestador.
- Interconecta el razonamiento del LLM con las capacidades deterministas de Big Data (Spark, bases de datos SQL).
- Aporta "herramientas" (Tools) al LLM para que este no invente números, sino que los busque activamente en los clústeres de datos.



Uso práctico: Un analista interroga al sistema en lenguaje natural.

- 1 El LLM usa su herramienta integrada para traducir la consulta humana a un código de consulta (Spark SQL).
- 2 Ejecuta la consulta contra el DataFrame en memoria.
- 3 Extrae el resultado numérico.
- 4 Redacta una explicación en base a ese dato duro.



Pregunta del entrenador: "¿Tasa de éxito de pase progresivo del mediocentro bajo presión en los últimos 15 min?"

- **Planner:** Descompone la consulta → Entidad (jugador), Métrica (tasa de pase), Contexto (presión alta basada en coordenadas), Filtro Temporal (15 min).
- **Executor:** Corre el SQL en Spark.
- **Synthesizer:** Valida lógicamente el 45% obtenido y genera la alerta narrativa argumentada.



- La automatización proactiva que monitorea patrones complejos sin requerir un prompt humano.
- Se utilizan máquinas de estado explícitas para enrutar decisiones.
- Compañías como **Machina Sports** ya proveen infraestructuras de IA agéntica enfocadas exclusivamente en el ecosistema deportivo, dotando a la IA de conocimiento táctico profundo y ejecución fiable.



El sistema opera en un ciclo de iteración continuo:

- ① **Think:** "Detecto una anomalía en la velocidad del jugador."
- ② **Act:** "Usaré la herramienta de historial médico para ver sus promedios."
- ③ **Observe:** "La recuperación cardíaca ha bajado un 15% respecto al mes pasado."
- ④ **Act:** "Emitiré una alerta de sustitución al cuerpo técnico."



Para garantizar estabilidad computacional frente a alucinaciones:

- **Router:** Desvía consultas puramente estadísticas hacia motores deterministas y consultas cualitativas hacia RAG.
- **Validator:** Obliga al agente a parsear su salida bajo un formato estricto (JSON), repitiendo el proceso si el formato es erróneo.
- **Loop Bounded:** Impone un máximo de "pensamientos" iterativos para evitar que el sistema agote los tokens en un bucle infinito de resolución.

RAG Multimodal y Generación de Narrativas



- Los datos tabulares solo representan una fracción de la realidad del partido.
- **RAG Multimodal:** Integrar simbióticamente flujos de vídeo HD, audio de retransmisión y telemetría estructurada.
- **Vectorización combinada:** Permite a la IA no solo buscar "tiros a puerta", sino buscar "jugadas con presión asfixiante" utilizando comprensión espaciotemporal.



- Sistemas de visión profunda analizan fotogramas y detectan la topología del juego y trayectorias.
- **Arquitecturas avanzadas (ej. GameSight):** Realizan un alineamiento en dos etapas, identificando primero las entidades visuales anónimas en el vídeo para luego enriquecerlas con metadatos y estadísticas históricas.
- Esto mejora drásticamente la calidad de los comentarios automatizados.
- Erradica la disonancia de contexto en los modelos de lenguaje genéricos.



- **Problema fundamental:** Inyectar payloads JSON crudos al contexto del LLM es ineficiente y provoca pérdida de precisión matemática (alucinaciones).
- **Solución:** Preprocesamiento determinista.
- Traducir diccionarios numéricos (`type_id = 10`, `outcome = 0`) a oraciones semánticas estructuradas antes del prompt: "El jugador X intentó una entrada pero no recuperó la posesión".

Cada vez es menos necesaria esta conversión, pero no es deseable introducir JSONs muy grandes.



- Creación de pipelines cerrados de monitorización continua de eventos para generar contenido automatizado y filtrado.
- Sistemas como **LLM-SportTweet** utilizan orquestación para leer flujos de eventos asíncronos y tomar decisiones editoriales determinando qué sucesos merecen ser convertidos en narrativas.



Antes de generar una alerta o informe, el LLM evalúa y puntúa internamente el evento táctico o la noticia:

- **Proximity:** Relevancia para el deporte/equipo analizado.
- **Freshness:** Criticidad e inmediatez del suceso.
- **Impact:** Grado de influencia en el resultado del partido.
- **Uniqueness:** Penalización de la redundancia frente a la memoria episódica a corto plazo.



- Solo si el evento supera el umbral de Scoring, el agente desencadena la generación.
- **Técnicas Few-Shot:** Forzar al modelo a adoptar un formato retórico y una longitud específica.
- Se adapta el vocabulario automáticamente:
 - **Modo Analista:** Foco estadístico en métricas de presión.
 - **Modo Aficionado:** Foco emocional y contexto histórico para redes sociales.



- La generación de narrativas en directo exige reducir la latencia al mínimo.
- Se investigan soluciones híbridas que acoplan redes neuronales convolucionales con LLMs.
- **Ejemplo:** Arquitecturas OS-CNN pre-computan y anticipan eventos cinemáticos que luego se inyectan en plantillas Prompt Engineering para modelos como LLaMA 3.3, logrando comentarios fluidos en tiempo real con latencias aptas para retransmisiones interactivas.



La fiabilidad estricta no es opcional en la élite.

Principio de Separación (Separation of Concerns):

- El LLM jamás computa matemáticas (complejas).
- El motor Spark realiza las agregaciones.
- El marco RAG localiza el dato duro.
- El LLM ingiere el dato protegido bajo el mandato estricto de "Utilice EXCLUSIVAMENTE los parámetros de la matriz".

Visualización Avanzada e Interfaces



- Los petabytes de datos son inútiles sin una democratización efectiva de los insights.
- **El paradigma 2026:** Las predicciones y gráficas estáticas previas al partido son obsoletas.
- Se demandan probabilidades recalibradas continuamente en función del flujo de eventos (e.g., probabilidades de micro-sucesos) y total transparencia analítica.



- **Cuerpo Técnico:** Resolución analítica máxima; fusión de cinemática de alta frecuencia y fisiología para prevención biomecánica de lesiones.
- **Front Office:** Proyecciones a largo plazo (impacto económico, modelado de carreras de atletas).
- **Broadcasters:** Baja latencia, inmersión rápida. Respuestas dinámicas a escenarios de tipo "What-If" para los espectadores.



- Sustitución de los paneles pasivos (Tableau, Power BI clásicos) por ecosistemas interactivos.
- La IA funciona como "científico de datos virtual permanente".
- Permite a los analistas realizar consultas declarativas profundas (e.g., "Compara nuestro índice de progresión central antes y después de la sustitución del minuto 60").



- **Problema de los Black-Boxes:** Los cuerpos técnicos no confían en índices predictivos opacos.
- La visualización debe incluir **Virtual Analyst Narratives**.
- El sistema no solo emite una probabilidad del 73%, sino que el agente detalla causalidades (e.g., métricas previas, tendencias posicionales) para justificar el número mostrado en pantalla.



- ① **Recepción:** El agente capta el pico en la gráfica.
- ② **Consulta interna:** Formula SQL y consulta el stream de datos en segundo plano.
- ③ **Recuperación RAG:** Acopla métricas actuales con cortes de vídeo del partido en curso.
- ④ **Generación de la narrativa:** Ofrece una síntesis causal explicativa en lenguaje natural para acompañar al gráfico.



- **Time-to-Insight:** Métrica fundacional en el desarrollo de paneles intra-partido.
- **Problema:** La decodificación del modelo generativo añade un retraso inherente.
- **Solución (Streaming UI):** Reducir el Time To First Token (TTFT) inyectando la respuesta palabra por palabra a la interfaz (vía WebSockets).
- Se mitiga la percepción de lentitud del usuario final de forma drástica.



- Durante la competición táctica, la fluidez del panel de control es primordial.
- Frente al teorema CAP, el diseño del panel favorece la tolerancia a la partición y la disponibilidad.
- Es preferible renderizar tendencias estimadas al instante que bloquear el dashboard a la espera de la consistencia total del evento cerrado.

Conclusiones



- El procesamiento de Big Data moderno consolida el streaming continuo con el análisis cognitivo.
- La orquestación agéntica y los RAG multimodales son las claves para dar contexto espacio-temporal a la inteligencia artificial.
- La arquitectura debe mitigar sistemáticamente las alucinaciones limitando el acceso de los modelos a datos precomputados deterministas.
- El futuro radica en visualizaciones conversacionales y proactivas que empoderen a todos los stakeholders del entorno deportivo en tiempo real.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Big Data Processing II

IA Generativa aplicada al deporte

Tema 6: Tendencias y Futuro

Procesamiento Masivo de Datos e IA en el Deporte

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Abril 2026





Material docente en abierto de la Universidad Rey Juan Carlos

Asignatura: Big Data Processing II

Titulación: Máster Universitario en Análisis de Datos Deportivos

Autores: Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández

Fecha: 2026

© 2026 Equipo docente. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional.

Los logotipos, marcas y referencias de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Depósito institucional: <https://burjcdigital.urjc.es>



- Comprender la transición de la analítica retrospectiva a la prescriptiva en el deporte.
- Explorar tecnologías emergentes actuales: Edge Computing, 5G, Visión Artificial.
- Analizar implicaciones estratégicas y de negocio de la analítica prescriptiva.
- Evaluar tecnologías del futuro próximo: computación cuántica, blockchain, BCI, datos sintéticos.
- Examinar los retos éticos, legales y de gobernanza de datos biométricos.



- 1 **Introducción: De la observación retrospectiva a la prescripción estratégica**
- 2 **Convergencia de infraestructura: El estadio de latencia cero**
- 3 **Visión artificial y desmaterialización de la telemetría**
- 4 **Analítica prescriptiva, orquestación multiagente y automatización**
- 5 **Tecnologías emergentes: Más allá del horizonte actual**
- 6 **Retos éticos, legales y gobernanza de datos**

Introducción: De la observación retrospectiva a la prescripción estratégica



- **Pasado:** Análisis descriptivo post-partido (batch processing).
- **Presente:** Procesamiento en tiempo real con Kafka y Spark Streaming.
- **Futuro:** Analítica prescriptiva que recomienda acciones óptimas automáticamente.
- Transición de "¿qué pasó?" → "¿por qué pasó?" → "¿qué debemos hacer?"
- **Desafío:** El valor del dato decae exponencialmente desde el momento del evento.
- **Objetivo:** Insights accionables durante la competición, no después.



Predictiva

- Anticipa probabilidades
- "¿Cuál es el riesgo de lesión?"
- Requiere interpretación humana

Prescriptiva

- Recomendando acciones
- "Sustituye al jugador X por Y en el minuto 65"
- Evalúa escenarios concurrentes
- Maximiza probabilidad de éxito



- Valoración proyectada: **3.400 millones de dólares para 2028.**
- Impulsores clave:
 - Reducción drástica de costes de computación para IA
 - Comercialización de derechos NIL (Nombre, Imagen, Semejanza)
 - Obsolescencia de cookies de terceros
 - Necesidad de captura de datos propios
- La tecnología pasa de soporte a **núcleo estratégico del negocio.**



- **Juegos Olímpicos de Invierno Milán 2026**
- **Copa Mundial FIFA 2026**
- Actúan como bancos de pruebas de tecnologías disruptivas
- Demuestran que eficiencia, personalización, arbitraje y equidad dependen de infraestructuras convergentes

Convergencia de infraestructura: El estadio de latencia cero



- Excelente para análisis retrospectivo
- Transmitir telemetría de 60.000 espectadores + biosensores de jugadores a centros remotos introduce latencias insalvables
- Retardo incompatible con decisiones intra-partido

Solución: Multi-access Edge Computing (MEC) + Redes 5G Privadas (P5G)



MEC (Edge Computing)

- Cómputo en el estadio
- Latencia: cientos ms → 1 dígito
- Análisis antes de que concluya jugada
- CV opera localmente

Redes 5G Privadas

- Carril dedicado
- Network Slicing
- Evita congestión Wi-Fi
- Ancho de banda garantizado



Protocolos críticos

- **mMTC:** Miles de IoT concurrentes
- **URLLC:** Latencia ultra-baja, alertas médicas

Resultado: Procesamiento distribuido con latencia mínima y ancho de banda optimizado.

Arquitectura federada

- Nodos en estadio
- Agregación continua
- Filtrado local
- Nube: histórico + training



- Redes locales + procesamiento en el borde
- Zonas VIP con gafas de realidad aumentada
- Superposición de estadísticas y mapas de calor en vivo
- Sincronización con acción física sin retraso perceptible

Visión artificial y desmaterialización de la telemetría



Wearables tradicionales

- GPS, acelerómetros
- Alta precisión
- Fricciones:
 - Calibración
 - Baterías
 - Incomodidad
 - Privacidad

Computer Vision (2026)

- Tendencia dominante
- Costes IA en caída
- CNN + pose esquelética
- Sin fricción física
- Escala masiva



- Identificar jugadores inequívocamente (pese a oclusiones)
- Rastrear coordenadas espaciales precisas
- Medir distancias entre líneas defensivas
- Cuantificar índice de intensidad de presión táctica (PPDA)
- Todo de forma completamente automatizada

Mercado: Crecimiento anual del 29.8%, de 2.390 M\$ a 8.700 M\$ para fin de década.



- **Copa Mundial FIFA 2026:** Colaboración FIFA-Lenovo
- Generación de modelos 3D de alta precisión mediante escaneos digitales
- No son estéticos: son la **Ground Truth** del procesamiento
- Interactúan con sistema de fuera de juego semiautomático
- Mejoran precisión al calcular momento de impacto y posición volumétrica
- Se inyectan en retransmisiones para explicar decisiones VAR



- Reducción de desenfoque por movimiento (motion blur)
- Procesamiento en tiempo real de cámaras corporales de árbitros
- Ángulos complicados mejorados algorítmicamente
- Transmisiones fluidas para revisión de jugadas

Visión artificial: de herramienta experimental a **infraestructura central**.

Analítica prescriptiva, orquestación multiagente y automatización



- Calcular trayectorias y métricas ya no es ventaja: es el **estándar mínimo**
- **Valor transformador:** Transición a ecosistemas prescriptivos
 - Descriptivo: "¿Qué pasó en el partido?"
 - Diagnóstico: "¿Por qué perdimos el control?"
 - **Prescriptivo: "¿Qué ajustes debemos implementar?"**



Propuesta de valor

- Reducción lesiones: 15-25%
- Optimización fichajes: 30-40%
- Inversión: 1-3M€
- Break-even: 18-36 meses

Factores críticos

- ✓ Patrocinio ejecutivo
- ✓ Formación staff
- ✓ Pilotos graduales
- × Resistencia cultural
- × Silos departamentales
- × Falta de talento



- **Hipótesis:** ¿La analítica prescriptiva es ventaja sostenible o commodity temporal?
- **Evidencia empírica:**
 - Liverpool FC (2018-2020): Inversión agresiva en ciencia de datos correlaciona con títulos
 - Democratización acelerada (Football AI Pro, soluciones SaaS) reduce brecha tecnológica
- **Conclusión:** Ventaja no reside en *tener* tecnología, sino en:
 - Calidad y granularidad de datos propios
 - Capacidad de integración organizacional
 - Velocidad de adaptación táctica en función de insights



Tendencia 2026: Migración de prototipos a producción (Tema 5: implementación técnica)

- Análisis táctico, búsqueda semántica, informes automatizados
- 40h análisis manual → 4h supervisión

Decisión estratégica:

Build

- Control total
- 2-5M€/año

Buy (SaaS)

- Rápido
- 200-500K€/año

Partner

- Co-desarrollo
- Riesgo compartido



- Los cuerpos técnicos rechazan "Cajas Negras"
- Un sistema efectivo no solo emite: "Cambie a defensa zonal"
- Debe acompañar evidencia trazable:
"Se recomienda cobertura zonal porque la fatiga de su lateral derecho está en percentil 90, y el rival capitaliza contraataques por esa banda en segundas partes."



Football AI Pro (FIFA 2026)

- FIFA + Lenovo
- IA generativa multilingüe
- Disponible 48 selecciones
- Democratiza analítica táctica

La brecha tecnológica se reduce: ventaja sostenible = calidad de datos + integración organizacional.

Nuevos perfiles

- LALIGA Business School
- Máster Transformación Digital
- Programa Ejecutivo IA Deporte
- Analistas Estrategia Algorítmica

Tecnologías emergentes: Más allá del horizonte actual



- **Estado actual:** Investigación experimental, no producción
- **Potencial transformador:**
 - Optimización combinatoria de alineaciones ($11! = 40\text{M}$ permutaciones)
 - Simulación de escenarios tácticos en paralelo cuántico
 - Resolución de problemas NP-hard en planificación de calendarios
- **Casos de uso explorados:**
 - IBM Quantum + Fórmula 1: Optimización de estrategia de neumáticos
 - Algoritmos variacionales (VQE) para análisis de grafos tácticos
- **Barrera:** Hardware cuántico aún limitado (qubits < 1000 , alta tasa de error)
- **Horizonte:** 2030-2035 para aplicaciones prácticas en deporte de élite



- **Problema actual:** Centralización de datos biométricos en proveedores tecnológicos
- **Propuesta blockchain:**
 - Atletas retienen propiedad criptográfica de sus datos
 - Smart contracts definen quién accede, cuándo y bajo qué condiciones
 - Registro inmutable de consentimientos y usos
- **Casos de uso en desarrollo:**
 - NFTs de firmas cinemáticas únicas (monetización NIL)
 - Marketplace descentralizado para licencias de datos
 - Auditoría transparente de cumplimiento GDPR/BIPA
- **Retos:** Escalabilidad (throughput), consumo energético, complejidad técnica



- **Más allá de wearables:** Captura directa de señales neuronales
- **Aplicaciones prospectivas:**
 - Medición de carga cognitiva en tiempo real (toma de decisiones bajo presión)
 - Detección temprana de fatiga central (antes de manifestación física)
 - Optimización de estados de flow en entrenamiento
- **Tecnologías actuales:**
 - EEG no invasivo: precisión limitada, ruido de movimiento
 - Implantes neuronales (Neuralink): invasivos, dilemas éticos
- **Horizonte:** 2028-2030 para BCI no invasivos portátiles
- **Barrera crítica:** Aceptación ética y regulación de neurotecnología



- **Problema:** Escasez de datos etiquetados + restricciones de privacidad (GDPR, BIPA)
- **Solución:** Generar datos sintéticos estadísticamente equivalentes
 - GANs (Redes Generativas Adversarias) entrenadas en datos reales
 - Producen millones de trayectorias sintéticas de jugadores
 - Preservan distribuciones estadísticas pero no identidades reales
- **Ventajas:**
 - Entrenamiento de modelos ML sin exponer datos sensibles
 - Compartir datasets entre clubes sin violar contratos
 - Aumentar diversidad de casos extremos (lesiones raras)
- **Desafío:** Garantizar que datos sintéticos no permiten re-identificación

Retos éticos, legales y gobernanza de datos



- Capturar millones de puntos biométricos, cinemáticos y tácticos confiere ventaja asimétrica
- Telemetría íntima introduce fricciones normativas sin precedentes
- Dilemas morales profundos
- **Gobernanza ética:** Pilar estructural, no requisito accesorio
- Evita que optimización degenera en vigilancia laboral encubierta



RGPD (Reglamento General de Protección de Datos):

- Dato biométrico = categoría especial, protección reforzada
- Consentimiento libre, explícito e informado obligatorio
- **Privacy-by-Design:** Privacidad desde el diseño
- **DPIA:** Evaluaciones de Impacto previas al despliegue
- Derechos de acceso, rectificación y supresión



- **BIPA (Illinois):** Uno de los estándares más estrictos
 - Políticas escritas y publicadas sobre retención
 - Destrucción obligatoria tras cumplir finalidad
 - Límite máximo: 3 años desde última interacción
- **TRAIGA (Texas) - Vigencia enero 2026:**
 - Penaliza captura ilícita de biometría
 - Prohíbe discriminación automatizada
 - Prohíbe social scoring
- **Ley de IA de Colorado:**
 - Evaluaciones de impacto periódicas
 - "Cuidado razonable" de implementaciones algorítmicas



DPA con proveedores

- Acuerdos restrictivos
- Evitar extracción subrepticia
- No usar datos para entrenar modelos sin consentimiento

Relaciones laborales

- Atleta = productor originario
- Sindicatos (NFL, MLB):
 - Bloquean uso punitivo
 - Prohíben negociación salarial
 - Consentimiento expreso
 - Derechos auditoría



- **NIL (Nombre, Imagen y Semejanza):** Expansión masiva
- Datos biomecánicos únicos = activos digitales
- Avatares 3D, firmas cinemáticas exclusivas
- **Conflicto:** Ligas empaquetan métricas para apuestas/VR sin compensar al jugador



Sesgos algorítmicos

- Datos históricos con deficiencias humanas
- Infrarrepresentación:
 - Solo ligas masculinas europeas
 - Inferencias sesgadas para otros grupos
- **Resultado:** Discriminación

Problema Caja Negra

- Sin explicación de variables
- Socava transparencia
- Etiquetado prematuro:
 - "Bajo potencial"
 - "Frágil"
 - Estigmas de mercado



- **Human-in-the-Loop (HITL):**
- IA opera como apoyo complementario
- No reemplaza evaluación integral humana
- Resultados prescriptivos requieren validación
- Combina:
 - Precisión algorítmica
 - Intuición y experiencia del especialista
 - Contexto situacional

Conclusiones



- La convergencia de Edge Computing, 5G y visión artificial redefine el ecosistema deportivo actual.
- La analítica prescriptiva es decisión estratégica de negocio, no solo técnica: ROI, madurez organizacional y ventaja competitiva determinan el éxito.
- Tecnologías emergentes (quantum, blockchain, BCI, datos sintéticos) marcarán la próxima década: preparación anticipada es clave.
- La democratización tecnológica (Football AI Pro) reduce brechas, trasladando ventaja a calidad de datos e integración organizacional.
- La gobernanza ética de datos biométricos no es opcional: es fundacional para la sostenibilidad del sistema.
- El futuro exige profesionales híbridos: dominio técnico + comprensión estratégica + ética aplicada.



- El texto, los ejemplos y los esquemas de esta edición abierta se distribuyen bajo CC BY-SA 4.0.
- Las imágenes de procedencia no acreditada de la versión de trabajo se han retirado y sustituido por esquemas conceptuales originales.
- Los logotipos y marcas citados se utilizan únicamente con finalidad identificativa y quedan excluidos de la licencia.
- Las referencias técnicas se conservan mediante citas y enlaces a sus fuentes correspondientes.
- Información de licencia: <https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Proyecto final transversal

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0
Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional, disponible en <https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.



Universidad
Rey Juan Carlos



Máster Universitario en Análisis de Datos Deportivos

Práctica final

Big Data Processing II

**Sistema de análisis deportivo en tiempo real con
Kafka, Spark, LangGraph y generación
automática de informes enriquecidos mediante
RAG**

1. Contexto

Una entidad del ámbito deportivo desea construir un sistema capaz de monitorizar eventos de un partido en tiempo real y generar automáticamente un informe útil para analistas, cuerpo técnico y personal de comunicación.

El sistema debe ser capaz de:

- recibir eventos deportivos en tiempo real;
- procesarlos de forma distribuida;
- integrar datos adicionales relevantes;
- calcular métricas e indicadores de rendimiento;
- recuperar información contextual desde una base documental;
- generar un informe automático en lenguaje natural;
- coordinar el proceso final mediante un flujo de agentes implementado con LangGraph.

El objetivo de la práctica es desarrollar una solución aplicada en Python que combine tecnologías Big Data, procesamiento en streaming e inteligencia artificial generativa dentro de un caso de uso deportivo realista.

2. Objetivo general

Desarrollar un pipeline completo de analítica deportiva que procese eventos en tiempo real mediante **Kafka** y **Spark Structured Streaming**, y que produzca un informe automático final enriquecido mediante **RAG**, coordinando la recuperación y la generación mediante **LangGraph** y agentes con tools.

3. Caso de uso

Se trabajará sobre el escenario de **análisis en tiempo real de un partido de fútbol**.

Durante el partido se generan eventos como pases, tiros, goles, recuperaciones, pérdidas, faltas o sprints. Estos eventos son publicados en Kafka y consumidos por un pipeline de Spark. A partir de ellos, el sistema calcula métricas agregadas por equipo y jugador.

Una vez obtenidos los resultados analíticos, el sistema genera un informe automático del partido. Para mejorar su calidad, el informe no debe basarse únicamente en estadísticas numéricas, sino también en información recuperada desde una colección documental. Por ejemplo:

- descripciones de jugadores,
- perfiles de equipos,

- información táctica,
- resúmenes históricos,
- contexto sobre estilos de juego,
- documentación técnica sobre métricas deportivas.

La fase final del sistema deberá estar orquestada mediante un flujo de agentes en LangGraph, de forma que la recuperación de contexto y la elaboración del informe se realicen de manera estructurada y modular.

4. Requisitos de la práctica

La práctica deberá desarrollarse en Python e incluir obligatoriamente los siguientes componentes:

4.1. Ingesta de datos en tiempo real con Kafka

El sistema deberá incluir un productor que envíe eventos deportivos a un tópico de Kafka y un consumidor basado en Spark Structured Streaming que procese dichos eventos.

Cada evento deberá incluir, al menos, los siguientes campos:

- event_id
- timestamp
- match_id
- team
- player
- event_type
- zone
- minute
- value

El alumnado podrá partir de un conjunto de datos base o de un generador sintético de eventos.

4.2. Procesamiento distribuido con Spark Structured Streaming

El pipeline deberá consumir los eventos desde Kafka y realizar, al menos, las siguientes tareas:

- lectura del stream;
- definición explícita del esquema;
- validación y limpieza básica de datos;
- transformación de tipos;
- cálculo de métricas agregadas;
- generación de resultados analíticos intermedios y finales.

Se espera que el procesamiento se realice con PySpark, utilizando operaciones adecuadas para datos en streaming.

4.3. Integración de datos heterogéneos

Además del flujo principal de eventos, la solución deberá integrar al menos una fuente adicional de información estática o semiestructurada, por ejemplo:

- información de jugadores;
- alineaciones;
- zonas del campo;
- estadísticas históricas;
- catálogo de métricas;
- documentos descriptivos del contexto deportivo.

Esta integración deberá utilizarse para enriquecer el análisis o la generación del informe final.

4.4. Cálculo de métricas deportivas

El sistema deberá calcular métricas relevantes para el análisis del partido.

Como mínimo, deberán obtenerse indicadores por equipo y por jugador.

Métricas mínimas por equipo:

- número total de eventos;
- tiros;
- goles;
- faltas;
- recuperaciones;
- intensidad por intervalos temporales.
- Métricas mínimas por jugador
- participación total;
- eventos ofensivos;
- eventos defensivos;
- contribución acumulada al juego.

Se valorará que las métricas tengan sentido deportivo y estén correctamente justificadas.

4.5. Persistencia de resultados

Los resultados del pipeline deberán almacenarse de forma estructurada.

Como mínimo, deberán persistirse:

- los datos procesados o depurados;

- las métricas agregadas finales;
- el informe generado.

Se recomienda usar formatos como Parquet, CSV o una base de datos ligera, siempre que la elección esté justificada.

4.6. Optimización y escalabilidad

La práctica deberá incluir un apartado de optimización técnica del pipeline. El alumnado deberá aplicar y justificar al menos tres medidas relacionadas con eficiencia o escalabilidad, por ejemplo:

- filtrado temprano;
- selección de columnas necesarias;
- particionado;
- uso razonado de caché o persistencia;
- elección adecuada del formato de salida;
- control del tamaño de lotes o microbatches;
- simplificación de transformaciones.

No se exige una optimización avanzada de producción, pero sí una reflexión técnica coherente.

4.7. Generación automática de informe

A partir de las métricas calculadas, el sistema deberá generar un informe automático del partido en lenguaje natural.

El informe deberá incluir, como mínimo:

- resumen general del partido;
- equipo o jugador destacado;
- momento o tramo de mayor intensidad;
- interpretación básica del rendimiento observado;
- conclusión final orientada a un stakeholder no técnico.

4.8. Enriquecimiento mediante RAG

La generación del informe deberá apoyarse en un módulo de Retrieval-Augmented Generation (RAG).

Este módulo tendrá como objetivo recuperar información relevante desde una pequeña base documental y utilizarla para enriquecer el informe final.

La colección documental podrá contener, por ejemplo:

- descripciones de jugadores;
- estilos tácticos de equipos;

- resúmenes de encuentros anteriores;
- notas técnicas sobre indicadores de rendimiento;
- explicaciones de conceptos deportivos.

El alumnado deberá implementar un flujo básico de RAG que incluya:

- preparación o indexación de documentos;
- recuperación de contexto relevante ante una consulta;
- uso del contexto recuperado en el prompt final del modelo generativo.

El objetivo no es construir un sistema conversacional complejo, sino demostrar cómo una arquitectura RAG puede aportar contexto útil y verificable al informe generado.

Importante: el informe final deberá distinguir con claridad entre:

- información derivada de los datos del partido procesados en tiempo real;
- información contextual aportada por los documentos recuperados.

4.9. Orquestación con LangGraph y agentes con tools

La fase final de análisis y generación del informe deberá implementarse mediante LangGraph.

No será suficiente con realizar una única llamada directa a un modelo generativo. El alumnado deberá construir un flujo basado en nodos o agentes que coordine, al menos, las siguientes funciones:

- consulta de métricas generadas por el pipeline;
- recuperación de documentos relevantes del módulo RAG;
- elaboración del informe final a partir de la información disponible.

Para ello, deberán emplearse agentes con tools. Como mínimo, la solución deberá incluir:

- Tool 1. Consulta de métricas del partido
 - o Una herramienta que permita acceder a las métricas ya calculadas por Spark y almacenadas por el sistema.
 - o Ejemplos de consultas:
 - equipo con mayor número de recuperaciones;
 - jugador con mayor participación;
 - tramo temporal con más intensidad;
 - número total de tiros o faltas.
- Tool 2. Recuperación documental
 - o Una herramienta conectada al índice o base documental del módulo RAG para recuperar contexto relevante.
 - o Ejemplos:
 - descripción táctica de un equipo;

- perfil de un jugador;
- explicación de una métrica concreta;
- contexto histórico relacionado con el partido analizado.

También podrá plantearse una arquitectura con dos agentes especializados y un nodo final de consolidación.

Lo importante es que:

- exista una orquestación real en LangGraph;
- los agentes utilicen tools de forma explícita;
- el uso de agentes aporte estructura y trazabilidad al proceso;
- el resultado final esté conectado de forma coherente con el pipeline de Big Data.

No se exige un sistema multiagente excesivamente complejo, pero sí una implementación clara, modular y funcional.

5. Salida final del sistema

La práctica deberá producir una salida final comprensible y bien presentada. Esta salida podrá adoptar una de las siguientes formas:

- notebook con tablas, gráficos e informe final;
- aplicación sencilla en Streamlit;
- informe HTML o PDF generado automáticamente.

La salida deberá mostrar de forma clara:

- métricas obtenidas;
- elementos más relevantes del análisis;
- contexto recuperado desde la base documental;
- informe narrativo final generado mediante el flujo en LangGraph.

6. Tecnologías mínimas

La práctica deberá desarrollarse utilizando, como mínimo:

- Python
- Apache Kafka
- PySpark / Spark Structured Streaming
- LangGraph

Además, será necesario incorporar alguna solución para:

- indexación y recuperación documental;
- embeddings o búsqueda semántica;
- generación de texto mediante un modelo generativo.

El alumnado podrá elegir la tecnología concreta, siempre que quede correctamente explicada.

7. Entregables

7.1. Código fuente

El alumnado deberá entregar el código completo, organizado de forma clara.

La estructura recomendada es la siguiente:

```
final_project/
├── data/
│   ├── static/
│   └── docs/
├── output/
│   ├── processed/
│   ├── aggregates/
│   └── reports/
├── src/
│   ├── kafka_producer.py
│   ├── streaming_pipeline.py
│   ├── metrics.py
│   ├── rag_pipeline.py
│   ├── tools.py
│   ├── graph_workflow.py
│   ├── report_generator.py
│   └── app.py
├── requirements.txt
├── README.md
└── memoria.pdf
```

7.2. Memoria técnica

La memoria deberá describir de forma clara la solución implementada y deberá incluir, al menos:

- objetivo del proyecto;
- arquitectura general;
- descripción de los datos;

- uso de Kafka;
- procesamiento con Spark;
- métricas calculadas;
- diseño del módulo RAG;
- diseño del flujo en LangGraph;
- tools implementadas y función de cada una;
- generación del informe;
- decisiones de optimización;
- limitaciones del sistema.

7.3. Presentación oral

La práctica se defenderá oralmente en la fecha indicada oficialmente en la convocatoria de exámenes.

La presentación deberá mostrar:

- el problema abordado;
- la arquitectura desarrollada;
- una demostración del flujo completo;
- el papel de Kafka, Spark, RAG y LangGraph;
- los principales resultados;
- las decisiones técnicas adoptadas.

8. Criterios generales de valoración

En la evaluación se tendrá en cuenta:

- el correcto funcionamiento del pipeline completo;
- la integración adecuada de Kafka y Spark;
- la calidad del análisis deportivo realizado;
- la coherencia del módulo RAG dentro de la solución;
- la correcta utilización de LangGraph y agentes con tools;
- la calidad del informe automático generado;
- la justificación técnica de las decisiones adoptadas;

- la claridad de la memoria y de la presentación oral.

9. Consideraciones importantes

- La práctica debe funcionar de extremo a extremo.
- No se valorará positivamente incluir tecnologías de forma superficial.
- Kafka debe utilizarse como mecanismo real de ingesta en tiempo real.
- El RAG debe aportar contexto útil al informe final.
- LangGraph debe emplearse para coordinar de forma explícita la fase final del sistema.
- Los agentes deberán utilizar tools reales conectadas con los datos y documentos del proyecto.
- La solución debe estar orientada a un caso de uso deportivo realista.
- El código deberá ser legible, modular y reproducible.

10. Integridad académica

Todo el trabajo entregado debe ser original o estar correctamente referenciado. El uso de asistentes de IA deberá limitarse a tareas de apoyo y nunca podrá sustituir la comprensión técnica del proyecto ni la autoría efectiva del código y de la memoria.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Apache Kafka

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

TEMA 2 – Apache Kafka

Se proponen diferentes ejercicios de ingesta y procesamiento de datos en streaming aplicados a un escenario de analítica deportiva. Para ello se utilizará **Apache Kafka** como sistema de mensajería distribuida. Se desarrollará un producer que publique mediciones de telemetría deportiva en un topic y distintos consumers que las consuman para **monitorizar condiciones de rendimiento y fisiología en tiempo real**.

En particular, se trabajará con un conjunto de datos de telemetría recogidos a través de sensores durante **3 partidos de fútbol para 15 jugadores**. Las mediciones incluyen parámetros de posición, velocidad, aceleración y frecuencia cardíaca. La generación en tiempo real de las mediciones se simulará a partir del script de Python *producer.py* y el archivo CSV *sensores_deportivos_fatiga.csv* proporcionados.

Ejercicio 1. Publicación de eventos con Kafka

Se pide implementar un **producer** que publique eventos con estas mediciones de telemetría en un topic de Kafka.

Apartado 1.1. Diseño del topic y del esquema del mensaje

Define el nombre del topic donde se publicarán los datos en streaming. Se recomienda seguir una convención de nombres jerárquica y con versionado de esquema.

A continuación, diseña el esquema de mensajes que se enviarán al topic. Debe contener los siguientes elementos:

- **Key:** debe elegirse una clave que permita preservar el orden de los eventos dentro de las particiones de un topic.
- **Value:** debe incluir todos los campos de medición y otros metadatos del mensaje relevantes. Se recomienda utilizar JSON e incorporar la versión del esquema de mensajes y un identificador único del evento.
- **Timestamp:** debe utilizarse el tiempo de evento asociado a cada medición y no el instante en el que se envía el mensaje.

Apartado 1.2. Implementación del producer

Modifica el script *producer.py* para que publique eventos en el topic de Kafka. Para ello, implementa las modificaciones necesarias en las líneas indicadas en el archivo. El identificador del evento puede generarse a partir de un [uuid](#).

Ejercicio 2. Consumo de eventos con Kafka

Se pide implementar distintos consumers de Kafka que se suscriban al topic definido en el ejercicio anterior y procesen en tiempo real los eventos publicados.

Apartado 2.1. Lectura de eventos

Implementa un consumer en Python que se suscriba al topic definido y lea los mensajes de forma continua. Para ello, se debe deserializar el contenido JSON de los mensajes enviados a los topics y mostrar por pantalla cada evento recibido.

Apartado 2.2. Alertas de frecuencia cardiaca

Implementa un segundo consumer que se suscriba al mismo topic e imprima únicamente por pantalla aquellos eventos que superen el umbral de frecuencia cardiaca de 180 bpm con una velocidad menor a 10 km/h.

Entregables

Se deberá entregar un único archivo comprimido en formato .zip en el Aula Virtual con los siguientes archivos Python:

- **producer.py**: Script del producer modificado para publicar eventos en el topic definido.
- **consumer_print.py**: Consumer que se suscribe al topic y muestra por pantalla cada evento recibido.
- **consumer_alert.py**: Consumer que se suscribe al topic e imprime únicamente los eventos que superen o igualen el umbral de frecuencia cardiaca configurado.

Debe incluirse una **memoria explicativa** en la que se describa el diseño del topic y el esquema de los mensajes empleados. Se deben especificar los requisitos del entorno y una guía clara de instalación y ejecución, indicando cómo lanzar el producer y los consumers.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Spark Structured Streaming

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

TEMA 2 – Spark Structured Streaming

Se proponen diferentes ejercicios de ingesta y procesamiento de datos en streaming aplicados a un escenario de analítica deportiva. Para ello se utilizará **Apache Spark Structured Streaming** para consumir y procesar en tiempo real los **eventos publicados** previamente en **Apache Kafka**. Se trabajará sobre el topic definido en la entrega anterior, asumiendo que el producer de Kafka ya está implementado. De este modo, el flujo de datos se genera en tiempo real a partir del **producer implementado en la entrega anterior**.

Para el desarrollo de la práctica se proporciona un entorno con Kafka y un Jupyter Notebook con PySpark mediante el fichero *docker-compose.yml*, así como un notebook base *spark_streaming.ipynb* con celdas marcadas para completar. El entorno levanta los servicios en local y monta el directorio de trabajo en el contenedor de Jupyter. Para simplificar la conectividad entre servicios, los contenedores comparten la red de la máquina anfitriona.

Ejercicio 1. Ingesta de eventos desde Kafka

Se pide implementar en PySpark un **stream de lectura desde Kafka** que consuma los eventos publicados en la práctica anterior.

La creación de la sesión de Spark necesaria para trabajar con Structured Streaming y el conector de Kafka ya se proporcionan dentro del notebook adjunto.

Apartado 1.1. Definición del esquema del mensaje

Define el esquema de Spark correspondiente al contenido de los mensajes publicados en Kafka. El **esquema debe incluir todos los campos** de metadatos y de medición de los eventos consumidos.

Apartado 1.2. Lectura del stream de eventos

Completa la constante `KAFKA_TOPIC` con el **nombre del topic definido** en la entrega anterior de Apache Kafka. Se debe reutilizar exactamente el mismo topic para consumir los eventos ya publicados.

Ejercicio 2. Procesamiento en streaming

Se pide transformar los mensajes leídos desde Kafka y **calcular métricas agregadas por jugador** mediante ventanas temporales en streaming. Se requiere usar Spark Structured Streaming.

Apartado 2.1. Parseo del mensaje

Implementa la transformación del stream para deserializar el contenido de los mensajes recibidos desde Kafka utilizando el esquema definido en el ejercicio anterior.

Debe tenerse en cuenta que el contenido de los mensajes en Kafka se recibe en bytes, por lo que será necesario **convertir su contenido a texto antes de aplicar el parseo del mensaje**. Para ello, puede utilizarse una transformación como `selectExpr('CAST(value AS STRING) AS json_str')`.

Apartado 2.2. Agregaciones por jugador en ventanas temporales

Implementa un flujo de agregación en streaming por jugador utilizando ventanas temporales deslizantes y watermark. La agregación debe agrupar, como mínimo, por **ventana temporal, partido y jugador**.

Se debe usar un **watermark de 2 minutos** para tolerar la llegada de eventos tardíos, así como **ventanas deslizantes de 1 minuto con deslizamiento de 30 segundos**.

Tras ello, se deben calcular las siguientes métricas por grupo, redondeando a dos decimales las métricas necesarias:

- Número de eventos
- Frecuencia cardiaca media
- Desviación estándar de la frecuencia cardiaca
- Frecuencia cardiaca máxima

Apartado 2.3. Salida del stream por consola

Completa la escritura del stream para mostrar los resultados en streaming utilizando el **modo update**. Para esta práctica no es necesario configurar checkpointing, ya que la salida se debe realizar **por consola** con fines de depuración y validación. La salida deberá permanecer activa mientras se estén publicando eventos desde Kafka.

Para facilitar la visualización de los resultados completos en consola, se recomienda desactivar el truncado de columnas mediante `option('truncate', 'false')` y consultar los logs del contenedor de Jupyter con el comando: `docker compose logs -f jupyter`.

Entregables

Se deberá entregar un único archivo comprimido en formato .zip en el Aula Virtual con los siguientes archivos:

- **spark_streaming.ipynb**: Notebook completado con todas las celdas / TODOS resueltas.

Debe incluirse una **memoria explicativa** en la que se describan el esquema del mensaje utilizado para deserializar el JSON y las transformaciones aplicadas sobre el stream. Asimismo, se deberá explicar la consulta en Spark Structured Streaming implementada. Se deben especificar los requisitos del entorno y una guía clara de instalación y ejecución. Esta memoria podrá entregarse como un documento único junto con la práctica anterior de Kafka, o bien como una memoria independiente para esta continuación.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Prompt Engineering y Post-training

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

Prompt Engineering aplicado al análisis deportivo

Objetivos

- Diseñar prompts profesionales y robustos.
- Controlar el comportamiento del modelo.
- Evitar alucinaciones.
- Generar salidas estructuradas reutilizables.

Ejercicio 1 — De prompt amateur a prompt profesional

Prompt original: «Analiza el rendimiento del jugador.»

- Identifique al menos 4 problemas del prompt.
- Rediseñe un prompt profesional incluyendo rol, contexto, restricciones y formato de salida.
- Explique por qué su versión mejora la fiabilidad.

Ejercicio 2 — Control de alucinaciones con datos incompletos

Datos: minutes=410, goals=2, assists=1, xG=1.3, shots_p90=2.1

- Diseñe un prompt que prohíba inventar datos e indique cómo actuar si faltan métricas.
- Solicite una salida estructurada.
- Explique cómo su prompt reduce el riesgo de alucinación.

Ejercicio 3 — Diseño de salida estructurada

- Diseñe un prompt que genere JSON con resumen técnico, fortalezas, debilidades, alertas y métricas utilizadas.
- Explique por qué este formato es preferible al texto libre.

Ejercicio 4 — Zero-shot vs Few-shot

Evento: Recuperación en campo propio, pase largo y finalización en 7 segundos.

- Diseñe un prompt zero-shot para clasificar el evento.
- Diseñe un prompt few-shot con ejemplos.
- Compare consistencia, claridad y formato.

Ejercicio 5 — Chain-of-Thought en análisis deportivo

Pregunta: ¿Por qué ha disminuido el rendimiento ofensivo?

- Diseñe un prompt normal.
- Diseñe un prompt que fuerce razonamiento paso a paso.
- Compare claridad, utilidad y riesgos.

Rúbrica de evaluación

- Excelente (9-10): Prompt completo, preciso, estructurado y bien justificado.
- Notable (7-8): Prompt correcto con pequeñas mejoras posibles.
- Aprobado (5-6): Cumple requisitos mínimos pero con ambigüedades.
- Insuficiente (<5): Prompt incompleto, ambiguo o sin justificación.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Retrieval-Augmented Generation

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

RAG aplicado al análisis deportivo

Se plantean diferentes ejercicios basados en técnicas simples de RAG.

Objetivo: comprender RAG a nivel de arquitectura y construir un RAG funcional en Python con LangChain.

Requisitos y entorno

Los ejercicios prácticos están pensados para ejecutarse en un entorno Python 3.10+.

Instalación mínima (elige UNA opción de LLM):

Opción A: OpenAI (requiere OPENAI_API_KEY)

```
pip install -U langchain langchain-core langchain-community langchain-openai langchain-chroma chromadb tiktoken
```

Opción B: Ollama local (sin API key, requiere Ollama instalado)

```
pip install -U langchain langchain-core langchain-community langchain-ollama langchain-chroma chromadb tiktoken
```

Embeddings locales recomendados (sin token HF)

```
pip install -U langchain-huggingface sentence-transformers
```

Se permite usar cualquier corpus pequeño (p.ej., 2–5 documentos TXT/PDF), pero se recomienda preparar un directorio ./data con 3–6 ficheros (se proporcionan algunos de base sobre asuntos deportivos pero se pueden ampliar con los que se deseen).

Bloque A — Teoría (sin código)

Ejercicio A1 — RAG vs Fine-Tuning vs Prompting

Para un asistente sobre normativa deportiva interna (documentos privados que cambian cada semana): (1) ¿Cuándo elegirías RAG? (2) ¿Cuándo elegirías fine-tuning? (3) ¿Qué papel juega el prompt engineering en ambos? Da una respuesta razonada (máx. 12 líneas).

Ejercicio A2 — Evaluación de RAG

Propón un plan de evaluación con al menos 3 métricas o criterios: (i) métricas de retrieval (p.ej., Recall@k), (ii) métricas de respuesta (p.ej., exactitud/fidelidad), (iii) métricas de robustez (p.ej., sensibilidad a perturbaciones). Indica cómo obtendrías datos de evaluación (conjunto de preguntas, gold answers, etc.).

Bloque B — Práctica (Python + LangChain)

Ejercicio B1 — RAG mínimo funcional (TXT → Chroma → QA)

Construye un RAG mínimo que:

- 1) Cargue varios .txt desde ./data
- 2) Divida en chunks con RecursiveCharacterTextSplitter
- 3) Genere embeddings (HuggingFaceEmbeddings recomendado)
- 4) Indexe en Chroma (persistente)
- 5) Responda una pregunta con create_retrieval_chain
- 6) Realiza prompts que se creen resúmenes generales y por temáticas del RAG.

Requisitos:

- El LLM debe responder SOLO con base en el contexto recuperado.
- La respuesta debe incluir una sección 'Fuentes' con los metadatos (p.ej., filename) de los chunks usados.

Entrega: fragmento de código + breve análisis (10–12 líneas).

Entrega

Entrega un repositorio o un zip con:

- Notebook o script para el ejercicio práctico.
- Un breve README con instrucciones de ejecución.
- Respuestas a los ejercicios teóricos.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de sistemas basados en agentes

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

Agentes aplicados al análisis deportivo

Se plantean diferentes ejercicios basados en agentes LLM + Tools + Skills (framework práctico: LangGraph).

Objetivo: diseñar flujos agénticos controlados e integrar herramientas externas.

1) Bloque teórico (sin código)

Ejercicio T1 — Tool vs Skill vs Agent (10–12 líneas)

Define con tus palabras (sin copiar definiciones) las diferencias entre tool, skill y agente LLM. Incluye un ejemplo de cada uno en el contexto de un asistente universitario (normativa, tutorías, evaluación, etc.).

Ejercicio T2 — Diseño de control: evitar loops y alucinaciones (12–15 líneas)

Explica por qué los agentes pueden entrar en bucles o tomar acciones incorrectas. Propón mecanismos concretos de control.

Ejercicio T3 — RAG como tool dentro de un agente (8–10 líneas)

Describe cómo integrarías un sistema RAG como tool dentro de un agente. ¿Qué decide el agente? ¿Qué devuelve la tool? ¿Qué riesgos aparecen (p. ej., retrieval irrelevante) y cómo los mitigarías?

2) Bloque práctico con LangGraph

Entrega recomendada: un repositorio con scripts o notebooks por ejercicio, y un README con instrucciones. En cada ejercicio, incluye un ejemplo de ejecución y una breve discusión (5–10 líneas) de lo observado.

Ejercicio P1 — Flujo sencillo (grafo lineal con estado)

Implementa un flujo en LangGraph con 3 nodos y estado compartido:

- 1) Nodo "preprocess": normaliza la pregunta (p. ej., minúsculas, recorta espacios, detecta idioma simple).

- 2) Nodo "llm_answer": el LLM responde con un formato fijo (título + 3 bullets).

- 3) Nodo "postprocess": añade una sección final "Calidad" con dos checks: "¿hay respuesta?" y "¿es clara?".

Requisitos:

- Define un estado TypedDict con campos: question (str), normalized_question (str), answer (str).
- El grafo debe compilar y ejecutarse con `app.invoke({...})`.



- Añade logging simple para ver el paso por nodos.

Pista: este ejercicio busca que entiendas el concepto de state y cómo LangGraph estructura un pipeline, sin herramientas todavía.

Ejercicio P2 (opcional) — Agente con tool RAG (conocimiento externo)

Integra una tool tipo RAG como nodo de LangGraph (puedes reutilizar tu vector store de ejercicios de RAG). El agente debe:

- 1) Detectar si la pregunta requiere conocimiento documental.
- 2) Si sí, llamar a "rag_tool" (retrieval) y luego responder con groundedness.
- 3) Si no, responder directamente.

Requisitos:

- El estado debe incluir: question, decision, retrieved_context, sources, answer.
- La respuesta debe terminar con "Fuentes:" listando metadatos.
- Debe existir una política de abstención: si no hay contexto suficiente, decir "No puedo responder con la información disponible."

3) Entregables y criterios de evaluación

Entregables mínimos:

- Código ejecutable (scripts o notebooks) por ejercicio práctico.
- README con instalación y ejecución
- Capturas o logs de ejecuciones representativas
- Respuestas a T1–T3

Anexo — Esqueleto mínimo de LangGraph (referencia)

```
from typing import TypedDict
from langgraph.graph import StateGraph, END
```

```
class State(TypedDict):
    question: str
    answer: str
```

```
def node_a(state: State) -> State:
    # ...
    return state
```

```
graph = StateGraph(State)
graph.add_node("a", node_a)
graph.set_entry_point("a")
graph.add_edge("a", END)
```

```
app = graph.compile()  
print(app.invoke({"question": "Hola", "answer": ""}))
```

MATERIAL DOCENTE EN ABIERTO

Categoría 6 · Programas de ordenador

Big Data Processing II

Alberto Fernández Isabel · Natalia Madrueño Sierro · Rubén Rodríguez Fernández

El paquete de software reúne notebooks, scripts Python, un productor Kafka, un entorno Docker/Spark, datos simulados y ejemplos de RAG, agentes y Text-to-SQL con Spark y LangGraph. Se organiza por temas e incluye dependencias e instrucciones para su uso autónomo.

Licencia y reutilización

El código se distribuye bajo licencia MIT. Los documentos explicativos originales se distribuyen bajo CC BY-SA 4.0. No se incluyen credenciales; los ficheros .env se excluyen y solo se conserva .env.example cuando es necesario.

Inventario funcional

Bloque	Recursos principales
Tema 1	Notebooks de streaming y solución
Tema 2	Productor Kafka, datos simulados, Docker y notebook Spark
Tema 4	Ejemplos RAG y sistemas basados en agentes
Tema 5	Agente Text-to-SQL con Spark y LangGraph

Repositorio y preservación

Repositorio público del código: <https://github.com/Fiutten/BDPII-Open-Subject-Code-URJC>

SWHID: pendiente de obtener y verificar después de archivar la versión pública definitiva. Este campo debe sustituirse por el identificador exacto antes del depósito.

Declaración sobre apoyo generativo

Las herramientas generativas se emplearon únicamente como apoyo editorial y técnico para organización, documentación y comprobación de coherencia. El equipo autor revisó y validó la versión final y asume plena responsabilidad sobre ella.